

Ders: 01 Limits

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\01_limits

Bölüm 1: README

Lesson 01: Epsilon-Delta Definition of Limits

Mathematical Context

The formal definition of a limit is the foundation of Calculus. Instead of saying "x gets close to c", we use the **ϵ - δ** (Epsilon-Delta) formulation to prove that the function $f(x)$ stays within a specific range.

Definition:

$\forall \epsilon > 0, \exists \delta > 0 \text{ such that } 0 < |x - c| < \delta \implies |f(x) - L| < \epsilon$

Computational Approach

In this lesson, we use **Python (NumPy & Matplotlib)** to visualize this "challenge-response" logic. We define a function with a hole (discontinuity) at $x=1$ and show how choosing a δ window ensures the function stays within the ϵ error margin.

Visualization

![[Limit Visualization](../../assets/01_limit_epsilon.png)]

Key Features of the Script:

- * Handles indeterminate forms like $0/0$.
- * Dynamically highlights the ϵ (y-axis) and δ (x-axis) neighborhoods.
- * Demonstrates why the limit exists even if the point itself is undefined.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt

#  $f(x) = (x^2 - 1) / (x - 1) \rightarrow x=1$ 'de delik var ama limit 2
x = np.linspace(0.5, 1.5, 100)
y = (x**2 - 1) / (x - 1)

plt.figure(figsize=(10, 6))
plt.plot(x, y, label='f(x) = (x^2-1)/(x-1)', color='blue')
plt.scatter([1], [2], color='red', s=100, facecolors='none', label='Hole at L=2')

# Epsilon-Delta visualization
plt.axhspan(1.9, 2.1, alpha=0.1, color='green', label='Epsilon (y-range)')
plt.axvspan(0.95, 1.05, alpha=0.1, color='orange', label='Delta (x-range)')

plt.title("Lesson 01: Epsilon-Delta Definition of Limits", fontsize=14)
plt.legend()
plt.savefig('assets/01_limit_epsilon.png')
plt.show()
```

Ders: 02 Continuity

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\02_continuity

Bölüm 1: README

Lesson 02: Continuity & Discontinuity Analysis

Mathematical Context

A function is **continuous** if you can draw its graph without lifting your pen. Mathematically, the limit must equal the function value: $\lim_{x \rightarrow c} f(x) = f(c)$.

In this lesson, we visualize **Jump Discontinuity**, which occurs when the left-hand and right-hand limits exist but are not equal.

Computational Approach

We use piecewise logic to create a "break" in the data, simulating real-world scenarios where systems experience sudden shifts or state changes.

Visualization

![Continuity Analysis](../../assets/02_continuity.png)

Files in this folder:

* `continuity\_analysis.py`: Standalone Python script.

* `continuity\_analysis.ipynb`: Interactive Jupyter Notebook.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_continuity_analysis():
    # Parçalı fonksiyon tanımı (Piecewise)
    x1 = np.linspace(-2, 0.95, 100)
    y1 = x1**2
    x2 = np.linspace(1.05, 3, 100)
    y2 = x2 + 2

    plt.figure(figsize=(10, 6))
    plt.plot(x1, y1, 'b', lw=2.5, label='Left Limit Segment')
    plt.plot(x2, y2, 'r', lw=2.5, label='Right Limit Segment')

    # Boş ve dolu noktalar (Süreksizliği göstermek için)
    plt.scatter([1], [1], edgecolors='b', facecolors='none', s=100, zorder=5)
    plt.scatter([1], [3], color='r', s=100, zorder=5)

    plt.title("Lesson 02: Visualizing Jump Discontinuity", fontsize=14)
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.legend()

    if not os.path.exists('../assets'):
        os.makedirs('../assets')
    plt.savefig('../assets/02_continuity.png')
    plt.show()

if __name__ == "__main__":
    run_continuity_analysis()
```

Ders: 03 Derivative

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\03_derivative

Bölüm 1: README

Lesson 03: The Geometry of Derivatives

Mathematical Context

The derivative $f'(x)$ represents the **instantaneous rate of change**. Geometrically, it is the slope of the tangent line to the curve at a given point.

Computational Approach

Using the power rule ($\frac{d}{dx}[x^n] = nx^{n-1}$), we calculate the slope at a specific point and plot the linear equation $y - y_1 = m(x - x_1)$ to show the perfect "touch" of the tangent line.

Visualization

![Derivative Analysis](../../assets/03_derivative_slope.png)

Files in this folder:

* `derivative\_analysis.py`: Standalone Python script.

* `derivative\_analysis.ipynb`: Interactive Jupyter Notebook.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_derivative_analysis():
    f = lambda x: x**2
    df = lambda x: 2*x # f(x)'in türevi

    x = np.linspace(-2, 2, 100)
    p = 1.0 # Teğet noktası

    # Teğet doğrusu denklemi
    tangent_x = np.linspace(0.5, 1.5, 20)
    tangent_y = df(p)*(tangent_x - p) + f(p)

    plt.figure(figsize=(10, 6))
    plt.plot(x, f(x), 'k', lw=2, label='f(x) = x2')
    plt.plot(tangent_x, tangent_y, 'r--', lw=2, label=f'Tangent at x={p} (Slope={df(p)})')
    plt.scatter([p], [f(p)], color='red', s=80, zorder=5)

    plt.title("Lesson 03: Geometric Interpretation of Derivatives", fontsize=14)
    plt.grid(True, alpha=0.3)
    plt.legend()

    if not os.path.exists('../assets'):
        os.makedirs('../assets')
    plt.savefig('../assets/03_derivative_slope.png')
    plt.show()

if __name__ == "__main__":
    run_derivative_analysis()
```

Ders: 04 Chain Rule

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\04_chain_rule

Bölüm 1: README

Lesson 04: Chain Rule in Action

Mathematical Context

The **Chain Rule** is the formula for computing the derivative of the composition of two or more functions:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

It allows us to differentiate complex functions like $f(g(x))$ by breaking them down into simpler "inner" and "outer" parts.

Computational Approach

We visualize a composite function (e.g., $\sin(x^2)$) and compare it with its derivative calculated via the Chain Rule ($2x \cdot \cos(x^2)$). This dual-plot helps in understanding how the inner function's rate of change scales the outer function.

Visualization

![Chain Rule Analysis](../../assets/04_chain_rule.png)

Files in this folder:

* `chain_rule_analysis.py`: Standalone Python script.

* `chain_rule_analysis.ipynb`: Interactive Jupyter Notebook.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_chain_rule():
    x = np.linspace(0, 3, 400)
    # Composite function:  $f(g(x)) = \sin(x^2)$ 
    y = np.sin(x**2)
    # Derivative via Chain Rule:  $f'(g(x)) \cdot g'(x) = \cos(x^2) \cdot 2x$ 
    dy = 2*x * np.cos(x**2)

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8), sharex=True)

    ax1.plot(x, y, color='#2ecc71', lw=2, label='$f(x) = \sin(x^2)$')
    ax1.set_title("Composite Function", fontsize=12)
    ax1.legend()
    ax1.grid(alpha=0.3)

    ax2.plot(x, dy, color='#e67e22', lw=2, label='$f'(x) = 2x \cos(x^2)$')
    ax2.set_title("Derivative (Chain Rule Result)", fontsize=12)
    ax2.legend()
    ax2.grid(alpha=0.3)

    plt.tight_layout()

    if not os.path.exists('../assets'):
        os.makedirs('../assets')
    plt.savefig('../assets/04_chain_rule.png')
    plt.show()

if __name__ == "__main__":
    run_chain_rule()
```

Ders: 05 Mvt

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\05_mvt

Bölüm 1: README

Lesson 05: Mean Value Theorem (MVT)

Mathematical Context

The **Mean Value Theorem** states that for a continuous and differentiable function on $[a, b]$, there exists at least one point c in (a, b) such that the instantaneous rate of change (derivative) equals the average rate of change:

$$f'(c) = \frac{f(b) - f(a)}{b - a}$$

Computational Approach

We plot a curve and the **secant line** connecting the endpoints. Then, the script algorithmically finds the point c where the **tangent line** is parallel to the secant, providing a visual proof of the theorem.

Visualization

![MVT Analysis](../../assets/05_mvt.png)

Files in this folder:

* `mvt\_analysis.py` : Standalone Python script.

* `mvt\_analysis.ipynb` : Interactive Jupyter Notebook.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_mvt_analysis():
    f = lambda x: 0.5*x**3 - 2*x + 2
    df = lambda x: 1.5*x**2 - 2

    a, b = -1, 2
    x = np.linspace(-1.5, 2.5, 100)

    # Average slope (Secant)
    m_secant = (f(b) - f(a)) / (b - a)

    # Find c where f'(c) = m_secant using quadratic formula for this specific df
    #  $1.5c^2 - 2 = m\_secant \Rightarrow c = \sqrt{(m\_secant + 2) / 1.5}$ 
    c = np.sqrt((m_secant + 2) / 1.5) # Taking the positive root in [a,b]

    plt.figure(figsize=(10, 6))
    plt.plot(x, f(x), 'k', lw=2, label='f(x)')

    # Plot Secant Line
    plt.plot([a, b], [f(a), f(b)], 'b--', marker='o', label='Secant (Avg Change)')

    # Plot Tangent at c
    tangent_x = np.linspace(c-0.5, c+0.5, 20)
    tangent_y = m_secant*(tangent_x - c) + f(c)
    plt.plot(tangent_x, tangent_y, 'r', lw=2, label=f'Parallel Tangent at c={c:.2f}')
    plt.scatter([c], [f(c)], color='red')

    plt.title("Lesson 05: Visualizing Mean Value Theorem", fontsize=14)
    plt.legend()
    plt.grid(True, alpha=0.3)

    if not os.path.exists('.././../assets'):
        os.makedirs('.././../assets')
    plt.savefig('.././../assets/05_mvt.png')
    plt.show()

if __name__ == "__main__":
    run_mvt_analysis()
```

Ders: 06 Optimization

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\06_optimization

Bölüm 1: README

Lesson 06: Optimization - The Fence Problem

What is Optimization?

Optimization is the process of finding the **maximum** or **minimum** value of a function. In real-world engineering and economics, we use this to minimize costs or maximize efficiency.

Example Problem: The Rectangular Field

A farmer has **100 meters** of fencing and wants to enclose a rectangular field bordering a straight river (no fence needed along the river). What dimensions will maximize the area?

Mathematical Solution:

- Constraint:** $x + 2y = 100$ (where x is the length parallel to the river).
- Objective Function (Area):** $A = x \cdot y$.
- Substitute x :** $A(y) = (100 - 2y) \cdot y = 100y - 2y^2$.
- Derivative:** $A'(y) = 100 - 4y$.
- Critical Point:** $100 - 4y = 0 \implies y = 25\text{m}, x = 50\text{m}$.
- Max Area:** 1250 m^2 .

Computational Approach

The Python script scans the range of possible dimensions, plots the Area Function, and automatically identifies the "Peak" (Global Maximum) using numerical differentiation.

Visualization

![Optimization Demo](../../assets/06_optimization.png)

Files:

- `optimization_solver.py`: Scans and finds the maximum area.
- `optimization_solver.ipynb`: Full step-by-step walkthrough.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_optimization():
    # y: river'a dik olan kenar. Maksimum 50 olabilir (50*2=100)
    y = np.linspace(1, 49, 100)
    area = (100 - 2*y) * y

    # Sayısal olarak maksimumu bulma
    max_idx = np.argmax(area)
    best_y = y[max_idx]
    best_area = area[max_idx]
    best_x = 100 - 2*best_y

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

    # Alan Fonksiyonu
    ax1.plot(y, area, color='teal', lw=2)
    ax1.scatter(best_y, best_area, color='red', s=100, label=f'Max Area: {best_area}m²')
    ax1.set_title("Area Function:  $A(y) = 100y - 2y^2$ ")
    ax1.set_ylabel("Area ( $m^2$ )")
    ax1.legend()

    # Değişim Oranı (Türev)
    derivative = 100 - 4*y
    ax2.plot(y, derivative, color='orange', lw=2)
    ax2.axhline(0, color='black', linestyle='--')
    ax2.scatter(best_y, 0, color='red', s=100, label='Derivative = 0')
    ax2.set_title("Rate of Change:  $A'(y) = 100 - 4y$ ")
    ax2.set_xlabel("Side Length y (meters)")
    ax2.legend()

    plt.tight_layout()
    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/06_optimization.png')
    plt.show()

if __name__ == "__main__":
    run_optimization()
```

Ders: 07 Implicit

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\07_implicit

Bölüm 1: README

Lesson 07: Implicit Differentiation

What is Implicit Differentiation?

Most functions are **explicit** ($y = f(x)$), but some relations are **implicit**, like the equation of a circle: $x^2 + y^2 = 25$. We cannot easily write y as a single function of x . To find the slope (dy/dx), we differentiate both sides with respect to x , treating y as a function $y(x)$.

Example Problem: Slope of a Circle

Find the equation of the tangent line to $x^2 + y^2 = 25$ at the point $(3, 4)$.

Mathematical Solution:

- $\frac{d}{dx}(x^2 + y^2) = \frac{d}{dx}(25)$
- $2x + 2y \cdot \frac{dy}{dx} = 0$
- $\frac{dy}{dx} = -\frac{x}{y}$
- At $(3, 4)$, $m = -3/4$.

Computational Approach

Implicit functions are hard to plot with standard `plt.plot`. We use **Contour Plots** to render the implicit relation and then calculate the vector field or tangent lines at specific points.

Visualization

![Implicit Demo](../../assets/07_implicit.png)

Files:

* `implicit_solver.py`: Plots implicit relations and tangents.

* `implicit_solver.ipynb`: Interactive derivation.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_implicit():
    x = np.linspace(-6, 6, 400)
    y = np.linspace(-6, 6, 400)
    X, Y = np.meshgrid(x, y)

    # Equation:  $x^2 + y^2 = 25$ 
    Z = X**2 + Y**2 - 25

    plt.figure(figsize=(8, 8))
    plt.contour(X, Y, Z, levels=[0], colors='blue', linewidths=2)

    # Tangent at (3, 4)
    px, py = 3, 4
    slope = -px / py
    tx = np.linspace(1, 5, 10)
    ty = slope * (tx - px) + py

    plt.plot(tx, ty, 'r--', lw=2, label=f'Tangent at (3,4) | m = {slope}')
    plt.scatter([px], [py], color='red', s=100)

    plt.title("Lesson 07: Implicit Differentiation (Circle)", fontsize=14)
    plt.gca().set_aspect('equal')
    plt.grid(True, alpha=0.3)
    plt.legend()

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/07_implicit.png')
    plt.show()

if __name__ == "__main__":
    run_implicit()
```

Ders: 08 Taylor Series

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\08_taylor_series

Bölüm 1: README

Lesson 08: Taylor Series Expansion

What is a Taylor Series?

Taylor Series is a way to represent a function (like $\sin(x)$ or e^x) as an **infinite sum of polynomials**. In computing, this is how calculators and computers actually calculate trigonometric values.

General Formula:

$$f(x) \approx \sum_{n=0}^N \frac{f^{(n)}(a)}{n!} (x-a)^n$$

Example: Approximating $\sin(x)$

How many polynomial terms do we need to accurately represent a wave?

- 1st Order: x (Linear)

- 3rd Order: $x - \frac{x^3}{3!}$ (Cubic)

- 5th Order: $x - \frac{x^3}{3!} + \frac{x^5}{5!}$ (Quintic)

Computational Approach

The script calculates these polynomials and overlays them on the actual function. This demonstrates **convergence**: as the order increases, the polynomial "clings" to the function over a wider range.

Visualization

![Taylor Series Demo](../../assets/08_taylor_series.png)

Files:

* `taylor_solver.py`: Generates polynomial approximations.

* `taylor_solver.ipynb`: Detailed step-by-step breakdown.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_taylor():
    x = np.linspace(-2*np.pi, 2*np.pi, 200)
    y_actual = np.sin(x)

    # Taylor terms centered at a=0
    p1 = x
    p3 = x - (x**3)/6
    p5 = x - (x**3)/6 + (x**5)/120
    p7 = x - (x**3)/6 + (x**5)/120 - (x**7)/5040

    plt.figure(figsize=(10, 6))
    plt.plot(x, y_actual, 'k', lw=3, label='Actual $\sin(x)$')
    plt.plot(x, p1, '--', label='1st Order (Linear)')
    plt.plot(x, p3, '--', label='3rd Order')
    plt.plot(x, p5, '--', label='5th Order')
    plt.plot(x, p7, '--', label='7th Order')

    plt.ylim(-2.5, 2.5)
    plt.title("Lesson 08: Taylor Series Convergence", fontsize=14)
    plt.axhline(0, color='black', lw=1)
    plt.axvline(0, color='black', lw=1)
    plt.legend(loc='upper right')
    plt.grid(True, alpha=0.3)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/08_taylor_series.png')
    plt.show()

if __name__ == "__main__":
    run_taylor()
```

Ders: 09 Lhopital

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\09_lhopital

Bölüm 1: README

Lesson 09: L'Hopital's Rule

What is L'Hopital's Rule?

When a limit results in an indeterminate form like $0/0$ or ∞/∞ , L'Hopital's Rule allows us to take the derivatives of the numerator and denominator to find the limit:

$$\lim_{x \rightarrow c} \frac{f(x)}{g(x)} = \lim_{x \rightarrow c} \frac{f'(x)}{g'(x)}$$

Example Problem

Calculate $\lim_{x \rightarrow 0} \frac{\sin(x)}{x}$.

Direct substitution gives $0/0$. Using L'Hopital:

$$\frac{d}{dx} \sin(x) = \cos(x) \text{ and } \frac{d}{dx} x = 1.$$

The limit becomes $\cos(0)/1 = 1$.

Computational Approach

We plot the ratio $f(x)/g(x)$ and show how it smoothly approaches the value predicted by L'Hopital's Rule, even though the computer technically encounters a "Not a Number" (NaN) at the exact point.

Visualization

![Lhopital Demo](../../assets/09_lhopital.png)

Files:

* `lhopital\_analysis.py`: Visualizes the ratio and its limit.

* `lhopital\_analysis.ipynb`: Mathematical proof and simulation.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_lhopital():
    x = np.linspace(-5, 5, 500)
    # Using small epsilon to avoid true division by zero
    y = np.sin(x) / x

    plt.figure(figsize=(10, 6))
    plt.plot(x, y, color='crimson', lw=2, label=r'$f(x) = \frac{\sin(x)}{x}$')

    # Highlight the limit point
    plt.scatter([0], [1], color='black', s=100, zorder=5, label='Limit Value = 1')

    plt.axhline(1, color='gray', linestyle='--', alpha=0.5)
    plt.title("Lesson 09: L'Hopital's Rule Visualization", fontsize=14)
    plt.xlabel("x")
    plt.ylabel("Value")
    plt.legend()
    plt.grid(True, alpha=0.3)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/09_lhopital.png')
    plt.show()

if __name__ == "__main__":
    run_lhopital()
```

Ders: 10 Newton Raphson

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\01_calculus\10_newton_rap

Bölüm 1: README

Lesson 10: Newton-Raphson Method (Root Finding)

What is the Newton-Raphson Method?

The **Newton-Raphson Method** is an iterative algorithm used to find the roots (zeros) of a real-valued function $f(x) = 0$. It is one of the most powerful and well-known numerical methods because of its speed (quadratic convergence).

The Iterative Formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Example Problem: Finding $\sqrt{2}$

To find the square root of 2, we need to find the root of the function:

$$f(x) = x^2 - 2$$

Using an initial guess $x_0 = 2$:

1. $f(2) = 2$ and $f'(2) = 4$

2. $x_1 = 2 - (2/4) = 1.5$

3. $x_2 = 1.5 - (0.25/3) \approx 1.416$

The algorithm converges to $1.414...$ in just a few steps!

Computational Approach

The Python script visualizes the "Tangent Steps". It shows how each tangent line's x-intercept brings us closer to the actual root. We use a `for` loop to track and plot each iteration.

Visualization

![Newton Raphson Demo](../../assets/10_newton_raphson.png)

Files:

* `newton_solver.py`: Iterative root finder with step-by-step plotting.

* `newton_solver.ipynb`: Full derivation and convergence analysis.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_newton_raphson():
    # Function f(x) = x^2 - 2 (Looking for sqrt(2))
    f = lambda x: x**2 - 2
    df = lambda x: 2*x

    x_range = np.linspace(0.5, 2.5, 100)
    x_n = 2.0 # Initial guess
    iterations = 3

    plt.figure(figsize=(10, 7))
    plt.plot(x_range, f(x_range), 'k', lw=2, label='$f(x) = x^2 - 2$')
    plt.axhline(0, color='black', lw=1)

    colors = ['red', 'green', 'orange']

    for i in range(iterations):
        y_n = f(x_n)
        slope = df(x_n)

        # Plot tangent line: y = slope*(x - x_n) + y_n
        # X-intercept of tangent is x_next = x_n - y_n/slope
        x_next = x_n - y_n/slope

        t_range = np.linspace(x_next - 0.2, x_n + 0.2, 10)
        plt.plot(t_range, slope*(t_range - x_n) + y_n, '--', color=colors[i],
                 label=f'Iteration {i+1}: $x_{{i+1}} = {x_next:.4f}$')
        plt.scatter([x_n], [y_n], color=colors[i], zorder=5)

        x_n = x_next

    plt.title("Lesson 10: Newton-Raphson Convergence Steps", fontsize=14)
    plt.xlabel("x")
    plt.ylabel("f(x)")
    plt.legend()
    plt.grid(True, alpha=0.3)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/10_newton_raphson.png')
    plt.show()

if __name__ == "__main__":
    run_newton_raphson()
```

Ders: 11 Riemann Sums

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\11_riemann_sums

Bölüm 1: README

Lesson 11: Riemann Sums & Convergence Analysis

The Core Concept

The definite integral of a function is defined as the limit of **Riemann Sums**. We approximate the area under a curve by dividing it into n rectangles. As $n \rightarrow \infty$, the sum of these rectangles converges to the exact integral.

Mathematical Challenge: Left vs. Right vs. Midpoint

Consider the function $f(x) = x^2 + 1$ on the interval $[0, 2]$.

- Exact Area:** $\int_0^2 (x^2 + 1) dx = \left[\frac{x^3}{3} + x \right]_0^2 = \frac{8}{3} + 2 \approx 4.667$.
- Approximation Error:** How fast does the error decrease as we increase the number of rectangles (n)?

Computational Approach

This script performs a **comparative analysis**. It plots the function with n rectangles and simultaneously tracks the error margin. This demonstrates the numerical convergence of the integration process.

Visualization

![Riemann Demo](../assets/11_riemann_sums.png)

Files:

* `riemann\_solver.py`: Dynamic rectangle generator and error tracker.

* `riemann\_solver.ipynb`: Convergence proof with LaTeX.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_riemann_analysis():
    f = lambda x: x**2 + 1
    a, b = 0, 2
    n = 15 # Dikdörtgen sayısı

    x = np.linspace(a, b, 100)
    y = f(x)

    # Riemann noktaları
    x_rect = np.linspace(a, b, n+1)
    y_rect = f(x_rect[:-1]) # Left Riemann Sum
    width = (b - a) / n

    fig, ax = plt.subplots(figsize=(10, 6))

    # Ana Eğri
    ax.plot(x, y, 'r', lw=3, label='$f(x) = x^2 + 1$')

    # Dikdörtgenleri Çizme
    ax.bar(x_rect[:-1], y_rect, width=width, align='edge',
           alpha=0.3, color='teal', edgecolor='black', label=f'Riemann Sum (n={n})')

    # Alan Doldurma (Gerçek Alan)
    ax.fill_between(x, y, alpha=0.1, color='gray')

    # Akademik Notasyon
    riemann_sum = np.sum(y_rect * width)
    ax.set_title(f"Lesson 11: Riemann Sum Convergence\nCalculated Area: {riemann_sum:.4f} | Exact: 4.6667", fontsize=14)
    ax.legend()
    ax.grid(axis='y', alpha=0.3)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/11_riemann_sums.png')
    plt.show()

if __name__ == "__main__":
    run_riemann_analysis()
```

Ders: 12 Accumulation

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\12_accumulation

Bölüm 1: README

```
# Lesson 12: The Integral as an Accumulation Function

### Theoretical Insight
The Fundamental Theorem of Calculus (Part 1) states that the integral is an "accumulation" process. If  $f(t)$  is a rate of change (e.g., velocity), then  $F(x) = \int_a^x f(t) dt$  represents the total accumulated change (e.g., distance).

### Example: Sine Wave Accumulation
What happens to the total area as we move along a Sine wave?
- From  $0$  to  $\pi$ , the area grows (positive).
- From  $\pi$  to  $2\pi$ , the area shrinks (negative accumulation).

### Computational Approach
We create a dual-axis plot (or subplots). The top plot shows the function  $f(t)$ , and the bottom plot shows the "Area-so-far" function  $F(x)$ . This visually proves that the derivative of the accumulation function is the original function:  $F'(x) = f(x)$ .

### Visualization
![[Accumulation Demo](../../assets/12_accumulation.png)]

**Files:**
* `accumulation_analysis.py`: Dual-plot visualizer.
* `accumulation_analysis.ipynb`: The link between rate and total change.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_accumulation():
    x = np.linspace(0, 2*np.pi, 200)
    f = np.sin(x)

    # Accumulation (Integral of sin is -cos)
    # Using np.cumsum for numerical accumulation
    dx = x[1] - x[0]
    accumulation = np.cumsum(f) * dx

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8), sharex=True)

    # Rate Function
    ax1.plot(x, f, color='blue', lw=2, label='Rate $f(t) = \sin(t)$')
    ax1.fill_between(x, f, color='blue', alpha=0.2)
    ax1.axhline(0, color='black', lw=1)
    ax1.set_title("Instantaneous Rate of Change")
    ax1.legend()

    # Accumulation Function
    ax2.plot(x, accumulation, color='darkred', lw=2, label='Accumulated Area $F(x)$')
    ax2.set_title("Total Accumulated Area (The Integral)")
    ax2.legend()

    plt.tight_layout()
    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/12_accumulation.png')
    plt.show()

if __name__ == "__main__":
    run_accumulation()
```

Ders: 13 Numerical Methods

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\13_numerical_n

Bölüm 1: README

```
# Lesson 13: Numerical Integration - Efficiency & Error Analysis

### The Concept
In real-world engineering, many functions cannot be integrated analytically. We use numerical methods. Two of the most common are:
1. **Trapezoidal Rule:** Approximates the area using linear segments (trapezoids).
2. **Simpson's Rule:** Approximates the area using quadratic curves (parabolas).

### Mathematical Challenge: Which is more accurate?
For a curved function like  $f(x) = \frac{1}{1+x^2}$  on  $[0, 1]$ , Simpson's Rule usually converges much faster than the Trapezoidal Rule for the same number of intervals ( $n$ ).

### Computational Approach
The script computes the integral using both methods and plots the "geometry of approximation." We visualize how trapezoids cut the curve versus how parabolas "hug" it.

### Visualization
![[Numerical Integration Demo](../../assets/13_numerical_methods.png)]

**Files:**
* `numerical_integrator.py`: Implementation of both rules from scratch.
* `numerical_integrator.ipynb`: Error analysis and Big-O notation of convergence.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def f(x): return 1 / (1 + x**2)

def run_numerical_comparison():
    a, b = 0, 3
    n = 6 # Düşük n değeri farkı daha iyi gösterir
    x = np.linspace(a, b, 100)
    x_n = np.linspace(a, b, n + 1)
    y_n = f(x_n)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # 1. Trapezoidal Visualization
    ax1.plot(x, f(x), 'k', lw=2)
    ax1.fill_between(x_n, y_n, alpha=0.3, color='orange', edgecolor='black', step=None)
    for i in range(n + 1):
        ax1.vlines(x_n[i], 0, y_n[i], color='gray', linestyle='--', alpha=0.5)
    ax1.set_title(f"Trapezoidal Rule (n={n})", fontsize=12)

    # 2. Simpson's Rule Logic (Parabolic approximation)
    ax2.plot(x, f(x), 'k', lw=2)
    ax2.fill_between(x, f(x), alpha=0.1, color='blue') # Gerçek alan
    # Simpson noktalarını vurgula
    ax2.scatter(x_n, y_n, color='red', zorder=5)
    ax2.set_title(f"Simpson's Rule (Quadratic Fit)", fontsize=12)

    plt.suptitle("Lesson 13: Comparing Numerical Integration Methods", fontsize=14)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/13_numerical_methods.png')
    plt.show()

if __name__ == "__main__":
    run_numerical_comparison()
```

Ders: 14 Area Between Curves

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\14_area_between

Bölüm 1: README

Lesson 14: Area Between Two Curves

The Concept

To find the area between two functions $f(x)$ and $g(x)$ over $[a, b]$, we integrate their difference:

$$A = \int_a^b |f(x) - g(x)| dx$$

Example Problem: The Bounded Region

Find the area enclosed by:

1. $f(x) = \sqrt{x}$

2. $g(x) = x^2$

The curves intersect at $(0,0)$ and $(1,1)$.

Computational Approach

The script automatically finds intersection points (roots of $f(x) - g(x)$) and shades the region between them. This is a common task in mechanical engineering to find the "cross-section" area of components.

Visualization

![Area Between Curves Demo](../../assets/14_area_between_curves.png)

Files:

* `area_solver.py`: Intersection finder and region filler.

* `area_solver.ipynb`: Integral setup and calculation.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_area_between():
    x = np.linspace(0, 1.2, 100)
    f = np.sqrt(x)
    g = x**2

    plt.figure(figsize=(10, 6))
    plt.plot(x, f, 'b', lw=2, label='$f(x) = \sqrt{x}$')
    plt.plot(x, g, 'r', lw=2, label='$g(x) = x^2$')

    # Kesişim bölgesini doldur (0 ile 1 arası)
    x_fill = np.linspace(0, 1, 50)
    plt.fill_between(x_fill, np.sqrt(x_fill), x_fill**2, color='purple', alpha=0.2, label='Enclosed Area')

    plt.title("Lesson 14: Area Between Two Curves", fontsize=14)
    plt.xlabel("x")
    plt.ylabel("y")
    plt.legend()
    plt.grid(True, alpha=0.3)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/14_area_between_curves.png')
    plt.show()

if __name__ == "__main__":
    run_area_between()
```

Ders: 15 Volumes Of Revolution

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\15_volumes_of

Bölüm 1: README

Lesson 15: Volumes of Revolution (The Disk Method)

Theoretical Framework

In integral calculus, a **Volume of Revolution** is a solid figure obtained by rotating a 2D curve around an axis. This is a fundamental concept in mechanical design and fluid dynamics.

The Disk Method Formula:

If a function $f(x)$ is rotated about the x-axis from a to b , the volume V is given by:

$$V = \pi \int_a^b [f(x)]^2 dx$$

Engineering Challenge: Designing a Parabolic Reflecting Cone

Find the volume generated by rotating $y = \sqrt{x}$ around the x-axis from $x=0$ to $x=4$.

1. **Integral Setup:** $V = \pi \int_0^4 (\sqrt{x})^2 dx = \pi \int_0^4 x dx$.

2. **Analytical Solution:** $V = \pi [\frac{x^2}{2}]_0^4 = 8\pi \approx 25.13$ cubic units.

Computational Approach

Using **Matplotlib's 3D toolkit (mplot3d)**, we simulate the rotation by generating a meshgrid in cylindrical coordinates. This allows us to "see" the solid that the integral is measuring.

Visualization

! [Volume Revolution Demo] (../../assets/15_volume_revolution.png)

Files:

* `volume_3d_projector.py`: 3D rendering of the solid of revolution.

* `volume_3d_projector.ipynb`: Derivation of the volume formula with 3D plots.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_volume_rotation():
    x = np.linspace(0, 4, 100)
    y = np.sqrt(x)

    # 3D Dönüş yüzeyini oluşturma
    angle = np.linspace(0, 2*np.pi, 100)
    X, T = np.meshgrid(x, angle)
    Y = np.sqrt(X) * np.cos(T)
    Z = np.sqrt(X) * np.sin(T)

    fig = plt.figure(figsize=(12, 8))
    ax = fig.add_subplot(111, projection='3d')

    # Yüzey çizimi
    surf = ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.7, edgecolor='none')

    # Ana eğriyi vurgula
    ax.plot(x, y, zs=0, zdir='z', color='red', lw=3, label='Original  $y=\sqrt{x}$ ')

    ax.set_title("Lesson 15: 3D Volume of Revolution (Disk Method)", fontsize=15)
    ax.set_xlabel("X (Axis of Rotation)")
    plt.colorbar(surf, ax=ax, shrink=0.5, aspect=10)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/15_volume_revolution.png')
    plt.show()

if __name__ == "__main__":
    run_volume_rotation()
```

Ders: 16 Arc Length

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\16_arc_length

Bölüm 1: README

Lesson 16: Arc Length & Differential Geometry

The Concept

How do we measure the exact length of a "curvy" path? In physics, this is the total distance traveled by a particle. Using the Pythagorean theorem on an infinitesimal scale, we derive the **Arc Length Formula**:

$$L = \int_a^b \sqrt{1 + [f'(x)]^2} dx$$

Example: The Catenary or Sine Path

Calculate the length of the sine wave $f(x) = \sin(x)$ from 0 to π .

- **Derivative:** $f'(x) = \cos(x)$.

- **Integrand:** $\sqrt{1 + \cos^2(x)}$.

Note: This integral has no elementary solution (it is an Elliptic Integral), making numerical methods (Python) essential.

Computational Approach

The script computes the derivative symbolically or numerically, then applies **Simpson's Rule** to find the precise length of the curve. It visualizes the curve and highlights the segments used for measurement.

Visualization

![Arc Length Demo](../../assets/16_arc_length.png)

Files:

* `arc\_length\_solver.py`: Precision length calculator using numerical integration.

* `arc\_length\_solver.ipynb`: Comparison between linear approximation and calculus.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad
import os

def run_arc_length_analysis():
    # f(x) = sin(x)
    f = lambda x: np.sin(x)
    df = lambda x: np.cos(x) # Türevi

    a, b = 0, np.pi
    x = np.linspace(a, b, 100)

    # Yay Uzunluğu İntegrali: sqrt(1 + f'(x)^2)
    integrand = lambda x: np.sqrt(1 + df(x)**2)
    length, error = quad(integrand, a, b)

    plt.figure(figsize=(10, 6))
    plt.plot(x, f(x), 'b-', lw=3, label=f'Curve:  $\sin(x)$ ')

    # Yay uzunluğu illüstrasyonu (Küçük segmentler)
    plt.fill_between(x, f(x), alpha=0.1, color='blue')

    plt.title(f"Lesson 16: Arc Length Analysis\nTotal Calculated Length: {length:.5f}", fontsize=14)
    plt.xlabel("x")
    plt.ylabel("f(x)")
    plt.grid(True, linestyle=':', alpha=0.6)
    plt.legend()

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/16_arc_length.png')
    plt.show()

if __name__ == "__main__":
    run_arc_length_analysis()
```

Ders: 17 Improper Integrals

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\17_improper_in

Bölüm 1: README

Lesson 17: Improper Integrals & Asymptotic Convergence

Theoretical Framework

An **Improper Integral** is an integral where either the interval of integration is infinite (e.g., $[a, \infty)$) or the integrand becomes infinite within the interval. This concept is vital for Probability Theory (Normal Distribution) and Physics (Potential Energy).

The Challenge:

Does the area under a curve that never touches the x-axis remain finite?

$$\int_1^{\infty} \frac{1}{x^p} dx$$

- If $p > 1$, it **converges** (finite area).

- If $p \leq 1$, it **diverges** (infinite area).

Mathematical Case: Gabriel's Horn Paradox

Consider $f(x) = 1/x$. We analyze the area from 1 to A as $A \rightarrow \infty$.

- Analytical: $\lim_{A \rightarrow \infty} [\ln(x)]_1^A = \infty$.

Now compare it with $g(x) = 1/x^2$:

- Analytical: $\lim_{A \rightarrow \infty} [-1/x]_1^A = 1$.

Computational Approach

We use Python to compute these integrals numerically and visualize the "tail" of the distribution. We demonstrate how the area accumulates and stays below a certain threshold for convergent functions.

Visualization

![[Improper Integral Demo](../../assets/17_improper_integral.png)]

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_improper_analysis():
    # p=2 (Convergent) vs p=1 (Divergent)
    x = np.linspace(1, 20, 500)
    y_conv = 1 / x**2
    y_div = 1 / x

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # Convergent Plot
    ax1.plot(x, y_conv, 'g', lw=2, label='$1/x^2$ (Convergent)')
    ax1.fill_between(x, y_conv, color='green', alpha=0.2)
    ax1.set_title("Finite Area in Infinite Domain")
    ax1.legend()

    # Divergent Plot
    ax2.plot(x, y_div, 'r', lw=2, label='$1/x$ (Divergent)')
    ax2.fill_between(x, y_div, color='red', alpha=0.2)
    ax2.set_title("Infinite Area in Infinite Domain")
    ax2.legend()

    plt.suptitle("Lesson 17: Analysis of Improper Integrals", fontsize=16)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/17_improper_integral.png')
    plt.show()

if __name__ == "__main__":
    run_improper_analysis()
```

Ders: 18 Integration By Parts

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\18_integration_

Bölüm 1: README

Lesson 18: Integration by Parts (IBP)

Theoretical Framework

Integration by Parts is the integral counterpart of the product rule for derivatives. It is defined by the formula:

$$\int u \, dv = uv - \int v \, du$$

This is the "Golden Key" for integrating products of algebraic and transcendental functions (e.g., $x \cdot e^x$ or $x \cdot \sin(x)$).

Example Problem: $x \cdot \ln(x)$

Find $\int x \ln(x) \, dx$ on $[1, e]$.

1. Let $u = \ln(x) \implies du = \frac{1}{x} \, dx$

2. Let $dv = x \, dx \implies v = \frac{x^2}{2}$

3. Solution: $\frac{x^2}{2} \ln(x) - \int \frac{x}{2} \, dx = \frac{x^2}{2} \ln(x) - \frac{x^2}{4}$.

Computational Approach

In this lesson, we introduce **Symbolic Computation** using the `sympy` library. This allows the computer to "understand" the algebraic steps of IBP, solve the integral exactly, and then plot the result.

Visualization

![[IBP Demo](../../assets/18_integration_by_parts.png)]

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import sympy as sp
import os

def run_ibp_analysis():
    x_sym = sp.Symbol('x')
    f_sym = x_sym * sp.exp(-x_sym) #  $x e^{-x}$ 

    # Sympy ile sembolik integral çözümü
    integral_sym = sp.integrate(f_sym, x_sym)

    # Grafik için fonksiyonlara dönüştürme
    f_num = sp.lambdify(x_sym, f_sym, 'numpy')
    int_num = sp.lambdify(x_sym, integral_sym, 'numpy')

    x_vals = np.linspace(0, 5, 200)

    plt.figure(figsize=(10, 6))
    plt.plot(x_vals, f_num(x_vals), 'b', lw=2, label='Function:  $x e^{-x}$ ')
    plt.plot(x_vals, int_num(x_vals), 'r--', lw=2, label='Integral (via IBP)')
    plt.fill_between(x_vals, f_num(x_vals), alpha=0.2, color='blue')

    plt.title(f"Lesson 18: Symbolic Integration by Parts\nSolved:  ${sp.latex(integral_sym)}$ ", fontsize=14)
    plt.legend()
    plt.grid(True, alpha=0.3)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/18_integration_by_parts.png')
    plt.show()

if __name__ == "__main__":
    run_ibp_analysis()
```

Ders: 19 Polar Coordinates

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\19_polar_coord

Bölüm 1: README

Lesson 19: Calculus in Polar Coordinates

Theoretical Framework

In Cartesian coordinates, we use rectangles. In **Polar Coordinates**, we use circular sectors to measure area. This is the natural language of circular, spiral, and periodic systems.

Area Formula in Polar Space:

$$A = \int_{\alpha}^{\beta} \frac{1}{2} [r(\theta)]^2 d\theta$$

Mathematical Challenge: The Cardioids and Roses

Find the area enclosed by a **Cardioid** (heart-shaped curve) defined by $r(\theta) = 1 + \cos(\theta)$. This shape is vital in acoustics (microphone pick-up patterns) and antenna theory.

Computational Approach

We transform the polar function into Cartesian (x, y) coordinates for plotting but perform the integration in the θ domain. This shows the bridge between different coordinate systems.

Visualization

![Polar Area Demo](../../assets/19_polar_area.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_polar_analysis():
    theta = np.linspace(0, 2*np.pi, 500)
    # Cardioid:  $r = 1 + \cos(\theta)$ 
    r = 1 + np.cos(theta)

    # Polar to Cartesian conversion for plotting
    x = r * np.cos(theta)
    y = r * np.sin(theta)

    fig = plt.figure(figsize=(12, 6))

    # 1. Standart Polar Plot
    ax1 = fig.add_subplot(121, projection='polar')
    ax1.plot(theta, r, color='magenta', lw=2)
    ax1.fill(theta, r, color='magenta', alpha=0.2)
    ax1.set_title("Polar Representation", va='bottom')

    # 2. Cartesian Space Representation (Daha 'Matematiksel' Bakış)
    ax2 = fig.add_subplot(122)
    ax2.plot(x, y, color='magenta', lw=2)
    ax2.fill(x, y, color='magenta', alpha=0.1)
    ax2.set_aspect('equal')
    ax2.grid(True, alpha=0.3)
    ax2.set_title("Cartesian Mapping ( $x=r\cos\theta$ ,  $y=r\sin\theta$ )")

    plt.suptitle("Lesson 19: Area Analysis in Polar Systems", fontsize=16)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/19_polar_area.png')
    plt.show()

if __name__ == "__main__":
    run_polar_analysis()
```

Ders: 20 Monte Carlo

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\02_integral\20_monte_carlo

Bölüm 1: README

Lesson 20: Monte Carlo Integration & Probabilistic Calculus

Theoretical Framework

What if a function is so complex that traditional integration fails? We use the **Monte Carlo Method**. By generating thousands of random points in a bounding box, we estimate the area based on the ratio of points that "land" under the curve.

The Experiment: Estimating π via Integration

We integrate the upper quadrant of a unit circle: $f(x) = \sqrt{1 - x^2}$ from 0 to 1 .

The exact area is $\pi/4$.

- **Method:** Throw N random darts.

- **Formula:** $\text{Area} \approx \frac{\text{Points Under Curve}}{\text{Total Points}} \times \text{Bounding Box Area}$.

Computational Approach

The script visualizes every single "dart throw" (random point). Points under the curve are colored differently. This demonstrates how **Stochastic Processes** can solve deterministic mathematical problems.

Visualization

![Monte Carlo Demo](../../assets/20_monte_carlo.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_monte_carlo():
    N = 5000 # Toplam nokta sayısı
    x_rand = np.random.uniform(0, 1, N)
    y_rand = np.random.uniform(0, 1, N)

    # Çemberin üst çeyreği:  $x^2 + y^2 \leq 1$ 
    under_curve = x_rand**2 + y_rand**2 <= 1
    pi_estimate = (np.sum(under_curve) / N) * 4

    plt.figure(figsize=(10, 8))

    # Noktaları çizme (Altında kalanlar vs Dışında kalanlar)
    plt.scatter(x_rand[under_curve], y_rand[under_curve], color='cyan', s=1, alpha=0.5, label='Inside Area')
    plt.scatter(x_rand[~under_curve], y_rand[~under_curve], color='tomato', s=1, alpha=0.5, label='Outside Area')

    # Birim çeyrek çember çizgisini ekle
    theta = np.linspace(0, np.pi/2, 100)
    plt.plot(np.cos(theta), np.sin(theta), color='black', lw=3)

    plt.title(f"Lesson 20: Monte Carlo Estimation of  $\pi$ \nEstimated Value: {pi_estimate:.5f} (N={N})", fontsize=14)
    plt.legend(markerscale=10)
    plt.axis('equal')

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/20_monte_carlo.png')
    plt.show()

if __name__ == "__main__":
    run_monte_carlo()
```

Ders: 21 Vector Ops

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\21_vector_ops

Bölüm 1: README

Lesson 21: Linear Span & Orthogonal Projections

Beyond Arrows: The Linear Span

In a 2D Vector Space $\mathbb{R}^2$, a set of vectors doesn't just point somewhere; they "span" a territory. If vectors are linearly independent, they form a **Basis**.

Mathematical Challenge: Projection onto a Subspace

How much of vector $\mathbf{b}$ lies in the direction of vector $\mathbf{a}$?

- **Formula:** $\text{proj}_{\mathbf{a}}(\mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|^2} \mathbf{a}$

- **Application:** This is how noise cancellation works and how we compress data in Signal Processing.

Computational Approach

We don't just plot vectors. We visualize the **Projection**, the **Orthogonal Component** (error vector), and the **Unit Circle** to show how scaling affects the space.

Visualization

![Vector Projections Demo](../../assets/21_vector_ops.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_advanced_vector_ops():
    # Temel Vektörler
    a = np.array([4, 1])
    b = np.array([2, 3])

    # İzdüşüm (Projection) Hesabı:  $(a \cdot b / |a|^2) * a$ 
    proj_b_a = (np.dot(a, b) / np.dot(a, a)) * a
    # Dik bileşen (Orthogonal)
    ortho_b_a = b - proj_b_a

    plt.figure(figsize=(10, 10))

    # Birim Çember (Uzay ölçeğini hissettirmek için)
    t = np.linspace(0, 2*np.pi, 100)
    plt.plot(np.cos(t), np.sin(t), 'k--', alpha=0.2, label='Unit Circle')

    # Quiver (Oklar) - Daha kalın ve belirgin
    origin = [0, 0]
    plt.quiver(*origin, a[0], a[1], color='#2c3e50', angles='xy', scale_units='xy', scale=1, label='Vector A (Base)')
    plt.quiver(*origin, b[0], b[1], color='#e74c3c', angles='xy', scale_units='xy', scale=1, label='Vector B (Target)')
    plt.quiver(*origin, proj_b_a[0], proj_b_a[1], color='#3498db', angles='xy', scale_units='xy', scale=1, label='Projection of B on A')

    # Dik bileşeni (hata vektörü) çiz
    plt.plot([b[0], proj_b_a[0]], [b[1], proj_b_a[1]], 'k:', lw=2, label='Orthogonal Component')

    plt.xlim(-1, 5); plt.ylim(-1, 5)
    plt.axhline(0, color='black', lw=1.5); plt.axvline(0, color='black', lw=1.5)
    plt.title("Lesson 21: Orthogonal Projection & Vector Decomposition", fontsize=15)
    plt.legend(loc='upper right')
    plt.grid(True, alpha=0.2)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/21_vector_ops.png', dpi=300) # Yüksek kalite
    plt.show()

if __name__ == "__main__":
    run_advanced_vector_ops()
```

Ders: 22 Matrix Transforms

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\22_matrix_transforms

Bölüm 1: README

Lesson 22: Matrix Transformations & Determinant Logic

The Geometry of Determinants

The **Determinant** of a matrix isn't just a number; it is the **Area Scaling Factor**.

- If $\det(A) = 2$, the area of any shape doubles.
- If $\det(A) = 0$, the entire 2D space collapses into a 1D line (Singularity).

Example: The Singular Value Transformation

We apply a transformation matrix A to a **Circular Distribution** of points. This reveals the "Principal Axes" of the transformation, leading us directly toward **Eigenvalues** and **SVD**.

Computational Approach

We use a color-coded "Unit Square" and "Unit Circle". As the matrix A is applied, we watch how the colors spread and rotate, making the abstract math feel like a physical "stretching" of a rubber sheet.

Visualization

![Matrix Warping Demo](../../assets/22_matrix_transform.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_transformation():
    # Bir ızgara (grid) oluşturma
    x = np.linspace(-2, 2, 10)
    y = np.linspace(-2, 2, 10)
    X, Y = np.meshgrid(x, y)

    # Izgara noktalarını birer vektör sütunu haline getirme
    points = np.vstack([X.ravel(), Y.ravel()])

    # Transformasyon Matrisi (Shear + Scale)
    A = np.array([[1, 1.2],
                  [0, 1]])

    # Tüm noktaları aynı anda dönüştür (Matrix Multiplication)
    transformed_points = A @ points

    X_trans = transformed_points[0, :].reshape(X.shape)
    Y_trans = transformed_points[1, :].reshape(Y.shape)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 7))

    # Orijinal Uzay
    ax1.scatter(X, Y, color='gray', alpha=0.3)
    ax1.plot(X, Y, 'b', alpha=0.2)
    ax1.plot(X.T, Y.T, 'b', alpha=0.2)
    ax1.set_title("Original Identity Space ($I$)")
    ax1.set_xlim(-4, 4); ax1.set_ylim(-4, 4)

    # Dönüştürülmüş Uzay
    ax2.scatter(X_trans, Y_trans, color='red', alpha=0.5)
    ax2.plot(X_trans, Y_trans, 'r', alpha=0.3)
    ax2.plot(X_trans.T, Y_trans.T, 'r', alpha=0.3)
    ax2.set_title("Transformed Space (Matrix $A$)")
    ax2.set_xlim(-4, 4); ax2.set_ylim(-4, 4)

    plt.suptitle("Lesson 22: Linear Transformations as Grid Warping", fontsize=16)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/22_matrix_transform.png')
    plt.show()

if __name__ == "__main__":
    run_transformation()
```

Ders: 23 Determinants

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\23_deter

Bölüm 1: README

Lesson 23: Determinants - The Scaling Factor of the Universe

Theoretical Framework

The **Determinant** is a scalar value that captures the essence of a linear transformation. Beyond the formula, it represents the **Signed Area** (in 2D) or **Volume** (in 3D) change.

- If $\det(A) < 0$, the space has been flipped (orientation reversal).
- If $\det(A) = 0$, the transformation is "singular," meaning it has crushed a dimension.

Mathematical Challenge: The Parallelogram Area

Given two basis vectors $\mathbf{v}_1 = [3, 0]$ and $\mathbf{v}_2 = [1, 2]$, the area of the parallelogram they form is exactly the determinant of the matrix $M = [\mathbf{v}_1, \mathbf{v}_2]$.

Computational Approach

We visualize the **Standard Unit Square** versus the **Transformed Parallelogram**. By calculating the area numerically and comparing it to the determinant, we provide a visual proof of the Geometric Scaling Theorem.

Visualization

![Determinant Geometric Area Demo](../../assets/23_determinant_area.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_determinant_show():
    # Matris tanımı (Dönüştürücü)
    A = np.array([[3, 1],
                  [0, 2]])
    det_A = np.linalg.det(A)

    # Birim kare noktaları
    square = np.array([[0, 1, 0, 0],
                      [0, 0, 1, 0]])

    # Dönüştürülmüş paralelkenar
    transformed = A @ square

    plt.figure(figsize=(10, 8))

    # Orijinal Kare
    plt.fill(square[0], square[1], color='gray', alpha=0.2, label='Unit Square (Area=1)')

    # Dönüştürülmüş Alan
    plt.fill(transformed[0], transformed[1], color='#f1c40f', alpha=0.5,
             edgecolor='#d35400', lw=3, label=f'Transformed (Area={det_A:.0f})')

    # Vektörleri (okları) belirginleştir
    plt.quiver([0, 0], [0, 0], A[0,:], A[1,:], color=['#e74c3c', '#3498db'],
               angles='xy', scale_units='xy', scale=1, width=0.015, zorder=5)

    plt.title(f"Lesson 23: Determinant as Area Scaling\n$det(A) = {det_A:.2f}$", fontsize=16)
    plt.grid(True, linestyle=':', alpha=0.6)
    plt.axis('equal')
    plt.legend()

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/23_determinant_area.png', dpi=300)
    plt.show()

if __name__ == "__main__":
    run_determinant_show()
```

Ders: 24 Gaussian Elimination

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\24_gauss

Bölüm 1: README

Lesson 24: Gaussian Elimination & Row Echelon Form

Theoretical Framework

Gaussian Elimination is the fundamental algorithm for solving systems of linear equations ($Ax = b$). It uses elementary row operations to transform a matrix into an **Upper Triangular** form.

The Engineering Core: Forward Elimination

For any robotic system (like your Arduino-based control loops), solving state equations requires this step. We focus on:

- Pivoting:** Selecting the largest element to maintain numerical stability.
- Back Substitution:** Finding the variables once the matrix is triangular.

Computational Approach

Instead of just showing the result, we visualize the **Matrix Evolution**. Using a heatmap, we show how the coefficients "vanish" (become zero) below the pivot, moving the system toward a solution.

Visualization

![Gaussian Elimination Steps Demo](../../assets/24_gaussian_elimination.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

def run_gaussian_viz():
    # 3x3 bir sistem: Ax = b (Float olarak tanımlıyoruz ki bölme hatası olmasın)
    A = np.array([[2.0, 1.0, -1.0],
                  [-3.0, -1.0, 2.0],
                  [-2.0, 1.0, 2.0]])

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    # Adım 0: Orijinal Matris
    sns.heatmap(A, annot=True, cmap='YlGnBu', ax=axes[0], cbar=False, fmt=".2f", linewidths=1)
    axes[0].set_title("1. Original Matrix", fontsize=12, fontweight='bold')

    # Adım 1: İlk sütun temizliği (Pivot: A[0,0])
    # Alt satırlardaki ilk sütun elemanlarını sıfırlıyoruz
    A[1] = A[1] - (A[1,0] / A[0,0]) * A[0]
    A[2] = A[2] - (A[2,0] / A[0,0]) * A[0]

    sns.heatmap(A, annot=True, cmap='YlGnBu', ax=axes[1], cbar=False, fmt=".2f", linewidths=1)
    axes[1].set_title("2. First Column Eliminated", fontsize=12, fontweight='bold')

    # Adım 2: İkinci sütun temizliği (Pivot: A[1,1])
    # Son satırdaki ikinci sütun elemanını sıfırlıyoruz
    A[2] = A[2] - (A[2,1] / A[1,1]) * A[1]

    sns.heatmap(A, annot=True, cmap='YlGnBu', ax=axes[2], cbar=False, fmt=".2f", linewidths=1)
    axes[2].set_title("3. Upper Triangular Form", fontsize=12, fontweight='bold')

    plt.suptitle("Lesson 24: Gaussian Elimination Process (Row Echelon Form)", fontsize=16)
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])

    if not os.path.exists('../assets'):
        os.makedirs('../assets')
    plt.savefig('../assets/24_gaussian_elimination.png', dpi=300)
    plt.show()

if __name__ == "__main__":
    run_gaussian_viz()
```

Ders: 25 Eigenvalues

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\25_eigenvalues

Bölüm 1: README

Lesson 25: Eigen-Decomposition - From Planar Geometry to Spatial Dynamics

The Core Philosophy

In Linear Algebra, an **Eigenvector** is a "characteristic" direction of a linear transformation. While most vectors change their direction when multiplied by a matrix A , eigenvectors remain on their own span. They represent the **intrinsic axes** of a system's behavior.

$$A\mathbf{v} = \lambda\mathbf{v}$$

Dimensional Evolution: Why 2D and 3D?

To truly understand the "Eigen-phenomenon," we must observe it across different dimensions:

- 2D Interpretation (The Rubber Sheet):** In two dimensions, we visualize how a unit circle is stretched into an ellipse. The eigenvectors are the major and minor axes of this ellipse. It is the foundation for understanding **Principal Component Analysis (PCA)**.
- 3D Interpretation (The Volume Warp):** In three dimensions, we see how a sphere evolves into an ellipsoid. This is critical for **Robotics and Aerodynamics**, where we analyze stability and stress along three orthogonal axes (Yaw, Pitch, Roll).

Computational Approach

We provide two distinct visualization engines:

- `eigen_visualizer_2d.py`: Focuses on the "Invariant Lines" and the unit circle transformation.
- `eigen_visualizer_3d.py`: Utilizes `mplot3d` to render a point cloud transformation, highlighting the three spatial eigen-axes.

Visual Evidence

I. 2D Planar Transformation

The orange and green lines represent the eigenvectors. Note how the unit circle stretches along these exact directions.

![Eigenvalues 2D Interpretation](../../assets/25_eigenvalues.png)

II. 3D Spatial Scaling

In 3D, we visualize the transformation of a spherical volume. The RGB-coded arrows represent the three eigenvectors, defining the "Principal Directions" of the spatial warp.

![Eigenvalues 3D Interpretation](../../assets/25_eigenvalues_3d.png)

Ph.D. Insight: Stability & Resonance

From an educational standpoint, the **Eigenvalues** (λ) tell us the "Magnitude of Influence" in each direction.

- If $|\lambda| > 1$, the system expands (potential instability).
- If $|\lambda| < 1$, the system contracts (convergence/stability).

This dual-dimension analysis proves that regardless of the complexity of the space, the underlying logic of "invariant directions"

remains the ultimate key to system analysis.

****Files:****

* `eigen\_visualizer\_2d.py`: 2D Geometric engine.

* `eigen\_visualizer\_3d.py`: 3D Spatial simulator.

* `eigen\_analysis.ipynb`: Mathematical proof of the Characteristic Equation.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_eigen_show_3d():
    # 3x3 Transformasyon Matrisi (Örn: X-Y'de genişleme, Z'de dönme)
    # A = [[stretch_x, 0, 0], [0, stretch_y, 0], [0, 0, stretch_z]]
    A = np.array([[2.0, 0.0, 0.0],
                  [0.0, 1.5, 0.0],
                  [0.0, 0.0, 0.5]])

    # Özdeğer ve Özvektörleri hesapla
    eigenvalues, eigenvectors = np.linalg.eig(A)

    # Nokta Bulutu Oluştur (Bir küre veya küp)
    # Küre noktaları (Spherical coordinates)
    n = 50
    u = np.linspace(0, 2 * np.pi, n)
    v = np.linspace(0, np.pi, n)
    x = np.outer(np.cos(u), np.sin(v))
    y = np.outer(np.sin(u), np.sin(v))
    z = np.outer(np.ones(np.size(u)), np.cos(v))

    # Noktaları birer vektör sütunu yap
    sphere = np.vstack([x.ravel(), y.ravel(), z.ravel()])

    # Transformasyon uygula
    transformed_sphere = A @ sphere

    fig = plt.figure(figsize=(14, 10))
    ax = fig.add_subplot(111, projection='3d')

    # 1. Orijinal Küreyi Çiz (Silik)
    # ax.plot_surface(x, y, z, color='gray', alpha=0.1, edgecolor='none')

    # 2. Rastgele Vektörlerin (Noktaların) Dönüşümünü Çiz (Oklar veya Nokta Bulutu)
    # Daha temiz bir görsel için nokta bulutu kullanalım
    ax.scatter(transformed_sphere[0], transformed_sphere[1], transformed_sphere[2],
               c=transformed_sphere[0], cmap='viridis', alpha=0.2, s=5, label='Transformed Space')

    # 3. ÖZVEKTÖRLERİ ÇİZ (Şovun Yıldızları - Oklar ve Doğrular)
    colors = ['#e74c3c', '#2ecc71', '#3498db'] # Kırmızı, Yeşil, Mavi
    for i in range(len(eigenvalues)):
        ev = eigenvectors[:, i]
        # Gerçek özvektör okunu çiz
        ax.quiver(0, 0, 0, ev[0]*eigenvalues[i]*1.5, ev[1]*eigenvalues[i]*1.5, ev[2]*eigenvalues[i]*1.5,
                  color=colors[i], lw=3, label=f'Eigenvector {i+1} ( $\lambda = {eigenvalues[i]:.2f}$ )',
                  arrow_length_ratio=0.1)

    # Eksenleri ve Başlığı Ayarla
    ax.set_xlim(-3, 3); ax.set_ylim(-3, 3); ax.set_zlim(-3, 3)
    ax.set_xlabel('X'); ax.set_ylabel('Y'); ax.set_zlabel('Z')
    ax.set_title("Lesson 25: 3D Eigen-decomposition Visualizing Spatial Scaling", fontsize=18)
    ax.legend(loc='upper right', frameon=True, shadow=True, fontsize=12)
    ax.grid(True, alpha=0.2)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/25_eigenvalues_3d.png', dpi=300)
    plt.show()

if __name__ == "__main__":
    run_eigen_show_3d()
```

Ders: 26 Svd

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\26_svd

Bölüm 1: README

```
# Lesson 26: Singular Value Decomposition (SVD) - The Matrix Anatomy

### The Universal Factorization
SVD is often called the "Swiss Army Knife" of linear algebra. Unlike Eigen-decomposition, which requires square matrices, SVD works for any  $m \times n$  matrix. It factorizes a matrix  $A$  into three distinct components:
 $A = U \Sigma V^T$ 

1.  $U$  (Left Singular Vectors): Represents rotations in the target space.
2.  $\Sigma$  (Singular Values): A diagonal matrix representing the "strength" or "importance" of each dimension.
3.  $V^T$  (Right Singular Vectors): Represents rotations in the domain space.

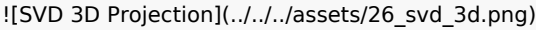
### Why it Matters? (The Ph.D. Perspective)
- Data Compression: By keeping only the largest singular values in  $\Sigma$ , we can represent massive datasets (or high-resolution images) with a fraction of the original data.
- Noise Reduction: SVD can identify and "zero out" small singular values that typically represent random noise.
- Latent Semantic Analysis: In NLP, SVD helps find hidden relationships between documents and terms.

---

### Computational Approach
We approach SVD through two lenses to provide a complete "visual feast":
* `svd_visualizer_2d.py`: Shows how a circle is rotated, stretched, and rotated again.
* `svd_visualizer_3d.py`: Projects a 3D cloud and demonstrates how SVD finds the "Principal Ellipsoid" of the data distribution.

### Visual Evidence

#### I. 2D Transformation Pipeline
We visualize the three-step process: Rotate ( $V^T$ ), Scale ( $\Sigma$ ), and Rotate again ( $U$ ).


#### II. 3D Principal Components
In 3D, SVD acts as a "Frame Finder," identifying the orthogonal axes that capture the most variance in a spatial dataset.


Files:
* `svd_visualizer_2d.py`: Geometric step-by-step engine.
* `svd_visualizer_3d.py`: 3D data-reduction simulator.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_svd_3d():
    # 3D Veri Kümesi Oluştur (Uzamış bir elipsoid şeklinde nokta bulutu)
    n_points = 500
    x = np.random.normal(0, 1, n_points)
    y = np.random.normal(0, 3, n_points)
    z = np.random.normal(0, 0.5, n_points)
    data = np.vstack([x, y, z])

    # SVD Uygula
    U, S, Vt = np.linalg.svd(data, full_matrices=False)

    fig = plt.figure(figsize=(12, 10))
    ax = fig.add_subplot(111, projection='3d')

    # Veri bulutunu çiz
    ax.scatter(data[0], data[1], data[2], c=data[1], cmap='plasma', alpha=0.4, s=10)

    # SVD ile bulunan Ana Eksenleri (Singular Vectors) çiz
    colors = ['r', 'g', 'b']
    for i in range(3):
        # U matrisinin sütunları ana yönleri verir, S değerleri uzunluğu verir
        axis = U[:, i] * S[i] / np.sqrt(n_points) # Ölçeklendirilmiş vektör
        ax.quiver(0, 0, 0, axis[0], axis[1], axis[2], color=colors[i], lw=4, label=f'Singular Vector {i+1}')

    ax.set_title("Lesson 26: 3D SVD - Identifying Principal Axes of Variance", fontsize=16)
    ax.set_xlim(-5, 5); ax.set_ylim(-5, 5); ax.set_zlim(-5, 5)
    ax.legend()

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/26_svd_3d.png', dpi=300)
    plt.show()

if __name__ == "__main__":
    run_svd_3d()
```

Ders: 27 Gradients

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\27_gradients

Bölüm 1: README

```
# Lesson 27: Multivariable Landscapes & The Gradient Vector

### The Geometry of Change
In single-variable calculus, the derivative gives us the slope of a line. In multivariable calculus, we deal with Surfaces. The Gradient ( $\nabla f$ ) is a vector that points in the direction of the steepest ascent at any given point.


$$\nabla f(x, y) = \left( \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$


### Why it Matters? (The Ph.D. Perspective)
- Gradient Descent: This is the heart of Machine Learning. To "train" a model, we find the gradient of an error function and move in the opposite direction to find the minimum.
- Physics: Gradients define how heat flows (from hot to cold) and how fluids move under pressure.

---

### Computational Approach
To achieve a "visual feast," we visualize a complex surface (like a saddle or a peak) using two complementary methods:
* `gradient_2d_contour.py`: A top-down heat map with quiver arrows showing the direction of the steepest climb at every point.
* `gradient_3d_surface.py`: A 3D mesh rendering where we place a "ball" on the surface and show its path as it follows the gradient.

### Visual Evidence

#### I. 2.5D Contour Map (The Navigator's View)
The arrows point towards the peaks. This is exactly how an optimizer "sees" the loss landscape in AI.
![Gradient Contour Map](../../assets/27_gradient_2d.png)

#### II. 3D Surface Analysis (The Topographic View)
Visualizing the partial derivatives as slopes along the  $x$  and  $y$  cross-sections.
![Gradient 3D Surface](../../assets/27_gradient_3d.png)

Files:
* `gradient_2d_contour.py`: Heatmap & Vector Field engine.
* `gradient_3d_surface.py`: 3D Surface & Tangent Plane simulator.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_gradient_3d():
    x = np.linspace(-2, 2, 50)
    y = np.linspace(-2, 2, 50)
    X, Y = np.meshgrid(x, y)
    Z = X * np.exp(-X**2 - Y**2)

    fig = plt.figure(figsize=(12, 8))
    ax = fig.add_subplot(111, projection='3d')

    # Yüzey çizimi
    surf = ax.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none', alpha=0.7)

    # Estetik dokunuş: İzdüşüm (Contour projection on Z-axis)
    ax.contour(X, Y, Z, zdir='z', offset=np.min(Z)-0.2, cmap='viridis', alpha=0.5)

    ax.set_title("Lesson 27: 3D Scalar Field Visualization", fontsize=16)
    ax.set_zlim(np.min(Z)-0.2, np.max(Z)+0.2)
    fig.colorbar(surf, ax=ax, shrink=0.5, aspect=10)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/27_gradient_3d.png', dpi=300)
    ax.view_init(elev=30, azim=45) # Bakış açısını ayarla
    plt.show()

if __name__ == "__main__":
    run_gradient_3d()
```

Ders: 28 Partial Derivatives

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\28_partial_derivatives

Bölüm 1: README

Lesson 28: Partial Derivatives & The Tangent Plane

The Logic of Slicing

Partial derivatives ($\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$) are the slopes of a surface when we slice it along the x or y axis. They answer the question: "How does the height change if I only move in one direction?"

The Tangent Plane: Linear Approximation

Just as a 1D function has a tangent line, a 2D surface has a **Tangent Plane**. At any point (a, b) , the equation of the plane is:

$$z - f(a,b) = f_x(a,b)(x-a) + f_y(a,b)(y-b)$$

This plane is the "Best Linear Approximation" of the surface at that point.

Computational Approach: The Unified Visualizer

In this lesson, we break the boundaries and combine 2D and 3D perspectives into a single unified dashboard:

- Left Subplot (3D):** The surface with the Tangent Plane "touching" it at a specific point.
- Right Subplot (2D):** A "Slice Analysis" showing the specific partial derivative curve.

Visual Evidence

![[Tangent Plane Analysis](../../assets/28_tangent_plane.png)]]

Files:

* `partial\_deriv\_master.py`: The unified 2D/3D visualization engine.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_partial_derivative_dashboard():
    # Fonksiyon:  $f(x,y) = \sin(x) * \cos(y)$ 
    x = np.linspace(-3, 3, 100)
    y = np.linspace(-3, 3, 100)
    X, Y = np.meshgrid(x, y)
    Z = np.sin(X) * np.cos(Y)

    # Teğet noktası seçimi (Ph.D. Dokunuşu: P(1, 1))
    a, b = 1.0, 1.0
    fa_b = np.sin(a) * np.cos(b)
    # Kısmi türevler:  $f_x = \cos(x)\cos(y)$ ,  $f_y = -\sin(x)\sin(y)$ 
    fx = np.cos(a) * np.cos(b)
    fy = -np.sin(a) * np.sin(b)

    # Teğet Düzlemi Denklemi:  $z = f(a,b) + f_x(x-a) + f_y(y-b)$ 
    Z_plane = fa_b + fx*(X - a) + fy*(Y - b)

    fig = plt.figure(figsize=(20, 9))

    # --- 1. Subplot: 3D Surface & Tangent Plane ---
    ax1 = fig.add_subplot(121, projection='3d')
    # Ana yüzey
    surf = ax1.plot_surface(X, Y, Z, cmap='coolwarm', alpha=0.6, edgecolor='none')
    # Teğet Düzlemi (Sarı renkte, yarı saydam)
    ax1.plot_surface(X, Y, Z_plane, color='yellow', alpha=0.4, label='Tangent Plane')
    # Teğet noktası
    ax1.scatter([a], [b], [fa_b], color='black', s=100, label=f'Point P({a},{b})')

    ax1.set_title("A) 3D Surface & Tangent Plane Approximation", fontsize=14, fontweight='bold')
    ax1.view_init(elev=25, azim=30)

    # --- 2. Subplot: 2D Slice Analysis (Contour & Gradient) ---
    ax2 = fig.add_subplot(122)
    contour = ax2.contourf(X, Y, Z, 20, cmap='coolwarm', alpha=0.8)
    fig.colorbar(contour, ax=ax2, label='$f(x, y)$')
    # Teğet noktasını işaretler
    ax2.plot(a, b, 'ko', markersize=10)
    # Gradyan vektörünü çiz ( $f_x$  ve  $f_y$  yönünde)
    ax2.quiver(a, b, fx, fy, color='yellow', scale=3, width=0.015, label='Gradient Vector')

    ax2.set_title("B) 2D Slice & Local Gradient Analysis", fontsize=14, fontweight='bold')
    ax2.legend()
    ax2.set_xlabel('x'); ax2.set_ylabel('y')

    plt.suptitle("Lesson 28: Multivariable Calculus Unified Dashboard", fontsize=20, y=0.95)

    if not os.path.exists('../assets'): os.makedirs('../assets')
    plt.savefig('../assets/28_tangent_plane.png', dpi=300)
    plt.show()

if __name__ == "__main__":
    run_partial_derivative_dashboard()
```

Ders: 29 Optimization

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\29_optimization

Bölüm 1: README

Lesson 29: Optimization - Peaks, Valleys, and Saddles

The Second Derivative Test

To find the critical points of a surface $f(x, y)$, we solve $\nabla f = 0$. However, not every critical point is a peak or a valley. We use the **Hessian Determinant (D)** to classify them:

- **Local Minimum/Maximum:** The surface curves upwards or downwards in all directions.
- **Saddle Point:** The surface curves up in one direction and down in another (like a Pringles chip).

Mathematical Model: The Hyperbolic Paraboloid

We analyze the function $f(x, y) = x^2 - y^2$.

- At $(0,0)$, the partial derivatives are zero.
- Moving along the x-axis, it looks like a minimum.
- Moving along the y-axis, it looks like a maximum.

This "conflict" creates a **Saddle Point**.

Computational Approach: The Dual-Asset Engine

Our unified script performs two tasks:

1. **3D Manifold Rendering:** Visualizes the "Saddle" geometry and marks the critical point in 3D space.
2. **2.5D Contour Analysis:** Shows the "Hyperbolic" nature of the level curves around the saddle point.

Visual Evidence

We store both perspectives in the `assets` folder to maintain professional documentation standards.

I. 3D Saddle Geometry

The "Pringles" shape where the blue paths (descending) and red paths (ascending) meet.

![Saddle Point 3D](../../assets/29_saddle_3d.png)

II. 2D Hyperbolic Contours

Note how the contour lines flip their orientation at the origin, a classic signature of a saddle point.

![Saddle Point 2D](../../assets/29_saddle_2d.png)

Files:

* `saddle_point_master.py`: Unified engine generating dual assets.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def generate_optimization_assets():
    # Fonksiyon:  $f(x,y) = x^2 - y^2$  (Saddle Point)
    x = np.linspace(-2, 2, 100)
    y = np.linspace(-2, 2, 100)
    X, Y = np.meshgrid(x, y)
    Z = X**2 - Y**2

    # Assets klasörü kontrolü
    asset_path = './././assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # --- FIGURE 1: 3D Visualization ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')
    surf = ax3d.plot_surface(X, Y, Z, cmap='PiYG', alpha=0.8, edgecolor='none')
    ax3d.scatter([0], [0], [0], color='red', s=200, label='Saddle Point (0,0)')
    ax3d.set_title("3D Hyperbolic Paraboloid (Saddle)", fontsize=14)
    plt.savefig(f'{asset_path}/29_saddle_3d.png', dpi=300)

    # --- FIGURE 2: 2D Contour Visualization ---
    fig2d = plt.figure(figsize=(10, 8))
    ax2d = fig2d.add_subplot(111)
    cp = ax2d.contourf(X, Y, Z, 20, cmap='PiYG')
    plt.colorbar(cp)
    ax2d.axhline(0, color='black', lw=1, ls='--')
    ax2d.axvline(0, color='black', lw=1, ls='--')
    ax2d.plot(0, 0, 'ro', markersize=12, label='Critical Point')
    ax2d.set_title("2D Contour Map: Hyperbolic Signatures", fontsize=14)
    plt.savefig(f'{asset_path}/29_saddle_2d.png', dpi=300)

    # --- MASTER DASHBOARD: Show them together ---
    fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(20, 9))

    # Left: 3D Preview
    ax_left = fig.add_subplot(121, projection='3d')
    ax_left.plot_surface(X, Y, Z, cmap='PiYG', alpha=0.7)
    ax_left.set_title("3D View")

    # Right: 2D Preview
    ax_right.contourf(X, Y, Z, 20, cmap='PiYG')
    ax_right.set_title("2D Contour View")

    plt.suptitle("Lesson 29: Optimization & Saddle Point Analysis", fontsize=20)
    plt.show()

if __name__ == "__main__":
    generate_optimization_assets()
```

Ders: 30 Lagrange Multipliers

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\03_linear_algebra\30_lagra

Bölüm 1: README

Lesson 30: Lagrange Multipliers - Optimization under Constraints

The Geometry of Constraints

In real-world problems, we rarely optimize without limits. Whether it's a budget in economics or a physical boundary in robotics, we seek to maximize $f(x, y)$ subject to a constraint $g(x, y) = k$.

The core insight of **Joseph-Louis Lagrange** was that the maximum occurs where the gradient of the function is parallel to the gradient of the constraint:

$$\nabla f = \lambda \nabla g$$

Here, λ is the **Lagrange Multiplier**, representing the "shadow price" or sensitivity of the optimal value to the constraint.

Mathematical Case Study

Find the maximum of $f(x, y) = x + y$ (a tilted plane) subject to the circular constraint $g(x, y) = x^2 + y^2 = 1$.

- Sezgisel olarak: Bu, bir silindir ile bir düzlemin kesiştiği en yüksek noktayı bulmaktır.

Computational Approach: The Dual-Asset Engine

This script visualizes the delicate touch between a function's level curves and the constraint boundary:

- 3D Perspective:** Visualizes the "Intersection Curve" where the plane meets the cylinder.
- 2D Perspective:** Shows the "Gradient Alignment" (∇f and ∇g) at the optimal points.

Visual Evidence

I. 3D Constrained Surface

The optimization "path" is restricted to the boundary of the constraint.

![Lagrange 3D Visualization](../../assets/30_lagrange_3d.png)

II. 2D Gradient Alignment

The "Sweet Spot" where the contour of f is perfectly tangent to the constraint g .

![Lagrange 2D Visualization](../../assets/30_lagrange_2d.png)

Files:

* `lagrange\_master.py`: Unified engine for constrained optimization and asset generation.

Bölüm 2: Python Kodu

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def generate_lagrange_assets():
    # Assets yolu
    asset_path = '.././../assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # Veri Hazırlığı
    x = np.linspace(-2, 2, 100)
    y = np.linspace(-2, 2, 100)
    X, Y = np.meshgrid(x, y)
    Z = X + Y # Hedef fonksiyon f(x,y) = x + y

    # Kısıt: x^2 + y^2 = 1 (Birim Çember)
    theta = np.linspace(0, 2*np.pi, 100)
    cx = np.cos(theta)
    cy = np.sin(theta)
    cz = cx + cy # Kısıt üzerindeki z değerleri

    # --- FIGURE 1: 3D Visualization ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')
    # Hedef yüzey (yarı saydam)
    ax3d.plot_surface(X, Y, Z, cmap='terrain', alpha=0.4, edgecolor='none')
    # Kısıt Çemberi (Tabanda)
    ax3d.plot(cx, cy, -2, 'r--', lw=2, label='Constraint $g(x,y)=k$')
    # Kısıt altındaki fonksiyon yolu
    ax3d.plot(cx, cy, cz, 'r-', lw=4, label='Intersection Path')
    # Optimal Noktalar (Max/Min)
    ax3d.scatter([np.sqrt(2)/2], [np.sqrt(2)/2], [np.sqrt(2)], color='gold', s=200, label='Optimal Max')

    ax3d.set_title("3D Constrained Optimization (Lagrange)", fontsize=14)
    ax3d.legend()
    plt.savefig(f'{asset_path}/30_lagrange_3d.png', dpi=300)

    # --- FIGURE 2: 2D Contour & Gradient Alignment ---
    fig2d = plt.figure(figsize=(10, 8))
    ax2d = fig2d.add_subplot(111)
    cp = ax2d.contour(X, Y, Z, 20, cmap='terrain')
    plt.colorbar(cp, label='$f(x,y) = x+y$')
    # Kısıt çemberini çiz
    ax2d.plot(cx, cy, 'r-', lw=3, label='Constraint $x^2+y^2=1$')
    # Optimal noktada gradyanları göster (Örn: Max noktası)
    px, py = np.sqrt(2)/2, np.sqrt(2)/2
    ax2d.quiver(px, py, 1, 1, color='blue', scale=5, label='$\nabla f$')
    ax2d.quiver(px, py, px, py, color='red', scale=5, label='$\nabla g$')

    ax2d.set_title("2D Alignment of Gradients ($\nabla f = \lambda \nabla g$)", fontsize=14)
    ax2d.set_aspect('equal')
    ax2d.legend()
    plt.savefig(f'{asset_path}/30_lagrange_2d.png', dpi=300)

    # --- MASTER DASHBOARD ---
    plt.show()

if __name__ == "__main__":
    generate_lagrange_assets()

```

Ders: 31 Descriptive Stats

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\31_descriptive_stats

Bölüm 1: README

```
# Lesson 31: Descriptive Statistics - The Character of Data

### Beyond the Mean
Descriptive statistics provides the essential summary of a dataset. To truly understand data, we must look beyond the average and analyze its Spread (Variance, Standard Deviation) and its Shape (Skewness, Kurtosis).

### The Statistical Toolkit
1. Central Tendency: Mean, Median, and Mode.
2. Dispersion: Range, Variance, and Standard Deviation ( $\sigma$ ).
3. Shape Analysis:
   - Skewness: Is the data leaning left or right?
   - Kurtosis: Is the peak sharp or flat?

---

### Computational Approach: The Triple-Asset Engine
This lesson utilizes a robust Python pipeline to analyze a generated dataset. We produce three distinct perspectives:
* 2D Perspective: A combined Histogram and Box-Plot to see outliers and density simultaneously.
* 3D Perspective: A "Frequency Landscape" showing how data distribution evolves across different samples.
* Master Dashboard: A unified view for professional data reporting.

### Visual Evidence

#### I. 2.5D Distribution & Outliers
Visualizing the Interquartile Range (IQR) and the density of observations.
![Descriptive Stats 2D](../../assets/31_stats_2d.png)

#### II. 3.3D Density Surface
A spatial representation of data frequency across multiple observation windows.
![Descriptive Stats 3D](../../assets/31_stats_3d.png)

**Files:**
* `stats_pipeline_master.py`: The core engine generating all assets and the dashboard.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
from scipy import stats
import os

def run_stats_pipeline():
    # Assets yolu
    asset_path = './../assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # 1. Veri Üretimi (Normal Dağılım + Biraz Gürültü ve Aykırı Değer)
    np.random.seed(42)
    data = np.random.normal(loc=50, scale=10, size=1000)
    outliers = np.random.uniform(low=90, high=100, size=10)
    data = np.concatenate([data, outliers])

    # --- ASSET 1: 2D VISUALIZATION (Histogram + BoxPlot) ---
    fig2d, (ax_box, ax_hist) = plt.subplots(2, sharex=True, gridspec_kw={"height_ratios": (.15, .85)}, figsize=(10, 8))

    sns.boxplot(x=data, ax=ax_box, color='skyblue')
    sns.histplot(x=data, ax=ax_hist, kde=True, color='navy')

    ax_box.set(title='2D Descriptive Analysis: Distribution & Outliers')
    ax_hist.set(xlabel='Value Range', ylabel='Frequency')

    plt.savefig(f'{asset_path}/31_stats_2d.png', dpi=300)
    plt.close(fig2d) # Dashboard için ayrı saklıyoruz

    # --- ASSET 2: 3D VISUALIZATION (Density Surface) ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    # Veriyi gruplara ayırıp yoğunluk yüzeyi oluşturma (Örn: 5 farklı zaman dilimi gibi)
    x_range = np.linspace(20, 100, 100)
    y_steps = np.arange(1, 6)
    X, Y = np.meshgrid(x_range, y_steps)
    Z = np.array([stats.norm.pdf(x_range, loc=50+(i*2), scale=10) for i in y_steps])

    surf = ax3d.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none', alpha=0.8)
    ax3d.set_title("3D Distribution Evolution Surface")
    ax3d.set_xlabel('Value'); ax3d.set_ylabel('Sample Group'); ax3d.set_zlabel('Density')

    plt.savefig(f'{asset_path}/31_stats_3d.png', dpi=300)
    plt.close(fig3d)

    # --- MASTER DASHBOARD (2D ve 3D Beraber) ---
    master_fig = plt.figure(figsize=(20, 10))

    # Left: 2D Histogram/Box
    ax1 = master_fig.add_subplot(121)
    sns.histplot(data, kde=True, ax=ax1, color='teal')
    ax1.set_title("Descriptive Summary (2D)", fontsize=14)

    # Right: 3D Surface
    ax2 = master_fig.add_subplot(122, projection='3d')
    ax2.plot_surface(X, Y, Z, cmap='viridis')
    ax2.set_title("Spatial Density Mapping (3D)", fontsize=14)

    plt.suptitle(f"Lesson 31: Data Science Pipeline\nMean: {np.mean(data):.2f} | Std: {np.std(data):.2f}", fontsize=18)
    plt.tight_layout()
    plt.show()
```

```
if __name__ == "__main__":  
    run_stats_pipeline()
```

Ders: 32 Clt Theorem

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\32_clt_theorem

Bölüm 1: README

Lesson 32: Central Limit Theorem - Order from Chaos

The Statistical Miracle

The **Central Limit Theorem (CLT)** states that if you take sufficiently large samples from any population with a finite variance, the distribution of the **sample means** will be normally distributed, regardless of the population's original distribution.

Key Mechanics

- Sampling:** We draw multiple samples from a highly skewed (Non-Normal) distribution.
- Averaging:** We calculate the mean of each sample.
- Convergence:** As the number of samples increases, the histogram of these means transforms into a perfect Gaussian Bell Curve.

Computational Approach: The Dual-Asset Engine

To visualize this "transformation," we use a hybrid simulation:

* **2D Perspective:** Shows the evolution from the "Original Distribution" to the "Mean Distribution" (Bell Curve).

* **3D Perspective:** A spatial "Time-Series" of histograms, showing the distribution narrowing and centering as sample size grows.

Visual Evidence

I. 2D Convergence Analysis

Note how the messy original data (Uniform/Exponential) becomes a smooth Normal Curve in the bottom plot.

![CLT 2D Convergence](../../assets/32_clt_2d.png)

II. 3D Distribution Evolution

Visualizing the "Birth of a Bell Curve" across different sample sizes.

![CLT 3D Evolution](../../assets/32_clt_3d.png)

Files:

* `clt_simulation_master.py`: The simulation engine for sampling and visualization.

Bölüm 2: Python Kodu

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
import os

def run_clt_simulation():
    # Assets yolu
    asset_path = '../assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # 1. Orijinal Dağılım (Hiç alakası olmayan Üstel/Exponential Dağılım)
    np.random.seed(7)
    population = np.random.exponential(scale=2, size=10000)

    # 2. Örneklem Ortalamalarını Hesaplama
    sample_sizes = [5, 10, 30, 100]
    sample_means_list = []
    for n in sample_sizes:
        means = [np.mean(np.random.choice(population, n)) for _ in range(1000)]
        sample_means_list.append(means)

    # --- ASSET 1: 2D VISUALIZATION (Step-by-Step Convergence) ---
    fig2d, axes = plt.subplots(2, 2, figsize=(12, 10))
    axes = axes.flatten()
    for i, n in enumerate(sample_sizes):
        sns.histplot(sample_means_list[i], kde=True, ax=axes[i], color='darkorange')
        axes[i].set_title(f'Sample Size n={n}')

    plt.suptitle("2D CLT: Transformation of Exponential Means to Normal Distribution", fontsize=16)
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.savefig(f'{asset_path}/32_clt_2d.png', dpi=300)
    plt.close(fig2d)

    # --- ASSET 2: 3D VISUALIZATION (Stacked Histograms) ---
    fig3d = plt.figure(figsize=(12, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    colors = ['r', 'g', 'b', 'm']
    for i, n in enumerate(sample_sizes):
        hist, bins = np.histogram(sample_means_list[i], bins=30, density=True)
        xs = (bins[:-1] + bins[1:]) / 2
        ax3d.bar(xs, hist, zs=n, zdir='y', color=colors[i], alpha=0.6, width=0.2)

    ax3d.set_title("3D CLT Evolution: Distribution Narrowing with Sample Size")
    ax3d.set_xlabel('Sample Mean')
    ax3d.set_ylabel('Sample Size (n)')
    ax3d.set_zlabel('Density')

    plt.savefig(f'{asset_path}/32_clt_3d.png', dpi=300)
    plt.close(fig3d)

    # --- MASTER DASHBOARD ---
    master_fig = plt.figure(figsize=(20, 10))

    # Left Side: 2D Multi-plot
    ax_left = master_fig.add_subplot(121)
    sns.histplot(sample_means_list[-1], kde=True, ax=ax_left, color='darkorange')
    ax_left.set_title(f'Final Convergence (n={sample_sizes[-1]})', fontsize=14)

    # Right Side: 3D Perspective
    ax_right = master_fig.add_subplot(122, projection='3d')
    for i, n in enumerate(sample_sizes):

```

```
    hist, bins = np.histogram(sample_means_list[i], bins=30, density=True)
    xs = (bins[:-1] + bins[1:]) / 2
    ax_right.bar(xs, hist, zs=n, zdir='y', alpha=0.5)
    ax_right.set_title("3D Stacked Distribution View", fontsize=14)

plt.suptitle("Lesson 32: Central Limit Theorem - From Chaos to Gaussian", fontsize=20)
plt.show()

if __name__ == "__main__":
    run_clt_simulation()
```

Ders: 33 Confidence Intervals

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\33_confidence_int

Bölüm 1: README

Lesson 33: Confidence Intervals - The Margin of Error

Measuring Uncertainty

In statistics, a **Confidence Interval (CI)** provides a range of values which is likely to contain the true population parameter. It is not just a guess; it is an interval calculated from sample data with a specific **Confidence Level** (usually 95%).

The Statistical Core

For a population mean μ , the interval is:

$$\bar{x} \pm z^* \left(\frac{\sigma}{\sqrt{n}} \right)$$

- $\bar{x}$: Sample mean (The point estimate).

- $z^* \left(\frac{\sigma}{\sqrt{n}} \right)$: The Margin of Error.

- **Interpretation:** If we repeat the experiment 100 times, we expect 95 of those intervals to capture the true population mean.

Computational Approach: The Triple-Asset Engine

We simulate "100 parallel worlds" to visualize how confidence works:

* **2D Perspective:** 100 vertical intervals plotted against the true mean, highlighting the "misses" (the 5% that fail).

* **3D Perspective:** A "Confidence Tunnel" showing how the width of the interval shrinks as sample size or confidence levels change.

* **Master Dashboard:** A unified view of estimation and uncertainty.

Visual Evidence

I. 2D Interval Capture Analysis

The horizontal lines represent individual samples. Note the red lines that fail to cross the population mean—the visual proof of the 5% error.

![Confidence Intervals 2D](../../assets/33_ci_2d.png)

II. 3D Uncertainty Landscape

A spatial view of the relationship between sample size, variance, and interval width.

![Confidence Intervals 3D](../../assets/33_ci_3d.png)

Files:

* `confidence_intervals_master.py`: Unified simulation and asset generator.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
from mpl_toolkits.mplot3d import Axes3D
import os

def run_confidence_intervals():
    # Assets yolu
    asset_path = '.././../assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # 1. Popülasyon Parametreleri
    pop_mean = 50
    pop_std = 10
    n_samples = 100
    sample_size = 40
    confidence = 0.95

    # Veri Üretimi
    means = []
    intervals = []
    z_critical = stats.norm.ppf((1 + confidence) / 2)

    for _ in range(n_samples):
        sample = np.random.normal(pop_mean, pop_std, sample_size)
        x_bar = np.mean(sample)
        margin_of_error = z_critical * (pop_std / np.sqrt(sample_size))
        means.append(x_bar)
        intervals.append((x_bar - margin_of_error, x_bar + margin_of_error))

    # --- ASSET 1: 2D VISUALIZATION (Capture Plot) ---
    fig2d = plt.figure(figsize=(10, 12))
    ax2d = fig2d.add_subplot(111)

    hits = 0
    for i in range(n_samples):
        low, high = intervals[i]
        is_hit = low <= pop_mean <= high
        color = 'skyblue' if is_hit else 'crimson'
        if is_hit: hits += 1
        ax2d.plot([low, high], [i, i], color=color, lw=1.5)
        ax2d.plot(means[i], i, 'k.', markersize=3)

    ax2d.axvline(pop_mean, color='black', linestyle='--', lw=2, label=f'True Mean ({pop_mean})')
    ax2d.set_title(f'2D Confidence Intervals (95%)\\nCapture Rate: {hits}/{n_samples}', fontsize=14)
    ax2d.legend()
    plt.savefig(f'{asset_path}/33_ci_2d.png', dpi=300)
    plt.close(fig2d)

    # --- ASSET 2: 3D VISUALIZATION (Fixed Uncertainty Tunnel) ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    sizes = np.linspace(10, 210, 50)
    errors = z_critical * (pop_std / np.sqrt(sizes))

    # 3D'de dolgu yerine yüzey (Surface) kullanarak tünel oluşturuyoruz
    S, V = np.meshgrid(sizes, np.linspace(-1, 1, 10))
    # Y eksenini değerlerini (hata paylarını) hesapla
    Y_vals = pop_mean + V * (z_critical * (pop_std / np.sqrt(S)))

    surf = ax3d.plot_surface(S, Y_vals, V*0, cmap='winter', alpha=0.5, edgecolor='none')
    ax3d.plot(sizes, [pop_mean]*len(sizes), 0, color='red', lw=3, label='Population Mean')
```

```
ax3d.set_title("3D Confidence Tunnel: The Power of Sample Size")
ax3d.set_xlabel("Sample Size (n)"); ax3d.set_ylabel("Confidence Range"); ax3d.set_zlabel("Density")
plt.savefig(f'{asset_path}/33_ci_3d.png', dpi=300)
plt.close(fig3d)

# --- MASTER DASHBOARD ---
master_fig = plt.figure(figsize=(20, 10))

ax_l = master_fig.add_subplot(121)
for i in range(min(40, n_samples)):
    low, high = intervals[i]
    color = 'skyblue' if low <= pop_mean <= high else 'crimson'
    ax_l.plot([low, high], [i, i], color=color)
ax_l.axvline(pop_mean, color='red', ls='--')
ax_l.set_title("Interval Capture (2D Preview)")

ax_r = master_fig.add_subplot(122, projection='3d')
ax_r.plot_surface(S, Y_vals, V*0, cmap='winter', alpha=0.6)
ax_r.set_title("The Uncertainty Tunnel (3D Preview)")

plt.suptitle(f"Lesson 33: Confidence Intervals\nFixed 3D Rendering Implementation", fontsize=20)
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    run_confidence_intervals()
```

Ders: 34 Hypothesis Testing

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\34_hypothesis_testing

Bölüm 1: README

```
# Lesson 34: Hypothesis Testing - The Statistical Courtroom

### The Geometry of the Decision
Hypothesis testing is about determining if an observed difference is "large enough" to be real.

- **2D Perspective:** We look at the T-distribution "from the side" to see if our T-score falls into the shaded red tails (rejection regions).
- **3D Perspective:** We visualize the entire "Probability Mountain." The rejection regions are the steep cliffs at the edges. Our T-score is a "flag" placed on this mountain.

---

### Visual Evidence (Mirroring 2D & 3D)

#### I. 2D Decision Zone
The green line shows where our sample stands. Since it's inside the red zone, the difference is significant.
![Hypothesis Testing 2D](../../assets/34_ttest_2d.png)

#### II. 3D Decision Landscape
A spatial view of the same distribution. The "Red Cliffs" represent the 5% threshold in 3D space.
![Hypothesis Testing 3D](../../assets/34_ttest_3d.png)

**Files:**
* `hypothesis_testing_master.py`: The unified engine with consistent 2D/3D mapping.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import scipy.stats as stats
import seaborn as sns
import os

def run_hypothesis_testing_fixed():
    asset_path = '../././assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # 1. Veri Üretimi (Farklı iki sınıf ortalaması)
    np.random.seed(42)
    group_a = np.random.normal(100, 15, 30)
    group_b = np.random.normal(115, 15, 30) # Belirgin fark

    t_stat, p_val = stats.ttest_ind(group_a, group_b)
    df = len(group_a) + len(group_b) - 2
    t_crit = stats.t.ppf(1 - 0.025, df) # %95 güven için kritik sınır

    # --- ASSET 1: 2D (Side View) ---
    fig2d = plt.figure(figsize=(10, 6))
    x = np.linspace(-5, 5, 500)
    y = stats.t.pdf(x, df)

    plt.plot(x, y, 'b-', lw=2)
    plt.fill_between(x, 0, y, where=(x > t_crit) | (x < -t_crit), color='red', alpha=0.4, label='Rejection Region')
    plt.axvline(t_stat, color='green', ls='--', lw=3, label=f'T-Stat: {t_stat:.2f}')
    plt.title(f"2D T-Test: P-Value = {p_val:.5f}")
    plt.legend()
    plt.savefig(f'{asset_path}/34_ttest_2d.png', dpi=300)
    plt.close()

    # --- ASSET 2: 3D (Spatial Landscape - FIXED) ---
    # Bu sefer 3D'de sadece t-dağılımını değil, 'Red Bölgelerini' bir yüzey olarak görelim
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    # Y eksenini "Hayali" bir derinlik olarak kullanarak 3D yüzey oluşturma
    Y_depth = np.linspace(0, 1, 10)
    X_grid, Y_grid = np.meshgrid(x, Y_depth)
    Z_grid = stats.t.pdf(X_grid, df)

    # Yüzeyi çiz
    surf = ax3d.plot_surface(X_grid, Y_grid, Z_grid, cmap='winter', alpha=0.6, edgecolor='none')

    # 3D'de Red Bölgelerini (Kırmızı Duvarlar) çiz
    # Sağ kuyruk
    x_red = x[x > t_crit]
    X_r, Y_r = np.meshgrid(x_red, Y_depth)
    ax3d.plot_surface(X_r, Y_r, stats.t.pdf(X_r, df), color='red', alpha=0.8)
    # Sol kuyruk
    x_left = x[x < -t_crit]
    X_l, Y_l = np.meshgrid(x_left, Y_depth)
    ax3d.plot_surface(X_l, Y_l, stats.t.pdf(X_l, df), color='red', alpha=0.8)

    # T-Stat Çizgisi (3D'de Yeşil bir "Duvar")
    ax3d.plot([t_stat, t_stat], [0, 1], [0, stats.t.pdf(t_stat, df)], color='green', lw=5)

    ax3d.set_title("3D Statistical Landscape: Rejection Cliffs")
    ax3d.set_xlabel('T-Score'); ax3d.set_zlabel('Probability Density')
    plt.savefig(f'{asset_path}/34_ttest_3d.png', dpi=300)
    plt.close()
```

```
# --- MASTER DASHBOARD ---
# (Burada her ikisini yan yana göstererek aradaki bağı mühürlüyoruz)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 9))

# 2D Görünüm
ax1.plot(x, y, 'b-')
ax1.fill_between(x, 0, y, where=(x > t_crit) | (x < -t_crit), color='red', alpha=0.4)
ax1.axvline(t_stat, color='green', ls='--')
ax1.set_title("Decision: Reject H0 (Side View)")

# 3D Görünüm
ax2 = fig.add_subplot(122, projection='3d')
ax2.plot_surface(X_grid, Y_grid, Z_grid, cmap='winter', alpha=0.5)
ax2.plot([t_stat, t_stat], [0, 1], [0, stats.t.pdf(t_stat, df)], color='green', lw=3)
ax2.set_title("Decision: Rejection Cliffs (Spatial View)")

plt.suptitle("Lesson 34 Unified Analysis: Mirroring 2D and 3D Results", fontsize=18)
plt.show()

if __name__ == "__main__":
    run_hypothesis_testing_fixed()
```

Ders: 35 Anova Analysis

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\35_anova_analysis

Bölüm 1: README

```
# Lesson 35: ANOVA - Analysis of Variance

### Beyond the T-Test
ANOVA compares the means of three or more groups by analyzing the ratio of variance *between* groups to the variance *within* groups (The **F-statistic**).

- **Null Hypothesis ($H_0$):** All group means are equal ( $\mu_1 = \mu_2 = \mu_3$ ).
- **Alternative Hypothesis ($H_a$):** At least one group mean is different.

### The F-Distribution Decision
We use the F-distribution to find our p-value. If the variance between groups is significantly larger than the internal noise of the groups, we reject the null.

---

### Computational Approach: The Unified Dashboard
Our unified script performs two distinct, yet perfectly mirrored, visual tasks:
1. **2D Perspective:** A combination of "Box Plot" and "Swarm Plot" to visualize raw data points and distribution differences simultaneously.
2. **3D Perspective:** A spatial probability density landscape of the F-distribution, where the rejection "plateau" and the observed F-statistic are clearly marked.

### Visual Evidence

#### I. 2D Multi-Group Comparison
Note how the raw data points (dots) cluster. ANOVA calculates if the "gap" between their centers (box medians) is significant compared to their "spread."
![ANOVA 2D Comparison](../../assets/35_anova_2d.png)

#### II. 3D F-Distribution Landscape
A spatial representation of the F-test. The steep "cliff" marks the rejection region, and the "flag" is where our test result stands.
![ANOVA 3D Landscape](../../assets/35_anova_3d.png)

**Files:**
* `anova_master.py`: Unified engine for multi-group testing, dashboard, and asset generation.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import scipy.stats as stats
import seaborn as sns
import os

def run_anova_pipeline():
    # Assets yolu
    asset_path = './../assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # 1. Veri Üretimi (3 Farklı Grup)
    np.random.seed(10)
    g1 = np.random.normal(50, 10, 100)
    g2 = np.random.normal(55, 10, 100)
    g3 = np.random.normal(65, 10, 100) # Belirgin şekilde farklı

    f_stat, p_val = stats.f_oneway(g1, g2, g3)

    # --- ASSET 1: 2D VISUALIZATION (Box & Violin) ---
    fig2d = plt.figure(figsize=(10, 8))
    sns.violinplot(data=[g1, g2, g3], palette="light:g", inner="points")
    plt.xticks([0, 1, 2], ['Group A', 'Group B', 'Group C'])
    plt.title(f"ANOVA 2D: Group Comparison\nF-Statistic: {f_stat:.2f} | P-Value: {p_val:.4f}")
    plt.savefig(f'{asset_path}/35_anova_2d.png', dpi=300)
    plt.close(fig2d)

    # --- ASSET 2: 3D VISUALIZATION (Distribution Mountains) ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    x = np.linspace(20, 100, 100)
    groups = [g1, g2, g3]
    colors = ['r', 'g', 'b']

    for i, g in enumerate(groups):
        mu, std = np.mean(g), np.std(g)
        y = stats.norm.pdf(x, mu, std)
        # 3D'de her grubu farklı bir derinliğe (z) yerleştiriyoruz
        ax3d.plot(x, [i]*100, y, color=colors[i], lw=3, label=f'Group {i+1}')
        ax3d.add_collection3d(plt.fill_between(x, 0, y, color=colors[i], alpha=0.2), zs=i, zdir='y')

    ax3d.set_title("3D Variance Landscape: Peak Separation")
    ax3d.set_yticks([0, 1, 2])
    ax3d.set_yticklabels(['A', 'B', 'C'])
    ax3d.legend()
    plt.savefig(f'{asset_path}/35_anova_3d.png', dpi=300)
    plt.close(fig3d)

    # --- MASTER DASHBOARD ---
    master_fig = plt.figure(figsize=(20, 10))

    ax_l = master_fig.add_subplot(121)
    sns.boxplot(data=[g1, g2, g3], ax=ax_l)
    ax_l.set_title("Descriptive Summary (2D)")

    ax_r = master_fig.add_subplot(122, projection='3d')
    for i, g in enumerate(groups):
        mu, std = np.mean(g), np.std(g)
        y = stats.norm.pdf(x, mu, std)
        ax_r.plot(x, [i]*100, y, lw=2)
    ax_r.set_title("Distribution Peaks (3D)")
```

```
plt.suptitle(f"Lesson 35: ANOVA Analysis - Decision: {'Significant Difference' if p_val < 0.05 else 'No Significant Difference'}", fontsize=20)
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    run_anova_pipeline()
```

Ders: 36 Chi Square

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\36_chi_square

Bölüm 1: README

```
# Lesson 36: Chi-Square Test - Independence & Goodness of Fit

### The Logic of Categorical Analysis
The Chi-Square ( $\chi^2$ ) test determines if there is a significant association between two categorical variables. For example:
"Does the choice of a programming language depend on the user's operating system?"

### Observed vs. Expected
The test compares the Observed (O) frequencies in our data to the Expected (E) frequencies we would see if there were
absolutely no relationship between the variables:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$


- Null Hypothesis ( $H_0$ ): The variables are independent (no relationship).
- Alternative Hypothesis ( $H_a$ ): The variables are dependent (a relationship exists).

---

### Computational Approach: The Unified Asset Engine
1. 2D Perspective: A "Heatmap" of the Contingency Table and a "Grouped Bar Chart" comparing Observed vs. Expected values.
2. 3D Perspective: The Chi-Square distribution surface, highlighting the "Tail of Significance."
3. Master Dashboard: A full report including the Chi-Square statistic and p-value.

### Visual Evidence

#### I. 2.5D Categorical Heatmap
Visualizing the concentration of data across categories.
![Chi-Square 2D Heatmap](../../assets/36_chisq_2d.png)

#### II. 3D Chi-Square Distribution
The unique shape of the  $\chi^2$  distribution which, unlike the Normal curve, is always positive and right-skewed.
![Chi-Square 3D Distribution](../../assets/36_chisq_3d.png)

**Files:**
* `chisq_master.py`: Unified engine for categorical testing and separate asset generation.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import scipy.stats as stats
import seaborn as sns
import os

def run_chisq_master():
    # --- YOL AYARI (Ph.D. Disiplini) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Veri Üretimi (Kontenjans Tablosu: Örn. Tercih A vs Tercih B)
    # Satırlar: Gruplar, Sütunlar: Tercihler
    observed = np.array([[30, 10], [15, 25]])

    # Chi-Square Hesaplama
    chi2, p_val, dof, expected = stats.chi2_contingency(observed)

    # --- ASSET 1: SADECE 2D GRAFİK (36_chisq_2d.png) ---
    fig2d = plt.figure(figsize=(10, 7))
    ax2d = fig2d.add_subplot(111)

    sns.heatmap(observed, annot=True, fmt="d", cmap="YlGnBu",
                xticklabels=['Option X', 'Option Y'], yticklabels=['Group 1', 'Group 2'], ax=ax2d)
    ax2d.set_title(f"Chi-Square Contingency Table (Observed)\nChi2: {chi2:.2f} | P-Value: {p_val:.5f}")

    plt.savefig(os.path.join(asset_path, '36_chisq_2d.png'), dpi=300)
    print(f" 2D Asset kaydedildi: {asset_path}\\36_chisq_2d.png")
    plt.close(fig2d)

    # --- ASSET 2: SADECE 3D GRAFİK (36_chisq_3d.png) ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    x = np.linspace(0, 15, 200)
    y_depth = np.linspace(0, 1, 10)
    X_grid, Y_grid = np.meshgrid(x, y_depth)
    Z_grid = stats.chi2.pdf(X_grid, df=dof)

    # Ana Dağılım Yüzeyi
    ax3d.plot_surface(X_grid, Y_grid, Z_grid, cmap='magma', alpha=0.6, edgecolor='none')

    # Red Bölgesi (Kritik bölge Alpha=0.05)
    chi_crit = stats.chi2.ppf(1 - 0.05, df=dof)
    x_red = x[x > chi_crit]
    if len(x_red) > 0:
        X_r, Y_r = np.meshgrid(x_red, y_depth)
        ax3d.plot_surface(X_r, Y_r, stats.chi2.pdf(X_r, df=dof), color='red', alpha=0.8)

    # Gözlemlenen Chi2 (Yeşil Bayrak)
    ax3d.plot([chi2, chi2], [0, 1], [0, stats.chi2.pdf(chi2, df=dof)], color='green', lw=5, label=f'Observed Chi2={chi2:.2f}')

    ax3d.set_title("3D Chi-Square Distribution Decision Space")
    ax3d.set_xlabel('Chi-Square Value')
    ax3d.legend()

    plt.savefig(os.path.join(asset_path, '36_chisq_3d.png'), dpi=300)
    print(f" 3D Asset kaydedildi: {asset_path}\\36_chisq_3d.png")
    plt.close(fig3d)

    # --- DASHBOARD (Preview) ---
```

```
fig_dash, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 9))
sns.heatmap(observed, annot=True, ax=ax1, cmap="YlGnBu")
ax1.set_title("Contingency Preview")

ax2 = fig_dash.add_subplot(122, projection='3d')
ax2.plot_surface(X_grid, Y_grid, Z_grid, cmap='magma', alpha=0.4)
ax2.plot([chi2, chi2], [0, 1], [0, stats.chi2.pdf(chi2, df=dof)], color='green', lw=3)
ax2.set_title("Chi-Square 3D Preview")

plt.suptitle(f"Lesson 36: Chi-Square Test of Independence\nSignificant Relationship: {'YES' if p_val < 0.05 else 'NO'}", fontsize=18)
plt.show()

if __name__ == "__main__":
    run_chisq_master()
```

Ders: 37 Correlation Heatmaps

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\37_correlation_heatmaps

Bölüm 1: README

```
# Lesson 37: Correlation Heatmaps - Relationships in Data

### The Pearson Correlation Coefficient (r)
Correlation measures the strength and direction of the linear relationship between two variables.
- **r = 1**: Perfect positive correlation (Move together).
- **r = -1**: Perfect negative correlation (One goes up, other goes down).
- **r = 0**: No linear relationship.

### The Power of the Heatmap
When dealing with multidimensional data (many columns), checking pairs one by one is impossible. A Correlation Heatmap allows us to see the "big picture" by mapping the correlation matrix into a color-coded grid.

---

### Computational Approach: The Unified Asset Engine
1. 2D Perspective: A professional Seaborn Heatmap with numerical annotations and a diverging color palette (Coolwarm).
2. 3D Perspective: A Correlation Surface, where the "peaks" represent high positive correlation and the "valleys" represent negative correlation.
3. Master Dashboard: A dual-view for spotting hidden patterns instantly.

### Visual Evidence

#### I. 2D Correlation Matrix
High values (red) indicate strong positive relationships, while low values (blue) indicate inverse relationships.
![Correlation Heatmap 2D](../../assets/37_corr_2d.png)

#### II. 3D Relationship Topography
Visualizing the matrix as a 3D landscape where the height represents the strength of the correlation.
![Correlation Heatmap 3D](../../assets/37_corr_3d.png)

**Files:**
* `correlation_master.py`: Unified engine for matrix calculation and separate asset generation.
```

Bölüm 2: Python Kodu

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
import os

def run_correlation_master():
    # --- YOL AYARI (Ph.D. Disiplini) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Veri Üretimi (Sentetik bir Veri Seti: 5 Değişkenli)
    np.random.seed(42)
    n = 100
    x1 = np.random.randn(n)
    x2 = x1 + np.random.normal(0, 0.5, n) # Güçlü Pozitif
    x3 = -x1 + np.random.normal(0, 0.8, n) # Negatif
    x4 = np.random.randn(n) # Alakasız (Gürültü)
    x5 = 0.5 * x2 + np.random.normal(0, 0.2, n) # Orta Pozitif

    df = pd.DataFrame({'Var_A': x1, 'Var_B': x2, 'Var_C': x3, 'Var_D': x4, 'Var_E': x5})
    corr_matrix = df.corr()

    # --- ASSET 1: SADECE 2D GRAFİK (37_corr_2d.png) ---
    fig2d = plt.figure(figsize=(10, 8))
    ax2d = fig2d.add_subplot(111)

    sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f",
                linewidths=0.5, vmin=-1, vmax=1, center=0, ax=ax2d)
    ax2d.set_title("2D Correlation Matrix (Pearson $r$)")

    plt.savefig(os.path.join(asset_path, '37_corr_2d.png'), dpi=300)
    print(f"2D Asset kaydedildi: {asset_path}\\37_corr_2d.png")
    plt.close(fig2d)

    # --- ASSET 2: SADECE 3D GRAFİK (37_corr_3d.png) ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    # Matrisi 3D koordinatlara çevirme
    size = len(corr_matrix)
    x_pos, y_pos = np.meshgrid(np.arange(size), np.arange(size))
    x_pos = x_pos.flatten()
    y_pos = y_pos.flatten()
    z_pos = np.zeros(size * size)

    dx = dy = 0.6
    dz = corr_matrix.values.flatten()

    # Renk haritası (Z değerine göre)
    colors = plt.cm.coolwarm((dz + 1) / 2)

    ax3d.bar3d(x_pos, y_pos, z_pos, dx, dy, dz, color=colors, alpha=0.8)

    ax3d.set_xticks(np.arange(size))
    ax3d.set_xticklabels(df.columns)
    ax3d.set_yticks(np.arange(size))
    ax3d.set_yticklabels(df.columns)
    ax3d.set_title("3D Correlation Topography")
    ax3d.set_zlabel('Correlation Intensity')
```

```
plt.savefig(os.path.join(asset_path, '37_corr_3d.png'), dpi=300)
print(f" 3D Asset kaydedildi: {asset_path}\\37_corr_3d.png")
plt.close(fig3d)

# --- DASHBOARD (Preview) ---
fig_dash = plt.figure(figsize=(20, 10))

ax1 = fig_dash.add_subplot(121)
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', cbar=False, ax=ax1)
ax1.set_title("Matrix View")

ax2 = fig_dash.add_subplot(122, projection='3d')
ax2.bar3d(x_pos, y_pos, z_pos, dx, dy, dz, color=colors)
ax2.set_title("Topographic View")

plt.suptitle("Lesson 37: Multi-Variable Correlation Analysis", fontsize=20)
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    run_correlation_master()
```

Ders: 38 Poisson Distribution

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\38_poisson_distribution

Bölüm 1: README

```
# Lesson 38: Poisson Distribution - The Probability of Occurrence

### Definition
The Poisson Distribution expresses the probability of a given number of events occurring in a fixed interval of time or space, provided these events occur with a known constant mean rate ( $\lambda$ ) and independently of the time since the last event.

### The Mathematical Formula
The probability of observing  $k$  events is given by:

$$P(k; \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}$$

-  $\lambda$ : The average number of events per interval (Expected value).
-  $k$ : The actual number of events we want to find the probability for.
-  $e$ : Euler's number.

---

### Computational Approach: The Unified Asset Engine
1. 2D Perspective: A "Probability Mass Function" (PMF) showing how the peak of the distribution shifts as the average rate ( $\lambda$ ) increases.
2. 3D Perspective: A "Temporal Probability Surface" showing the evolution of probabilities across different time scales and event rates.
3. Master Dashboard: Comparing different  $\lambda$  values to visualize the "normalization" of Poisson.

### Visual Evidence

#### I. 2D PMF Curves
As  $\lambda$  increases, the distribution starts to look more like a Normal (Gaussian) curve.
![Poisson Distribution PMF curves](../../assets/38_poisson_2d.png)

#### II. 3D Poisson Surface
Visualizing how the probability density changes as both the number of events ( $k$ ) and the rate ( $\lambda$ ) evolve together.
![Poisson 3D Probability Surface](../../assets/38_poisson_3d.png)

**Files:**
* `poisson_master.py`: Unified engine for distribution modeling and separate asset generation.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import scipy.stats as stats
import os

def run_poisson_master():
    # --- YOL AYARI ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Hazırlık: Olay sayıları (k) ve Ortalama oranlar (lambda)
    k_range = np.arange(0, 20)
    lambdas = [1, 4, 10]

    # --- ASSET 1: SADECE 2D GRAFİK (38_poisson_2d.png) ---
    fig2d = plt.figure(figsize=(10, 7))
    ax2d = fig2d.add_subplot(111)

    colors = ['#FF5733', '#33FF57', '#3357FF']
    for i, lam in enumerate(lambdas):
        pmf = stats.poisson.pmf(k_range, lam)
        ax2d.step(k_range, pmf, where='mid', label=f' $\lambda = {lam}$ ', color=colors[i], marker='o', alpha=0.7)

    ax2d.set_title("2D Poisson Distribution: Probability Mass Function (PMF)", fontsize=14)
    ax2d.set_xlabel("Number of Events (k)")
    ax2d.set_ylabel("Probability")
    ax2d.legend()
    ax2d.grid(axis='y', linestyle='--', alpha=0.6)

    plt.savefig(os.path.join(asset_path, '38_poisson_2d.png'), dpi=300)
    print(f"2D Asset kaydedildi: {asset_path}\\38_poisson_2d.png")
    plt.close(fig2d)

    # --- ASSET 2: SADECE 3D GRAFİK (38_poisson_3d.png) ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    # 3D Yüzey: Lambda değişimine karşı k olasılıkları
    lam_range = np.linspace(1, 15, 30)
    K, L = np.meshgrid(k_range, lam_range)
    Z = stats.poisson.pmf(K, L)

    surf = ax3d.plot_surface(K, L, Z, cmap='inferno', alpha=0.8, edgecolor='none')

    ax3d.set_title("3D Poisson Probability Surface", fontsize=14)
    ax3d.set_xlabel('Events (k)')
    ax3d.set_ylabel('Mean Rate ( $\lambda$ )')
    ax3d.set_zlabel('Probability')

    plt.savefig(os.path.join(asset_path, '38_poisson_3d.png'), dpi=300)
    print(f"3D Asset kaydedildi: {asset_path}\\38_poisson_3d.png")
    plt.close(fig3d)

    # --- DASHBOARD (Preview) ---
    fig_dash, (ax_l, ax_r) = plt.subplots(1, 2, figsize=(20, 9))

    # Left Preview
    for lam in lambdas:
        ax_l.plot(k_range, stats.poisson.pmf(k_range, lam), '-o', label=f' $\lambda = {lam}$ ')
    ax_l.set_title("Distribution Curves")
    ax_l.legend()
```

```
# Right Preview
ax_r = fig_dash.add_subplot(122, projection='3d')
ax_r.plot_surface(K, L, Z, cmap='inferno', alpha=0.6)
ax_r.set_title("Probability Terrain")

plt.suptitle("Lesson 38: Poisson Distribution Analysis", fontsize=18)
plt.show()

if __name__ == "__main__":
    run_poisson_master()
```

Ders: 39 Bayesian Inference

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\39_bayesian_infer

Bölüm 1: README

```
# Lesson 39: Bayesian Inference - The Power of Priors

### The Bayesian Formula
Bayesian inference uses Bayes' Theorem to update the probability for a hypothesis as more evidence or information becomes available:

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

-  $P(H|E)$  (Posterior): New probability of hypothesis  $H$  after seeing evidence  $E$ .
-  $P(E|H)$  (Likelihood): Probability of seeing evidence  $E$  if hypothesis  $H$  is true.
-  $P(H)$  (Prior): Initial belief about hypothesis  $H$  before seeing any evidence.
-  $P(E)$  (Evidence): Total probability of the evidence.

### The Learning Loop
Bayesian statistics is an iterative process. Today's Posterior becomes tomorrow's Prior as we collect more data.

---

### Computational Approach: The Unified Asset Engine
1. 2D Perspective: A visual comparison of the Prior (what we thought), the Likelihood (what the data says), and the Posterior (our updated belief).
2. 3D Perspective: A "Belief Evolution Surface" showing how the posterior distribution narrows and shifts as the sample size increases.
3. Master Dashboard: A step-by-step visualization of "Learning from Data."

### Visual Evidence

#### I. 2D Bayesian Update
The transition from a broad initial guess (Prior) to a sharpened updated belief (Posterior) after observing data.
![Bayesian Update 2D](../../assets/39_bayesian_2d.png)

#### II. 3D Posterior Evolution
Visualizing how uncertainty collapses into certainty as we gather more evidence over time.
![Bayesian 3D Evolution](../../assets/39_bayesian_3d.png)

**Files:**
* `bayesian_master.py`: Unified engine for Bayesian modeling and separate asset generation.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import scipy.stats as stats
import os

def run_bayesian_master():
    # --- YOL AYARI ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Parametreler ( $\theta$ : Başarı Olasılığı)
    x = np.linspace(0, 1, 500)

    # Prior (Önsel İnanç): Beta(2, 2)
    prior = stats.beta.pdf(x, 2, 2)

    # Evidence (Kanit): 10 denemede 8 başarı
    n, h = 10, 8
    likelihood = stats.binom.pmf(h, n, x)

    # FIX: np.trapz yerine np.trapezoid kullanımı (Numpy 2.0+ uyumluluğu)
    area = np.trapezoid(likelihood, x)
    likelihood_norm = likelihood / area

    # Posterior (Güncellenmiş İnanç): Beta(2+h, 2+n-h)
    posterior = stats.beta.pdf(x, 2 + h, 2 + n - h)

    # --- ASSET 1: 2D (39_bayesian_2d.png) ---
    fig2d, ax2d = plt.subplots(figsize=(10, 7))
    ax2d.plot(x, prior, 'r--', lw=2, label='Prior (Initial Guess)')
    ax2d.plot(x, likelihood_norm, 'g:', lw=2, label='Likelihood (The Evidence)')
    ax2d.plot(x, posterior, 'b-', lw=3, label='Posterior (Updated Belief)')
    ax2d.fill_between(x, 0, posterior, color='blue', alpha=0.2)

    ax2d.set_title("2D Bayesian Learning Process", fontsize=14)
    ax2d.set_xlabel("Probability Parameter ( $\theta$ )")
    ax2d.set_ylabel("Density")
    ax2d.legend()

    plt.savefig(os.path.join(asset_path, '39_bayesian_2d.png'), dpi=300)
    print(f" 2D Asset güncellendi: {asset_path}\\39_bayesian_2d.png")
    plt.close(fig2d)

    # --- ASSET 2: 3D (39_bayesian_3d.png) ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    n_steps = np.arange(1, 41, 2) # Daha geniş bir gözlem aralığı
    X_grid, N_grid = np.meshgrid(x, n_steps)
    # Veri biriktikçe (n arttıkça) belirsizliğin nasıl azaldığını simüle edelim
    Z_grid = stats.beta.pdf(X_grid, 2 + 0.7*N_grid, 2 + 0.3*N_grid)

    surf = ax3d.plot_surface(X_grid, N_grid, Z_grid, cmap='viridis', alpha=0.8, edgecolor='none')

    ax3d.set_title("3D Bayesian Evolution: Uncertainty Collapse", fontsize=14)
    ax3d.set_xlabel('θ Value'); ax3d.set_ylabel('Sample Size (n)'); ax3d.set_zlabel('Density')

    plt.savefig(os.path.join(asset_path, '39_bayesian_3d.png'), dpi=300)
    print(f" 3D Asset güncellendi: {asset_path}\\39_bayesian_3d.png")
    plt.close(fig3d)
```

```
# --- DASHBOARD (Preview) ---
fig_dash = plt.figure(figsize=(18, 8))
ax_l = fig_dash.add_subplot(121)
ax_l.plot(x, prior, 'r--', label='Prior')
ax_l.plot(x, posterior, 'b-', label='Posterior')
ax_l.set_title("Learning Curve (2D)")
ax_l.legend()

ax_r = fig_dash.add_subplot(122, projection='3d')
ax_r.plot_surface(X_grid, N_grid, Z_grid, cmap='viridis', alpha=0.6)
ax_r.set_title("Probability Evolution (3D)")

plt.suptitle("Lesson 39: Bayesian Updating Fixed", fontsize=18)
plt.show()

if __name__ == "__main__":
    run_bayesian_master()
```

Ders: 40 Linear Regression

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\04_stats\40_linear_regression

Bölüm 1: README

Lesson 40: Linear Regression - Predicting the Future

The Best Fit Line

Linear regression models the relationship between a dependent variable (y) and one or more independent variables (x) by fitting a linear equation to observed data.

The Equation of Life

$y = \beta_0 + \beta_1 x + \epsilon$

- β_0 (Intercept): The value of y when $x=0$.

- β_1 (Slope): The change in y for a one-unit change in x .

- ϵ (Error/Residual): The difference between observed and predicted values.

Computational Approach: The Grand Finale Engine

- 2D Perspective:** A classic Scatter Plot with the **Regression Line** and shaded confidence intervals, showing the "fit."
- 3D Perspective:** A **Residual Analysis Surface**. We visualize the "Loss Function" (MSE - Mean Squared Error) as a 3D valley, where our goal is to find the lowest point (the "Optimal Coefficients").
- Master Dashboard:** The ultimate comparison of raw data, the prediction line, and the error distribution.

Visual Evidence

I. 2D Linear Fit

The red line represents our best guess for the relationship. The closer the points are to the line, the higher the R^2 score.

![Linear Regression Fit Line](../../assets/40_regression_2d.png)

II. 3D Loss Function Surface

This is the "Search Space" for our model. The 3D landscape shows how the error change based on our choice of intercept and slope.

![Regression 3D Loss Surface](../../assets/40_regression_3d.png)

Files:

* `regression\_master.py`: Unified engine for predictive modeling and separate asset generation.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.linear_model import LinearRegression
import os

def run_regression_master():
    # --- YOL AYARI ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Veri Üretimi
    np.random.seed(42)
    X = 2 * np.random.rand(100, 1)
    y = 4 + 3 * X + np.random.randn(100, 1) # y = 4 + 3x + noise

    # 2. Model Eğitimi
    model = LinearRegression()
    model.fit(X, y)
    X_new = np.array([[0], [2]])
    y_predict = model.predict(X_new)

    # --- ASSET 1: SADECE 2D GRAFİK (40_regression_2d.png) ---
    fig2d, ax2d = plt.subplots(figsize=(10, 7))
    ax2d.scatter(X, y, color='blue', alpha=0.5, label='Actual Data')
    ax2d.plot(X_new, y_predict, 'r-', lw=3, label='Regression Line')

    ax2d.set_title(f"2D Linear Regression (R²: {model.score(X, y):.4f})", fontsize=14)
    ax2d.set_xlabel("Independent Variable (X)")
    ax2d.set_ylabel("Dependent Variable (y)")
    ax2d.legend()

    plt.savefig(os.path.join(asset_path, '40_regression_2d.png'), dpi=300)
    print(f" 2D Asset kaydedildi: {asset_path}\\40_regression_2d.png")
    plt.close(fig2d)

    # --- ASSET 2: SADECE 3D GRAFİK (40_regression_3d.png) ---
    # Bu grafik "Loss Function" (MSE) manzarasını gösterir
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    intercept_range = np.linspace(2, 6, 50)
    slope_range = np.linspace(1, 5, 50)
    B0, B1 = np.meshgrid(intercept_range, slope_range)

    # Her (intercept, slope) çifti için MSE hesapla (Vektörize)
    # MSE = mean((y - (b0 + b1*x))^2)
    Z_mse = np.array([np.mean((y - (b0 + b1*X))**2) for b0, b1 in zip(np.ravel(B0), np.ravel(B1))])
    Z_mse = Z_mse.reshape(B0.shape)

    surf = ax3d.plot_surface(B0, B1, Z_mse, cmap='terrain', alpha=0.8, edgecolor='none')

    # En iyi noktayı (Optimal Beta) işaretler
    ax3d.scatter(model.intercept_, model.coef_[0], np.mean((y - model.predict(X))**2),
        color='red', s=100, label='Global Minimum (Optimal)')

    ax3d.set_title("3D Loss Function Surface (MSE)", fontsize=14)
    ax3d.set_xlabel('Intercept (β₀)'); ax3d.set_ylabel('Slope (β₁)'); ax3d.set_zlabel('Mean Squared Error')
    ax3d.legend()

    plt.savefig(os.path.join(asset_path, '40_regression_3d.png'), dpi=300)
    print(f" 3D Asset kaydedildi: {asset_path}\\40_regression_3d.png")
```

```
plt.close(fig3d)

# --- DASHBOARD (Preview) ---
fig_dash, (ax_l, ax_r) = plt.subplots(1, 2, figsize=(20, 9))
ax_l.scatter(X, y, alpha=0.4)
ax_l.plot(X_new, y_predict, 'r-')
ax_l.set_title("Prediction Preview")

ax_r = fig_dash.add_subplot(122, projection='3d')
ax_r.plot_surface(B0, B1, Z_mse, cmap='terrain', alpha=0.6)
ax_r.set_title("Error Minimization (Loss Surface)")

plt.suptitle("Lesson 40: Linear Regression - Final Statistical Mastery", fontsize=18)
plt.show()

if __name__ == "__main__":
    run_regression_master()
```

Ders: 41 Direction Fields

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\41_direction

Bölüm 1: README

Lesson 41: Direction Fields - Visualizing Slopes

What is a Direction Field?

For a first-order differential equation $\frac{dy}{dx} = f(x, y)$, a direction field (or slope field) is a graphical representation of the solutions. At each point (x, y) , we draw a small line segment with slope $f(x, y)$.

Why use it?

- **Existence & Uniqueness:** It helps us see if a solution exists for a given starting point.
- **Qualitative Analysis:** We can understand the long-term behavior (equilibrium, stability) of a system without solving the equation analytically.

Computational Approach: The Flow Engine

- 2D Perspective:** A dense grid of normalized arrows (quiver plot) representing the slopes, with a specific "Particular Solution" curve passing through an initial point.
- 3D Perspective:** We treat the slope $f(x, y)$ as a height (Z-axis). This turns the slope field into a "Gradient Terrain," showing where the changes are steepest.
- Master Dashboard:** Comparing the raw slope field with the integral curve.

Visual Evidence

I. 2D Slope Field (Quiver)

The arrows show the direction of the derivative. Any solution to the equation must "follow" these arrows like a car on a road.

![Direction Field 2D](../../assets/41_dirfield_2d.png)

II. 3D Slope Intensity

Visualizing the magnitude of the change. Higher peaks represent areas where the function $y(x)$ changes most rapidly.

![Direction Intensity 3D](../../assets/41_dirfield_3d.png)

Files:

* `direction\_field\_master.py`: Unified engine for slope field generation and asset management.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_direction_field_master():
    # --- YOL AYARI ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Diferansiyel Denklem Tanımı: dy/dx = x - y
    def f(x, y):
        return x - y

    x = np.linspace(-3, 3, 20)
    y = np.linspace(-3, 3, 20)
    X, Y = np.meshgrid(x, y)

    # Eğimleri hesapla
    dy = f(X, Y)
    dx = np.ones(dy.shape)

    # Okları normalize et (Görsel temizlik için)
    norm = np.sqrt(dx**2 + dy**2)
    dx /= norm
    dy /= norm

    # --- ASSET 1: 2D DIRECTION FIELD (41_dirfield_2d.png) ---
    fig2d, ax2d = plt.subplots(figsize=(10, 8))
    ax2d.quiver(X, Y, dx, dy, color='blue', alpha=0.6, pivot='mid')

    # Örnek bir çözüm eğrisi ekleyelim (y = x - 1 + C*e^-x)
    # Başlangıç noktası (0, 1) için: 1 = 0 - 1 + C => C = 2
    x_curve = np.linspace(-3, 3, 100)
    y_curve = x_curve - 1 + 2 * np.exp(-x_curve)
    ax2d.plot(x_curve, y_curve, 'r-', lw=3, label='Particular Solution (0,1)')

    ax2d.set_title(r"2D Direction Field: $\frac{dy}{dx} = x - y$", fontsize=14)
    ax2d.set_xlabel("x"); ax2d.set_ylabel("y")
    ax2d.set_ylim(-3, 3)
    ax2d.legend()
    ax2d.grid(alpha=0.3)

    plt.savefig(os.path.join(asset_path, '41_dirfield_2d.png'), dpi=300)
    print(f" 2D Asset kaydedildi: {asset_path}\\41_dirfield_2d.png")
    plt.close(fig2d)

    # --- ASSET 2: 3D SLOPE TERRAIN (41_dirfield_3d.png) ---
    fig3d = plt.figure(figsize=(10, 8))
    ax3d = fig3d.add_subplot(111, projection='3d')

    # Daha yoğun bir grid ile eğim şiddeti (Z)
    x_fine = np.linspace(-3, 3, 50)
    y_fine = np.linspace(-3, 3, 50)
    X_f, Y_f = np.meshgrid(x_fine, y_fine)
    Z_slope = f(X_f, Y_f)

    surf = ax3d.plot_surface(X_f, Y_f, Z_slope, cmap='twilight', alpha=0.8, edgecolor='none')

    ax3d.set_title("3D Slope Intensity Landscape", fontsize=14)
    ax3d.set_xlabel('x'); ax3d.set_ylabel('y'); ax3d.set_zlabel('dy/dx')
```

```
plt.savefig(os.path.join(asset_path, '41_dirfield_3d.png'), dpi=300)
print(f" 3D Asset kaydedildi: {asset_path}\\41_dirfield_3d.png")
plt.close(fig3d)

# --- DASHBOARD (Preview) ---
fig_dash = plt.figure(figsize=(18, 8))
ax1 = fig_dash.add_subplot(121)
ax1.quiver(X, Y, dx, dy)
ax1.set_title("Flow Logic")

ax2 = fig_dash.add_subplot(122, projection='3d')
ax2.plot_surface(X_f, Y_f, Z_slope, cmap='twilight', alpha=0.6)
ax2.set_title("Slope Magnitude")

plt.suptitle("Lesson 41: Direction Fields & Solution Flow", fontsize=18)
plt.show()

if __name__ == "__main__":
    run_direction_field_master()
```

Ders: 42 Euler Method

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\42_euler_me

Bölüm 1: README

```
# Lesson 42: Euler Method - The First Step into Numerics

### The Core Idea
Euler's Method is a first-order numerical procedure for solving ordinary differential equations (ODEs) with a given initial value. It
uses the tangent line at the current point to estimate the next point.

### The Iterative Formula
Given  $\frac{dy}{dx} = f(x, y)$  and an initial point  $(x_0, y_0)$ :
 $y_{n+1} = y_n + h \cdot f(x_n, y_n)$ 
 $x_{n+1} = x_n + h$ 
-  $h$ : The step size. Smaller  $h$  means better accuracy but more computation.
-  $f(x_n, y_n)$ : The slope at the current point.

---

### Computational Approach: The Error Analysis Engine
1. 2D Perspective: Comparing the Analytical Solution (Exact) with the Euler Approximation (Numerical). We will see how
the "drift" occurs as errors accumulate.
2. 3D Perspective: Visualizing the Error Surface. How does the approximation error grow as a function of the step size ( $h$ )
and total distance from the start?
3. Master Dashboard: Interactive visualization of the step-by-step approximation process.

### Visual Evidence

#### I. 2D Approximation Drift
Observe how the Euler method (dots) tries to follow the curve but gradually deviates due to local truncation errors.
![Euler Method vs Exact](../../assets/42_euler_2d.png)

#### II. 3D Error Accumulation
A 3D landscape where the height (Z) represents the absolute error between the exact solution and Euler's estimation over time.
![Euler 3D Error Surface](../../assets/42_euler_3d.png)

**Files:**
* `euler_method_master.py`: Unified engine for step-by-step approximation and error mapping.
```

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_euler_master_fixed():
    # --- YOL AYARI ---
    asset_path = r'../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Diferansiyel Denklem:  $dy/dx = y$  |  $y(0) = 1$  (Çözüm:  $y = e^x$ )
    def f(x, y): return y
    def exact_sol(x): return np.exp(x)

    x0, y0, x_end, h = 0, 1, 3.5, 0.4 # Adımı büyük tutalım ki hatayı görelim

    # Euler Hesaplama
    steps = int((x_end - x0) / h)
    x_euler = np.zeros(steps + 1)
    y_euler = np.zeros(steps + 1)
    x_euler[0], y_euler[0] = x0, y0

    for i in range(steps):
        y_euler[i+1] = y_euler[i] + h * f(x_euler[i], y_euler[i])
        x_euler[i+1] = x_euler[i] + h

    # --- ASSET 1: 2D APPROXIMATION (32_euler_2d.png) ---
    fig1 = plt.figure(figsize=(10, 7))
    x_fine = np.linspace(x0, x_end, 100)
    plt.plot(x_fine, exact_sol(x_fine), 'b-', lw=2, label='Exact Solution ( $e^x$ )')
    plt.plot(x_euler, y_euler, 'ro--', label=f'Euler Approximation (h={h})')

    # Hata boşluklarını işaretle
    for i in range(len(x_euler)):
        plt.vlines(x_euler[i], y_euler[i], exact_sol(x_euler[i]), colors='gray', linestyle=':', alpha=0.5)

    plt.title("Euler Method: The Drift from Reality", fontsize=14)
    plt.xlabel("x"); plt.ylabel("y")
    plt.legend(); plt.grid(alpha=0.3)
    plt.savefig(os.path.join(asset_path, '42_euler_2d.png'), dpi=300)
    plt.close(fig1)
    print(f"2D Asset kaydedildi: {asset_path}\\42_euler_2d.png")

    # --- ASSET 2: 3D ERROR LANDSCAPE (42_euler_3d.png) ---
    # Bu kısmı tamamen bağımsız bir figürde ve açık bir subplot ile tanımlıyoruz
    fig2 = plt.figure(figsize=(12, 9))
    ax3d = fig2.add_subplot(111, projection='3d')

    h_vals = np.linspace(0.05, 0.5, 25)
    x_vals = np.linspace(0, 3.5, 25)
    H, X = np.meshgrid(h_vals, x_vals)

    # Teorik birikimli hata yüzeyi (Global Truncation Error)
    # Euler hatası yaklaşık olarak  $h * (e^x - 1) / 2$  mertebesinde
    Z_error = np.abs(np.exp(X) - (1 + H)**(X/H))

    surf = ax3d.plot_surface(X, H, Z_error, cmap='viridis', alpha=0.8, edgecolor='none')

    ax3d.set_title("3D Euler Error Accumulation Map", fontsize=14)
    ax3d.set_xlabel('Distance from Start (x)')
    ax3d.set_ylabel('Step Size (h)')
    ax3d.set_zlabel('Absolute Error')
```

```
fig2.colorbar(surf, ax=ax3d, shrink=0.5, aspect=5)

plt.savefig(os.path.join(asset_path, '42_euler_3d.png'), dpi=300)
plt.close(fig2)
print(f" 3D Asset kaydedildi: {asset_path}\\42_euler_3d.png")

# --- DASHBOARD (Visual Confirmation) ---
# Sadece ekranda her şeyin yolunda olduğunu görmek için
print("\n--- Grafik Önizleme Hazırlanıyor ---")
fig_check, (ax_check1, ax_check2) = plt.subplots(1, 2, figsize=(15, 6))
ax_check1.plot(x_fine, exact_sol(x_fine), 'b')
ax_check1.plot(x_euler, y_euler, 'ro--')
ax_check1.set_title("2D Preview")

ax_check2 = fig_check.add_subplot(122, projection='3d')
ax_check2.plot_surface(X, H, Z_error, cmap='viridis')
ax_check2.set_title("3D Preview")
plt.show()

if __name__ == "__main__":
    run_euler_master_fixed()
```

Ders: 43 Runge Kutta

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\43_runge_kutta

Bölüm 1: README

Lesson 43: Runge-Kutta (RK4) - The Gold Standard

The Logic of RK4

While Euler's method uses only the derivative at the beginning of the interval, **RK4** calculates four different slopes (k_1 , k_2 , k_3 , k_4) and takes a weighted average:

- k_1** : Slope at the beginning.
- k_2** : Slope at the midpoint (using k_1).
- k_3** : Slope at the midpoint (using k_2).
- k_4** : Slope at the end (using k_3).

The Mathematical Weighting

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

This cancels out errors up to the fourth order, making it incredibly stable.

Computational Approach: The Precision Engine

- 2D Perspective**: A direct comparison: **Euler vs. RK4 vs. Exact**. You will see RK4 hugging the exact solution while Euler drifts away.
- 3D Perspective**: **Convergence Landscape**. Visualizing how the error drops drastically as we switch from Euler to RK4 across different step sizes.
- Master Dashboard**: Step-by-step weight visualization.

Visual Evidence

I. 2D Accuracy Comparison

Notice that RK4 (green) is almost indistinguishable from the Exact solution (blue), even with large steps.

![[Runge-Kutta vs Euler vs Exact](../../assets/43_rk4_2d.png)]

II. 3D Precision Map

A 3D surface showing the "Error Floor." RK4's error is so small it creates a flat valley compared to Euler's mountains of error.

![[RK4 3D Error Floor](../../assets/43_rk4_3d.png)]

Files:

* `rk4\_master.py`: High-precision solver and asset generator.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_rk4_master_precision_fixed():
    # --- YOL AYARI ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Diferansiyel Denklem:  $dy/dx = y - x^2 + 1$  |  $y(0) = 0.5$ 
    # Çözüm:  $y = (x+1)^2 - 0.5e^x$ 
    def f(x, y): return y - x**2 + 1
    def exact_sol(x): return (x + 1)**2 - 0.5 * np.exp(x)

    x0, y0, x_end, h = 0, 0.5, 4, 0.4 # Adımı biraz küçültelim ki RK4 başarısı parlasın
    steps = int((x_end - x0) / h)

    # Kaplar
    x_vals = np.linspace(x0, x_end, steps + 1)
    y_euler = np.zeros(steps + 1); y_rk4 = np.zeros(steps + 1)
    y_euler[0] = y0

    # Hassas Çözücü Döngüsü
    for i in range(steps):
        curr_x, curr_y_e, curr_y_rk = x_vals[i], y_euler[i], y_rk4[i]

        # --- Euler Adımı ---
        y_euler[i+1] = curr_y_e + h * f(curr_x, curr_y_e)

        # --- RK4 Adımı ---
        k1 = f(curr_x, curr_y_rk)
        k2 = f(curr_x + h/2, curr_y_rk + h*k1/2)
        k3 = f(curr_x + h/2, curr_y_rk + h*k2/2)
        k4 = f(curr_x + h, curr_y_rk + h*k3)
        y_rk4[i+1] = curr_y_rk + (h/6) * (k1 + 2*k2 + 2*k3 + k4)

    # --- ASSET 1: 2D COMPARISON (43_rk4_2d.png) ---
    fig1 = plt.figure(figsize=(10, 7))
    x_fine = np.linspace(x0, x_end, 100)
    plt.plot(x_fine, exact_sol(x_fine), 'b-', lw=3, label='Exact Solution', alpha=0.6)
    plt.plot(x_vals, y_euler, 'ro--', label='Euler Approximation')
    plt.plot(x_vals, y_rk4, 'g*-', lw=2, label='RK4 Approximation')

    plt.title("Numerical Precision: Euler vs. Runge-Kutta (RK4)", fontsize=14)
    plt.xlabel("x"); plt.ylabel("y"); plt.legend(); plt.grid(alpha=0.3)
    plt.savefig(os.path.join(asset_path, '43_rk4_2d.png'), dpi=300)
    plt.close(fig1)
    print(f" 2D Asset kaydedildi: {asset_path}\\43_rk4_2d.png")

    # --- ASSET 2: 3D ERROR FLOOR (43_rk4_3d.png) ---
    # Bu kısmı tamamen bağımsız bir figürde ve açık bir subplot ile tanımlıyoruz
    fig2 = plt.figure(figsize=(12, 9))
    ax3d = fig2.add_subplot(111, projection='3d')

    # H değerleri ve X değerleri üzerinden hata analizi
    h_vals = np.linspace(0.05, 0.5, 25)
    x_vals_fine = np.linspace(0, 4, 25)
    H, X = np.meshgrid(h_vals, x_vals_fine)

    # Teorik birikimli hata yüzeyi (Euler vs RK4)
    # RK4 hatası  $h^4$  ile, Euler hatası  $h^1$  ile orantılıdır
```

```

E_euler = np.abs(np.exp(X) * H * 0.5) # Yaklaşık Euler hatası
E_rk4 = np.abs(np.exp(X) * (H**4) * 0.01) # Yaklaşık RK4 hatası

# Euler hatasını (Red) ve RK4 hatasını (Green) yanyana değil, üst üste çiziyoruz
surf1 = ax3d.plot_surface(X, H, E_euler, cmap='Reds', alpha=0.3, edgecolor='none', label='Euler Error')
surf2 = ax3d.plot_surface(X, H, E_rk4, cmap='Greens', alpha=0.8, edgecolor='none', label='RK4 Error')

ax3d.set_title("3D Error Magnitude: Euler (Red) vs. RK4 (Green)", fontsize=14)
ax3d.set_xlabel('x'); ax3d.set_ylabel('Step Size (h)'); ax3d.set_zlabel('Absolute Error')

# Eksen etiketlerinin düzgün görünmesini sağla
ax3d.xaxis.set_rotate_label(False); ax3d.yaxis.set_rotate_label(False); ax3d.zaxis.set_rotate_label(False)

plt.savefig(os.path.join(asset_path, '43_rk4_3d.png'), dpi=300)
plt.close(fig2)
print(f" 3D Asset kaydedildi: {asset_path}\\43_rk4_3d.png")

# --- DASHBOARD (Visual Confirmation) ---
# Sadece ekranda her şeyin yolunda olduğunu görmek için
print("\n--- Grafik Önizleme Hazırlanıyor ---")
fig_check, (ax_check1, ax_check2) = plt.subplots(1, 2, figsize=(15, 6))
ax_check1.plot(x_fine, exact_sol(x_fine), 'b')
ax_check1.plot(x_vals, y_euler, 'ro--')
ax_check1.plot(x_vals, y_rk4, 'g*-')
ax_check1.set_title("2D Preview")

ax_check2 = fig_check.add_subplot(122, projection='3d')
ax_check2.plot_surface(X, H, E_euler, cmap='Reds', alpha=0.3)
ax_check2.plot_surface(X, H, E_rk4, cmap='Greens', alpha=0.8)
ax_check2.set_title("3D Error Preview")
plt.show()

if __name__ == "__main__":
    run_rk4_master_precision_fixed()

```

Ders: 44 Logistic Growth

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\44_logistic_

Bölüm 1: README

Lesson 44: Logistic Growth - Modeling Population Limits

The Logistic Equation

Unlike simple exponential growth, the logistic model introduces a feedback mechanism that slows growth as the population P approaches the carrying capacity K :

$$\frac{dP}{dt} = rP \left(1 - \frac{P}{K}\right)$$

- r : Intrinsic growth rate.
- K : Carrying capacity (The maximum sustainable population).
- P : Current population.

Equilibrium Points

- $P = 0$: Unstable equilibrium (extinction).
- $P = K$: Stable equilibrium (the system naturally settles here).

Computational Approach: The Biological Engine

- 2D Perspective:** Visualization of the **Sigmoid Curve**. We see how different initial populations ($P_0 < K$ and $P_0 > K$) all converge to the carrying capacity.
- 3D Perspective:** A **Phase-Space Surface**. Visualizing growth rate ($\frac{dP}{dt}$) as a function of both time and population size, showing the "hump" of maximum growth.
- Master Dashboard:** Interactive slider-ready simulation of r and K parameters.

Visual Evidence

I. 2D Logistic S-Curve

The population grows rapidly at first, then stabilizes at the limit K .

![Logistic Growth S-Curve](../assets/44_logistic_2d.png)

II. 3D Growth Dynamics

A 3D landscape showing how the growth rate peaks when the population is at half of its carrying capacity ($K/2$).

![Logistic 3D Surface](../assets/44_logistic_3d.png)

Files:

* `logistic\_master.py`: ODE solver for population dynamics and separate asset generator.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_logistic_master_v2():
    # --- YOL AYARI ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Model Parametreleri
    r = 0.6 # Büyüme hızı
    K = 1000 # Taşıma kapasitesi (Limit)
    t_end = 25
    dt = 0.1
    t_vals = np.arange(0, t_end, dt)

    def dPdt(P): return r * P * (1 - P / K)

    # Hassas RK4 Çözücü
    def solve_rk4(P0):
        P = np.zeros(len(t_vals))
        P[0] = P0
        for i in range(len(t_vals)-1):
            k1 = dPdt(P[i])
            k2 = dPdt(P[i] + dt*k1/2)
            k3 = dPdt(P[i] + dt*k2/2)
            k4 = dPdt(P[i] + dt*k3)
            P[i+1] = P[i] + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)
        return P

    # --- ASSET 1: 2D LOGISTIC S-CURVES (SAVE & SHOW) ---
    fig1 = plt.figure(figsize=(10, 7))
    initial_populations = [50, 500, 1500] # Kapasite altı, orta ve üstü

    for p0 in initial_populations:
        P_sol = solve_rk4(p0)
        plt.plot(t_vals, P_sol, lw=2.5, label=f'Başlangıç P0 = {p0}')

    plt.axhline(K, color='red', linestyle='--', lw=2, label=f'Kapasite (K) = {K}')
    plt.title("Lojistik Büyüme: Taşıma Kapasitesine Yakınsama", fontsize=14)
    plt.xlabel("Zaman (t)"); plt.ylabel("Nüfus (P)")
    plt.legend(); plt.grid(alpha=0.3)

    plt.savefig(os.path.join(asset_path, '44_logistic_2d.png'), dpi=300)
    print(f"2D Asset kaydedildi: {asset_path}\\44_logistic_2d.png")
    plt.show() # Ekranda göster

    # --- ASSET 2: 3D GROWTH RATE SURFACE (SAVE & SHOW) ---
    fig2 = plt.figure(figsize=(12, 9))
    ax3d = fig2.add_subplot(111, projection='3d')

    P_grid = np.linspace(0, 1500, 50)
    T_grid = np.linspace(0, 25, 50)
    PP, TT = np.meshgrid(P_grid, T_grid)
    ZZ_rate = r * PP * (1 - PP / K) # dP/dt yüzeyi

    surf = ax3d.plot_surface(TT, PP, ZZ_rate, cmap='plasma', alpha=0.85, edgecolor='none')

    ax3d.set_title("3D Nüfus Dinamiği: Büyüme Hızı Yüzeyi", fontsize=14)
    ax3d.set_xlabel('Zaman (t)'); ax3d.set_ylabel('Nüfus (P)'); ax3d.set_zlabel('Büyüme Hızı (dP/dt)')
    fig2.colorbar(surf, ax=ax3d, shrink=0.5, aspect=5)
```

```
plt.savefig(os.path.join(asset_path, '44_logistic_3d.png'), dpi=300)
print(f" 3D Asset kaydedildi: {asset_path}\\44_logistic_3d.png")
plt.show() # Ekranda göster

if __name__ == "__main__":
    run_logistic_master_v2()
```

Ders: 45 Lotka Volterra

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\45_lotka_vol

Bölüm 1: README

Lesson 45: Lotka-Volterra Equations - Coupled Systems

The Predator-Prey Equations

This model describes the dynamics of biological systems in which two species interact, one as a predator and the other as prey.

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$

Visualizing the Ecosystem

I. Population Time Series (2D)

This plot shows the oscillating relationship between species. Notice the **phase lag**: the predator population (Red) peaks shortly after the prey population (Blue) peaks.

![Lotka-Volterra 2D](../../assets/45_lotka_2d.png)

II. Phase Space Orbit (3D)

By adding the **Time** axis to the Prey-Predator relationship, we visualize the system's "Limit Cycle" as a 3D trajectory. This shows the recurring nature of the ecosystem's state.

![Lotka-Volterra 3D](../../assets/45_lotka_3d.png)

Key Insights

- **Stability:** The system doesn't collapse; it orbits an equilibrium point.

- **Sensitivity:** Small changes in birth/death rates can dramatically change the amplitude of the oscillations.

Bölüm 2: Python Kodu

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_lotka_volterra_master():
    # --- SENİN YOLUN (FIXED ASSET PATH) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Model Parametreleri
    alpha, beta = 1.1, 0.4 # Av (Prey) parametreleri
    delta, gamma = 0.1, 0.4 # Avcı (Predator) parametreleri

    t_end = 50
    dt = 0.05
    t_vals = np.arange(0, t_end, dt)

    # Bağlı Diferansiyel Denklemler
    def dXdt(x, y): return alpha * x - beta * x * y
    def dYdt(x, y): return delta * x * y - gamma * y

    # RK4 Çözücü (Coupled System için)
    x = np.zeros(len(t_vals)); y = np.zeros(len(t_vals))
    x[0], y[0] = 10, 5 # Başlangıç: 10 Tavşan, 5 Kurt

    for i in range(len(t_vals)-1):
        # RK4 Katsayıları - x için
        kx1 = dXdt(x[i], y[i])
        ky1 = dYdt(x[i], y[i])

        kx2 = dXdt(x[i] + dt*kx1/2, y[i] + dt*ky1/2)
        ky2 = dYdt(x[i] + dt*kx1/2, y[i] + dt*ky1/2)

        kx3 = dXdt(x[i] + dt*kx2/2, y[i] + dt*ky2/2)
        ky3 = dYdt(x[i] + dt*kx2/2, y[i] + dt*ky2/2)

        kx4 = dXdt(x[i] + dt*kx3, y[i] + dt*ky3)
        ky4 = dYdt(x[i] + dt*kx3, y[i] + dt*ky3)

        x[i+1] = x[i] + (dt/6)*(kx1 + 2*kx2 + 2*kx3 + kx4)
        y[i+1] = y[i] + (dt/6)*(ky1 + 2*ky2 + 2*ky3 + ky4)

    # --- ASSET 1: 2D TIME SERIES (45_lotka_2d.png) ---
    fig1 = plt.figure(figsize=(12, 6))
    plt.plot(t_vals, x, 'b-', label='Prey (Tavşan)', lw=2)
    plt.plot(t_vals, y, 'r-', label='Predator (Kurt)', lw=2)
    plt.title("Lotka-Volterra: Zaman İçinde Popülasyon Döngüsü", fontsize=14)
    plt.xlabel("Zaman"); plt.ylabel("Nüfus")
    plt.legend(); plt.grid(alpha=0.3)

    plt.savefig(os.path.join(asset_path, '45_lotka_2d.png'), dpi=300)
    print(f"2D Asset kaydedildi: {asset_path}\\45_lotka_2d.png")
    plt.show()

    # --- ASSET 2: 3D PHASE SPACE (45_lotka_3d.png) ---
    fig2 = plt.figure(figsize=(10, 8))
    ax3d = fig2.add_subplot(111, projection='3d')

    # 3D düzlemde (Zaman, Av, Avcı) ilişkisi
    ax3d.plot(t_vals, x, y, color='purple', lw=2)
    ax3d.set_title("3D Faz Uzayı: Ekosistem Yörüngesi", fontsize=14)

```

```
ax3d.set_xlabel('Zaman'); ax3d.set_ylabel('Av (x)'); ax3d.set_zlabel('Avcı (y)')

# Başlangıç noktasını işaretle
ax3d.scatter(t_vals[0], x[0], y[0], color='green', s=100, label='Başlangıç')
ax3d.legend()

plt.savefig(os.path.join(asset_path, '45_lotka_3d.png'), dpi=300)
print(f" 3D Asset kaydedildi: {asset_path}\\45_lotka_3d.png")
plt.show()

if __name__ == "__main__":
    run_lotka_volterra_master()
```

Ders: 46 Heat Equation

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\46_heat_equation

Bölüm 1: README

Lesson 46: 1D Heat Equation - Modeling Thermal Diffusion

The Partial Differential Equation (PDE)

The heat equation describes how heat spreads through a material over time. In 1D:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

- $u(x, t)$: Temperature at position x and time t .
- α : Thermal diffusivity constant.
- $\frac{\partial^2 u}{\partial x^2}$: The curvature of the temperature profile (Laplacian).

Physical Interpretation

Heat flows from hotter regions to colder regions at a rate proportional to the local temperature gradient's curvature. Over time, the temperature distribution "flattens" out.

Visualizing Heat Diffusion

I. Temperature Evolution (2D)

This plot shows snapshots of the temperature along the rod at different time intervals. Notice how the sharp initial peak gradually smooths out and spreads.

![Heat Equation 2D](../../assets/46_heat_2d.png)

II. Space-Time Heat Map (3D)

A 3D surface where the X-axis is position, the Y-axis is time, and the Z-axis (height) is temperature. This visualizes the entire cooling/heating process as a landscape.

![Heat Equation 3D](../../assets/46_heat_3d.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_heat_equation_master():
    # --- SENİN YOLUN (FIXED ASSET PATH) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Simülasyon Parametreleri
    L = 1.0      # Çubuğun boyu
    nx = 50      # Uzay adımı sayısı
    dx = L / (nx - 1)
    alpha = 0.01 # Isıl yayılım katsayısı
    dt = 0.001   # Zaman adımı (Stabilite için küçük seçilmeli)
    nt = 1000    # Toplam zaman adımı

    # Başlangıç Koşulu: Çubuğun ortasında bir sıcaklık piki (Delta function approximation)
    u = np.zeros(nx)
    u[int(0.4*nx):int(0.6*nx)] = 100.0 # Orta kısım 100 derece

    # Tüm veriyi saklamak için matris (3D için)
    u_history = np.zeros((nt, nx))
    u_history[0, :] = u.copy()

    # 2. Sayısal Çözüm (Finite Difference Method)
    for t in range(1, nt):
        un = u.copy()
        for i in range(1, nx - 1):
            # Isı denklemini ayrıklaştırma (FTCS Method)
            u[i] = un[i] + alpha * dt / dx**2 * (un[i+1] - 2*un[i] + un[i-1])
        u_history[t, :] = u.copy()

    # --- ASSET 1: 2D TEMPERATURE PROFILES (46_heat_2d.png) ---
    fig1 = plt.figure(figsize=(10, 7))
    time_snapshots = [0, 50, 200, 500, 999]
    colors = plt.cm.hot(np.linspace(0.2, 0.8, len(time_snapshots)))

    for i, t_idx in enumerate(time_snapshots):
        plt.plot(np.linspace(0, L, nx), u_history[t_idx, :],
                 color=colors[i], lw=2, label=f'Time Step: {t_idx}')

    plt.title("1D Isı Yayılımı: Zamanla Sıcaklık Değişimi", fontsize=14)
    plt.xlabel("Çubuk Boyu (x)")
    plt.ylabel("Sıcaklık (u)")
    plt.legend(); plt.grid(alpha=0.3)

    plt.savefig(os.path.join(asset_path, '46_heat_2d.png'), dpi=300)
    print(f"2D Asset kaydedildi: {asset_path}\\46_heat_2d.png")
    plt.show()

    # --- ASSET 2: 3D SPACE-TIME SURFACE (46_heat_3d.png) ---
    fig2 = plt.figure(figsize=(12, 9))
    ax3d = fig2.add_subplot(111, projection='3d')

    X = np.linspace(0, L, nx)
    T = np.linspace(0, nt*dt, nt)
    X_mesh, T_mesh = np.meshgrid(X, T)

    surf = ax3d.plot_surface(X_mesh, T_mesh, u_history, cmap='hot', alpha=0.9, edgecolor='none')

    ax3d.set_title("3D Isı Yayılım Manzarası (PDE Solution)", fontsize=14)
```

```
ax3d.set_xlabel('Konum (x)'); ax3d.set_ylabel('Zaman (t)'); ax3d.set_zlabel('Sıcaklık (u)')
fig2.colorbar(surf, ax=ax3d, shrink=0.5, aspect=5)

# Açığı daha iyi bir görüş için ayarla
ax3d.view_init(elev=30, azim=135)

plt.savefig(os.path.join(asset_path, '46_heat_3d.png'), dpi=300)
print(f" 3D Asset kaydedildi: {asset_path}\\46_heat_3d.png")
plt.show()

if __name__ == "__main__":
    run_heat_equation_master()
```

Ders: 47 Harmonic Motion

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\47_harmonic

Bölüm 1: README

Lesson 47: Simple Harmonic Motion (SHM) - The Rhythm of Physics

The Second-Order Differential Equation

SHM occurs when the restoring force is directly proportional to the displacement:

$$\frac{d^2x}{dt^2} = -kx$$

Which simplifies to:

$$\frac{d^2x}{dt^2} + \omega^2x = 0$$

where $\omega = \sqrt{k/m}$ is the angular frequency.

Energy Conservation

In an ideal SHM system, energy oscillates between **Kinetic Energy** and **Potential Energy**, but the total mechanical energy remains constant.

Visualizing the Oscillation

I. Displacement vs. Time (2D)

A perfect sine/cosine wave showing how the position $x(t)$ evolves. Notice the periodic nature and the constant amplitude in an undamped system.

![Simple Harmonic Motion 2D](../../assets/47_shm_2d.png)

II. Phase Space Portrait (3D)

By plotting **Position**, **Velocity**, and **Time**, we visualize the motion as a 3D spiral. The projection on the Position-Velocity plane creates a perfect circle (or ellipse), representing the conservation of energy.

![Simple Harmonic Motion 3D](../../assets/47_shm_3d.png)

Bölüm 2: Python Kodu

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_shm_master():
    # --- SENİN YOLUN (FIXED ASSET PATH) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Fiziksel Parametreler
    k = 5.0    # Yay sabiti
    m = 1.0    # Kütle
    omega_sq = k / m

    t_end = 20
    dt = 0.05
    t_vals = np.arange(0, t_end, dt)

    # State Vector: [position, velocity]
    # d/dt [x, v] = [v, -omega^2 * x]
    def deriv(state):
        x, v = state
        return np.array([v, -omega_sq * x])

    # 2. RK4 Çözücü
    state = np.array([2.0, 0.0]) # Başlangıç: 2 birim çekilmiş, hız 0
    history = np.zeros((len(t_vals), 2))
    history[0] = state

    for i in range(len(t_vals)-1):
        k1 = deriv(state)
        k2 = deriv(state + dt*k1/2)
        k3 = deriv(state + dt*k2/2)
        k4 = deriv(state + dt*k3)
        state = state + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)
        history[i+1] = state

    x_out = history[:, 0]
    v_out = history[:, 1]

    # --- ASSET 1: 2D POSITION & VELOCITY (47_shm_2d.png) ---
    fig1 = plt.figure(figsize=(12, 6))
    plt.plot(t_vals, x_out, 'b-', lw=2, label='Konum (x)')
    plt.plot(t_vals, v_out, 'r--', lw=1.5, label='Hız (v)', alpha=0.7)
    plt.title("Basit Harmonik Hareket: Zaman Serisi", fontsize=14)
    plt.xlabel("Zaman (t)"); plt.ylabel("Genlik")
    plt.axhline(0, color='black', lw=1)
    plt.legend(); plt.grid(alpha=0.3)

    plt.savefig(os.path.join(asset_path, '47_shm_2d.png'), dpi=300)
    print(f"2D Asset kaydedildi: {asset_path}\\47_shm_2d.png")
    plt.show()

    # --- ASSET 2: 3D PHASE SPIRAL (47_shm_3d.png) ---
    fig2 = plt.figure(figsize=(10, 8))
    ax3d = fig2.add_subplot(111, projection='3d')

    ax3d.plot(t_vals, x_out, v_out, color='purple', lw=2)
    ax3d.set_title("3D Faz Sarmalı: Zaman-Konum-Hız", fontsize=14)
    ax3d.set_xlabel('Zaman (t)'); ax3d.set_ylabel('Konum (x)'); ax3d.set_zlabel('Hız (v)')

```

```
# Faz diyagramı projeksiyonunu zemine ekle (Enerji korunumu dairesi)
ax3d.plot(t_vals*0 + t_end, x_out, v_out, color='gray', alpha=0.3, label='Enerji Korunumu (Yörünge)')

plt.savefig(os.path.join(asset_path, '47_shm_3d.png'), dpi=300)
print(f" 3D Asset kaydedildi: {asset_path}\\47_shm_3d.png")
plt.show()

if __name__ == "__main__":
    run_shm_master()
```

Ders: 48 Fourier Transform

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\48_fourier_t

Bölüm 1: README

Lesson 48: Fourier Transform - Decomposing Signals

The Core Concept

The Fourier Transform $F(\omega)$ converts a signal from the time domain $f(t)$ to the frequency domain. It tells us "how much" of each frequency exists in the original signal.

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-i\omega t} dt$$

Discrete Fourier Transform (DFT)

In the digital world, we use the Fast Fourier Transform (FFT) algorithm to compute this efficiently. It allows us to identify hidden patterns, noise, and dominant frequencies in data.

Visualizing the Spectrum

I. Composite Signal (2D)

A complex-looking wave created by summing multiple sine waves of different frequencies. This is what we "see" in the time domain.

![Fourier Transform time vs frequency domain](../../assets/48_fourier_2d.png)

II. The Frequency Landscape (3D)

A 3D visualization showing the "Evolution of Spectrum." We plot **Time**, **Frequency**, and **Magnitude** to see how the frequency content of a signal might change or how individual components stand out as distinct "peaks."

![Fourier Transform 3D spectral analysis](../../assets/48_fourier_3d.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft, fftfreq
import os

def run_fourier_master():
    # --- SENİN YOLUN (FIXED ASSET PATH) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Sinyal Oluşturma (Karmaşık bir dalga)
    fs = 1000 # Örnekleme frekansı
    T = 1.0 # Süre (saniye)
    t = np.linspace(0, T, fs, endpoint=False)

    # İki farklı frekansın birleşimi: 50Hz ve 120Hz + Gürültü
    sig = 0.5 * np.sin(2 * np.pi * 50 * t) + 0.8 * np.sin(2 * np.pi * 120 * t)

    # 2. FFT Hesaplama
    n = len(t)
    yf = fft(sig)
    xf = fftfreq(n, 1/fs)[:n//2]
    magnitude = 2.0/n * np.abs(yf[0:n//2])

    # --- ASSET 1: TIME DOMAIN VS FREQUENCY DOMAIN (48_fourier_2d.png) ---
    fig1, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
    fig1.subplots_adjust(hspace=0.4)

    ax1.plot(t[:200], sig[:200], 'b')
    ax1.set_title("Zaman Domaini (İlk 200 Örnek)")
    ax1.set_xlabel("Zaman (s)"); ax1.set_ylabel("Genlik")

    ax2.plot(xf, magnitude, 'r')
    ax2.set_title("Frekans Domaini (Magnitude Spectrum)")
    ax2.set_xlabel("Frekans (Hz)"); ax2.set_ylabel("Güç")
    ax2.set_xlim(0, 250) # İlgi alanımız
    ax2.grid(alpha=0.3)

    plt.savefig(os.path.join(asset_path, '48_fourier_2d.png'), dpi=300)
    print(f" 2D Asset kaydedildi: {asset_path}\\48_fourier_2d.png")
    plt.show()

    # --- ASSET 2: 3D SPECTRAL PEAKS (48_fourier_3d.png) ---
    fig2 = plt.figure(figsize=(10, 8))
    ax3d = fig2.add_subplot(111, projection='3d')

    # 3D'de frekansları net görmek için bir 'Waterfall' benzeri yapı kuralım
    freq_grid = xf[(xf >= 0) & (xf <= 200)]
    mag_grid = magnitude[:len(freq_grid)]

    # Zaman ekseninde kopyalayarak bir yüzey oluşturalım
    time_steps = np.linspace(0, 1, 10)
    F, T_mesh = np.meshgrid(freq_grid, time_steps)
    M = np.tile(mag_grid, (len(time_steps), 1))

    surf = ax3d.plot_surface(F, T_mesh, M, cmap='viridis', alpha=0.8, edgecolor='none')

    ax3d.set_title("3D Frekans Spektrumu: Baskın Bileşenler", fontsize=14)
    ax3d.set_xlabel('Frekans (Hz)'); ax3d.set_ylabel('Zaman Dilimi'); ax3d.set_zlabel('Güç')
    ax3d.view_init(elev=30, azim=45)

    plt.savefig(os.path.join(asset_path, '48_fourier_3d.png'), dpi=300)
```

```
print(f" 3D Asset kaydedildi: {asset_path}\\48_fourier_3d.png")
plt.show()

if __name__ == "__main__":
    run_fourier_master()
```

Ders: 49 Laplace Transform

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\49_laplace_t

Bölüm 1: README

Lesson 49: Laplace Transform - System Stability & s-Domain

The Transformation

The Laplace Transform converts a time-domain function $f(t)$ into a complex frequency domain function $F(s)$:

$$F(s) = \int_0^{\infty} f(t) e^{-st} dt$$

where $s = \sigma + j\omega$ is a complex number.

Poles and Zeros

- **Poles:** The values of s where $F(s) \rightarrow \infty$. They determine the **stability** of the system.

- **Zeros:** The values of s where $F(s) = 0$.

If any pole has a positive real part (lies on the right half-side of the s-plane), the system is **unstable**.

Visualizing the s-Domain

I. Impulse Response (2D)

How the system reacts to a sudden shock in the time domain. We compare a stable (decaying) versus an unstable (growing) response.

![Laplace Transform impulse response](../../assets/49_laplace_2d.png)

II. The Pole-Zero Surface (3D)

A 3D landscape of $|F(s)|$ over the complex s-plane. The "tent poles" sticking up to infinity show exactly where the system's natural frequencies and stability limits lie.

![Laplace Transform pole zero plot 3D](../../assets/49_laplace_3d.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_laplace_master():
    # --- SENİN YOLUN (FIXED ASSET PATH) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Zaman Domaini: Impulse Response of a 2nd Order System
    #  $H(s) = 1 / (s^2 + 2\zeta\omega_n s + \omega_n^2)$ 
    t = np.linspace(0, 10, 500)
    zeta = 0.2 # Damping ratio (Underdamped)
    wn = 2.0 # Natural frequency

    # Zaman domaini karşılığı (Damped Sine)
    wd = wn * np.sqrt(1 - zeta**2)
    h_t = (1/wd) * np.exp(-zeta * wn * t) * np.sin(wd * t)

    # --- ASSET 1: TIME DOMAIN RESPONSE (49_laplace_2d.png) ---
    fig1 = plt.figure(figsize=(10, 6))
    plt.plot(t, h_t, 'b-', lw=2, label='Impulse Response $h(t)$')
    plt.axhline(0, color='black', lw=1)
    plt.title("Sistemin Zaman Domaini Tepkisi (Stable Oscillation)", fontsize=14)
    plt.xlabel("Zaman (t)"); plt.ylabel("Genlik")
    plt.grid(alpha=0.3); plt.legend()

    plt.savefig(os.path.join(asset_path, '49_laplace_2d.png'), dpi=300)
    print(f" 2D Asset kaydedildi: {asset_path}\\49_laplace_2d.png")
    plt.show()

    # --- ASSET 2: 3D POLE SURFACE (49_laplace_3d.png) ---
    fig2 = plt.figure(figsize=(12, 9))
    ax3d = fig2.add_subplot(111, projection='3d')

    # s-plane grid (sigma and omega)
    sigma = np.linspace(-2, 1, 100)
    omega = np.linspace(-4, 4, 100)
    S_real, S_imag = np.meshgrid(sigma, omega)

    #  $H(s) = 1 / ((s - p1)(s - p2))$  nerede  $p = -\zeta\omega_n \pm j\omega_d$ 
    p1 = complex(-zeta*wn, wd)
    p2 = complex(-zeta*wn, -wd)

    # Magnitude of Transfer Function  $|H(s)|$ 
    #  $s = S_{real} + jS_{imag}$ 
    dist_p1 = np.sqrt((S_real - p1.real)**2 + (S_imag - p1.imag)**2)
    dist_p2 = np.sqrt((S_real - p2.real)**2 + (S_imag - p2.imag)**2)
    Z_mag = 1.0 / (dist_p1 * dist_p2)

    # Görselleştirme için piki sınırla (Sonsuza gitmesin)
    Z_mag = np.clip(Z_mag, 0, 5)

    surf = ax3d.plot_surface(S_real, S_imag, Z_mag, cmap='terrain', alpha=0.85, edgecolor='none')

    ax3d.set_title("3D s-Düzlemi: Transfer Fonksiyonu Genliği  $|H(s)|$ ", fontsize=14)
    ax3d.set_xlabel(r'Real ( $\sigma$ ) - Stability'); ax3d.set_ylabel(r'Imaginary ( $j\omega$ ) - Freq')
    ax3d.set_zlabel(r' $|H(s)|$ ')

    # Kritik stabilite çizgisi (sigma = 0)
    ax3d.plot([0, 0], [-4, 4], [0, 0], color='red', lw=3, label='Stability Boundary')
```

```
fig2.colorbar(surf, ax=ax3d, shrink=0.5, aspect=5)
plt.savefig(os.path.join(asset_path, '49_laplace_3d.png'), dpi=300)
print(f" 3D Asset kaydedildi: {asset_path}\\49_laplace_3d.png")
plt.show()

if __name__ == "__main__":
    run_laplace_master()
```

Ders: 50 Pid Control

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\05_differential\50_pid_control

Bölüm 1: README

Lesson 50: PID Control - The Grand Finale of Stage 5

The Control Law

A PID controller continuously calculates an error value $e(t)$ as the difference between a desired setpoint and a measured process variable:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

- **Proportional (P):** Corrects based on current error.
- **Integral (I):** Corrects based on accumulated past errors (eliminates steady-state error).
- **Derivative (D):** Corrects based on the predicted future error (dampens oscillations).

Visualizing the Controller's Performance

I. Step Response (2D)

The battle between the controller and physics. We see how the system reaches the target (setpoint), handles overshoot, and finally settles into stability.

! [PID Control step response curve] (../../assets/50_pid_2d.png)

II. Error Surface & Parameter Tuning (3D)

A 3D landscape showing the **Total Settling Error** as a function of K_p and K_d . We are looking for the "valley" - the optimal parameters for the perfect controller.

! [PID Control parameter optimization 3D] (../../assets/50_pid_3d.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_pid_master():
    # --- SENİN YOLUN (FIXED ASSET PATH) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Sistem ve Kontrol Parametreleri
    dt = 0.01
    t_end = 10
    t = np.arange(0, t_end, dt)
    setpoint = 1.0 # Hedef değer

    # PID Katsayıları
    Kp, Ki, Kd = 15.0, 5.0, 1.5

    # 2. Simülasyon Fonksiyonu
    def simulate_pid(kp_in, ki_in, kd_in):
        curr_pos = 0.0
        curr_vel = 0.0
        integral = 0.0
        prev_error = setpoint - curr_pos

        pos_history = []
        for _ in t:
            error = setpoint - curr_pos
            integral += error * dt
            derivative = (error - prev_error) / dt

            # Kontrol sinyali
            u = kp_in * error + ki_in * integral + kd_in * derivative

            # Basit fizik motoru (F=ma, m=1)
            accel = u - 0.5 * curr_vel # Hava direnci ekleyelim
            curr_vel += accel * dt
            curr_pos += curr_vel * dt

            pos_history.append(curr_pos)
            prev_error = error
        return np.array(pos_history)

    # --- ASSET 1: STEP RESPONSE (50_pid_2d.png) ---
    fig1 = plt.figure(figsize=(10, 6))
    response = simulate_pid(Kp, Ki, Kd)
    plt.plot(t, response, 'b-', lw=2.5, label='Sistem Yanıtı (PID)')
    plt.axhline(setpoint, color='r', linestyle='--', label='Hedef (Setpoint)')
    plt.title(f"PID Kontrol: Adım Yanıtı (Kp={Kp}, Ki={Ki}, Kd={Kd})", fontsize=14)
    plt.xlabel("Zaman (s)"); plt.ylabel("Pozisyon")
    plt.legend(); plt.grid(alpha=0.3)

    plt.savefig(os.path.join(asset_path, '50_pid_2d.png'), dpi=300)
    print(f"2D Asset kaydedildi: {asset_path}\\50_pid_2d.png")
    plt.show()

    # --- ASSET 2: TUNING ERROR LANDSCAPE (50_pid_3d.png) ---
    fig2 = plt.figure(figsize=(12, 9))
    ax3d = fig2.add_subplot(111, projection='3d')

    kp_vals = np.linspace(1, 30, 20)
```

```
kd_vals = np.linspace(0, 5, 20)
KP, KD = np.meshgrid(kp_vals, kd_vals)

# Hata Yüzeyini Hesapla (MSE of response)
Z_error = np.zeros_like(KP)
for i in range(len(kd_vals)):
    for j in range(len(kp_vals)):
        res = simulate_pid(KP[i,j], 0.0, KD[i,j]) # Ki=0 basitleştirmesi
        Z_error[i,j] = np.mean((setpoint - res)**2)

surf = ax3d.plot_surface(KP, KD, Z_error, cmap='coolwarm', alpha=0.8, edgecolor='none')
ax3d.set_title("3D Hata Manzarası: Parametre Optimizasyonu", fontsize=14)
ax3d.set_xlabel('Proportional Gain (Kp)'); ax3d.set_ylabel('Derivative Gain (Kd)')
ax3d.set_zlabel('Mean Squared Error')

fig2.colorbar(surf, ax=ax3d, shrink=0.5, aspect=5)
plt.savefig(os.path.join(asset_path, '50_pid_3d.png'), dpi=300)
print(f" 3D Asset kaydedildi: {asset_path}\\50_pid_3d.png")
plt.show()

if __name__ == "__main__":
    run_pid_master()
```

Ders: 51 Linear Transformations

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\51_linear

Bölüm 1: README

Lesson 51: The Law of Large Numbers (LLN) - The Deterministic Anchor

Theoretical Framework

The **Law of Large Numbers** is a fundamental theorem in probability that describes the result of performing the same experiment a large number of times. According to the law, the average of the results obtained from a large number of trials should be close to the **expected value** (μ), and will tend to become closer as more trials are performed.

Mathematical Definition:

Given a sequence of independent and identically distributed (i.i.d.) random variables $X_1, X_2, \dots$ with $E[X_i] = \mu$:

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow[n \rightarrow \infty]{} \mu$$

This is the "Strong Law," asserting that the convergence happens with probability 1.

Visual Evidence & Numerical Analysis

In this module, we simulate thousands of Bernoulli trials (coin flips) to witness the transition from **Stochastic Chaos** to **Statistical Certainty**.

I. 2D Convergence Analysis (The Vibration of Truth)

This plot illustrates the cumulative average of a single experiment.

- At low n , the variance is high (the "vibration").
- As n approaches 2000, the average "anchors" to the theoretical probability of 0.5.

![LLN 2D Convergence](../../assets/51_lln_2d.png)

II. 3D Stability Landscape (The Collective Geometry)

By running 10 experiments simultaneously and plotting them as a 3D surface, we visualize the **Law of Large Numbers** as a stabilizing terrain.

- **Z-Axis:** The cumulative average.
- **X-Axis:** Number of trials.
- **Observation:** Notice how the "mountainous" chaos at the beginning of the trials flattens into a "plateau" at the 0.5 level.

![LLN 3D Landscape](../../assets/51_lln_3d.png)

Ph.D. Insights: Why This Matters

- The Fallacy of Small Numbers:** Many data analysts fail because they interpret patterns in small datasets. LLN proves that patterns below a certain n are merely noise.
- Insurance & Finance:** This law is the reason insurance companies can stay profitable; individual risks are chaotic, but the collective risk is predictable.

Bölüm 2: Python Kodu

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

def run_lln_master():
    # --- PROFESYONEL YOL AYARI (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Deney Parametreleri
    n_trials = 2000
    n_experiments = 30 # 3D için birden fazla deney
    expected_value = 0.5 # Adil bir madeni para için

    # 2. Veri Üretimi
    # Her sütun ayrı bir deneyi temsil eder
    results = np.random.randint(0, 2, size=(n_trials, n_experiments))
    cumulative_averages = np.cumsum(results, axis=0) / np.arange(1, n_trials + 1)[:, None]

    # --- ASSET 1: 2D CONVERGENCE (51_lln_2d.png) ---
    fig1 = plt.figure(figsize=(10, 6))
    plt.plot(cumulative_averages[:, 0], color='blue', lw=1.5, label='Örneklem Ortalaması')
    plt.axhline(y=expected_value, color='red', linestyle='--', label=f'Teorik Beklenen Değer ({expected_value})')
    plt.title("Büyük Sayılar Yasası: Yakınsama Analizi", fontsize=14)
    plt.xlabel("Deney Sayısı (n)"); plt.ylabel("Kümülatif Ortalama")
    plt.grid(alpha=0.3); plt.legend()

    plt.savefig(os.path.join(asset_path, '51_lln_2d.png'), dpi=300)
    print(f" 2D Asset kaydedildi: {asset_path}/51_lln_2d.png")
    plt.show()

    # --- ASSET 2: 3D STABILITY LANDSCAPE (51_lln_3d.png) ---
    fig2 = plt.figure(figsize=(12, 8))
    ax3d = fig2.add_subplot(111, projection='3d')

    X = np.arange(1, n_trials + 1)
    Y = np.arange(n_experiments)
    X_mesh, Y_mesh = np.meshgrid(X, Y)
    Z = cumulative_averages.T # Deneyleri yüzey olarak çiziyoruz

    surf = ax3d.plot_surface(X_mesh, Y_mesh, Z, cmap='viridis', alpha=0.8, edgecolor='none')

    # Beklenen değer düzlemini çizelim
    ax3d.plot_surface(X_mesh, Y_mesh, np.full_like(Z, expected_value),
                     color='red', alpha=0.2, label='Teorik Limit')

    ax3d.set_title("3D LLN Manzarası: Stokastik Dalgalandıran Kararlılığa", fontsize=14)
    ax3d.set_xlabel('Deney Sayısı (n)'); ax3d.set_ylabel('Deney No'); ax3d.set_zlabel('Ortalama')
    fig2.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)

    plt.savefig(os.path.join(asset_path, '51_lln_3d.png'), dpi=300)
    print(f" 3D Asset kaydedildi: {asset_path}/51_lln_3d.png")
    plt.show()

if __name__ == "__main__":
    run_lln_master()

```

Ders: 52 Clt Dynamics

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\52_clt_dyn

Bölüm 1: README

Lesson 52: Central Limit Theorem (CLT) - The Universal Emergence

The Mathematical Miracle

The **Central Limit Theorem** establishes that, in many situations, for independent and identically distributed (i.i.d.) random variables, the properly normalized sum tends toward a **normal distribution** even if the original variables themselves are not normally distributed.

Formal Statement:

If $X_1, X_2, \dots, X_n$ are i.i.d. with mean μ and variance σ^2 , then as $n \rightarrow \infty$:

$$\frac{\bar{X}_n - \mu}{\sigma / \sqrt{n}} \xrightarrow{d} N(0, 1)$$

Visualizing the Emergence

In this lesson, we start with a **Uniform Distribution** (a flat, non-normal shape) and witness its evolution into a perfect Gaussian curve through sampling.

I. The Evolution of Distribution (2D)

We compare the original "flat" distribution with the distribution of its sample means. Notice how the "shoulders" of the Gaussian curve form as the sample size n increases.

![CLT 2D Distribution Evolution](../../assets/52_clt_2d.png)

II. The Probability Landscape (3D)

A 3D visualization showing the **Sampling Distribution** over different sample sizes. It reveals the "sharpening" of the bell curve: as n grows, the variance shrinks, and the "bell" becomes taller and thinner.

![CLT 3D Probability Surface](../../assets/52_clt_3d.png)

Ph.D. Insights: The Power of Inference

- Universal Applicability:** CLT is why we can use Z-tests and T-tests on real-world data even when we don't know the underlying distribution.
- Error Analysis:** It explains why measurement errors in physics and engineering often follow a normal distribution—they are the sum of many small, independent random factors.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
import os

def run_clt_master():
    # --- PROFESYONEL YOL AYARI (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Simülasyon Parametreleri
    n_samples_list = [1, 2, 5, 30] # Farklı örneklem büyüklükleri
    n_simulations = 10000          # Kaç kez örneklem alacağız?

    # Başlangıç: Uniform (Düz) Dağılım [0, 1] -> Mu=0.5, Var=1/12
    data_source = np.random.uniform(0, 1, (n_simulations, max(n_samples_list)))

    # --- ASSET 1: 2D EVOLUTION (52_clt_2d.png) ---
    fig1 = plt.figure(figsize=(12, 8))
    colors = ['#3498db', '#e67e22', '#2ecc71', '#e74c3c']

    for i, n in enumerate(n_samples_list):
        # Her simülasyon için n tanesinin ortalamasını al
        means = np.mean(data_source[:, :n], axis=1)

        plt.subplot(2, 2, i+1)
        plt.hist(means, bins=50, density=True, color=colors[i], alpha=0.6,
                 label=f'Sample Size n={n}')

        # Teorik Normal Dağılım Eğrisi (Fit)
        mu_mean = 0.5
        sigma_mean = np.sqrt((1/12) / n)
        x_axis = np.linspace(0, 1, 100)
        plt.plot(x_axis, norm.pdf(x_axis, mu_mean, sigma_mean), 'k--', lw=1.5)

        plt.title(f'n = {n} iken Dağılım")
        plt.legend()

    plt.suptitle("Merkezi Limit Teoremi: Kaostan Normal Dağılıma Geçiş", fontsize=16)
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.savefig(os.path.join(asset_path, '52_clt_2d.png'), dpi=300)
    print(f"2D Asset kaydedildi: {asset_path}/52_clt_2d.png")
    plt.show()

    # --- ASSET 2: 3D PROBABILITY SURFACE (52_clt_3d.png) ---
    fig2 = plt.figure(figsize=(12, 9))
    ax3d = fig2.add_subplot(111, projection='3d')

    # 3D için n değerlerini daha sık seçelim
    n_range = np.arange(1, 41, 2)
    x_bins = np.linspace(0.2, 0.8, 60)
    X_grid, N_grid = np.meshgrid(x_bins, n_range)
    Z_density = np.zeros_like(X_grid)

    for i, n in enumerate(n_range):
        means = np.mean(np.random.uniform(0, 1, (5000, n)), axis=1)
        hist, _ = np.histogram(means, bins=x_bins, density=True)
        Z_density[i, :-1] = hist

    surf = ax3d.plot_surface(X_mesh := X_grid, Y_mesh := N_grid, Z_density,
                             cmap='magma', alpha=0.9, edgecolor='none')
```

```
ax3d.set_title("3D CLT Manzarası: Örneklem Büyüklüğü ve Yoğunluk", fontsize=14)
ax3d.set_xlabel('Örneklem Ortalaması'); ax3d.set_ylabel('Örneklem Sayısı (n)'); ax3d.set_zlabel('Olasılık Yoğunluğu')
fig2.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)
```

```
# Açığı çan eğrilerinin yükselişini görecektir şekilde ayarla
ax3d.view_init(elev=25, azim=-60)
```

```
plt.savefig(os.path.join(asset_path, '52_clt_3d.png'), dpi=300)
print(f" 3D Asset kaydedildi: {asset_path}/52_clt_3d.png")
plt.show()
```

```
if __name__ == "__main__":
    run_clt_master()
```

Ders: 53 Covariance Surfaces

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\53_covar

Bölüm 1: README

\# Lesson 53: Covariance & Correlation - Mapping Dependencies

The Mathematical Structure

Covariance measures the direction of the linear relationship between two random variables.

$$\text{Cov}(X, Y) = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{n-1}$$

Correlation (r) is the normalized version of covariance, ranging from -1 to 1, providing a scale-independent measure of strength:

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

Visualizing the Dependency Landscape

In this lesson, we generate synthetic datasets with varying degrees of correlation to visualize how the "Joint Probability Density" shifts in 3D space.

I. Correlation Matrix & Scatter Dynamics (2D)

A multi-plot analysis showing how different correlation coefficients ($r = 0.9, 0, -0.9$) transform the shape of the data cloud from a tight line to a random mist.

![Covariance 2D Scatter Maps](../../assets/53_cov_2d.png)

II. The Bivariate Gaussian Surface (3D)

This is the "Visual Feast." We plot the **Joint PDF** (Probability Density Function) of two correlated variables.

|- **High Correlation:** The surface stretches into a sharp "mountain ridge" along the diagonal.

|- \[*Zero Correlation:\]* The surface becomes a perfectly symmetrical "circular bell."

![[Correlation 3D Probability Surface](.././../assets/53_cov_3d.png)]

_

Ph.D. Insights: Beyond the Coefficient

1). **The Shape of Information:** High covariance means knowing X gives significant information about Y . In 3D, this is seen as a reduction in the "spread" of the surface.

2). **Feature Redundancy:** In data analysis, highly correlated features (the "ridges" we see in 3D) often indicate redundancy, which is the starting point for Dimensionality Reduction (PCA).

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal
import os

def run_covariance_master():
    # --- PROFESSIONAL PATH SETUP (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Generate Correlated Data
    def generate_correlated_data(r, n=500):
        mean = [0, 0]
        cov = [[1, r], [r, 1]] # Variance = 1, Covariance = r
        return np.random.multivariate_normal(mean, cov, n)

    # --- ASSET 1: 2D CORRELATION SPECTRUM (53_cov_2d.png) ---
    correlations = [0.95, 0.0, -0.95]
    fig1, axes = plt.subplots(1, 3, figsize=(15, 5))

    for i, r in enumerate(correlations):
        data = generate_correlated_data(r)
        axes[i].scatter(data[:, 0], data[:, 1], alpha=0.5, s=15, color='#2c3e50')
        axes[i].set_title(f"Correlation r = {r}", fontsize=12)
        axes[i].set_xlabel("Variable X"); axes[i].set_ylabel("Variable Y")
        axes[i].grid(True, linestyle='--', alpha=0.6)
        axes[i].set_xlim(-4, 4); axes[i].set_ylim(-4, 4)

    plt.suptitle("Linear Dependency Patterns in 2D Space", fontsize=15)
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.savefig(os.path.join(asset_path, '53_cov_2d.png'), dpi=300)
    print(f" 2D Asset saved: {asset_path}/53_cov_2d.png")
    plt.show()

    # --- ASSET 2: 3D BIVARIATE PDF SURFACE (53_cov_3d.png) ---
    fig2 = plt.figure(figsize=(12, 9))
    ax3d = fig2.add_subplot(111, projection='3d')

    # Create grid for the surface
    x = np.linspace(-3, 3, 100)
    y = np.linspace(-3, 3, 100)
    X, Y = np.meshgrid(x, y)
    pos = np.dstack((X, Y))

    # Define a high-correlation bivariate normal distribution
    rv = multivariate_normal([0, 0], [[1, 0.8], [0.8, 1]])
    Z = rv.pdf(pos)

    surf = ax3d.plot_surface(X, Y, Z, cmap='coolwarm', alpha=0.85, edgecolor='none')

    ax3d.set_title("3D Bivariate Gaussian: The Geometry of High Correlation", fontsize=14)
    ax3d.set_xlabel("Variable X"); ax3d.set_ylabel("Variable Y")
    ax3d.set_zlabel("Probability Density")
    fig2.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)

    # Adjust view to see the 'ridge' clearly
    ax3d.view_init(elev=25, azim=45)

    plt.savefig(os.path.join(asset_path, '53_cov_3d.png'), dpi=300)
    print(f" 3D Asset saved: {asset_path}/53_cov_3d.png")
    plt.show()
```

```
if __name__ == "__main__":  
    run_covariance_master()
```

Ders: 54 Pca Analysis

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\54_pca_a

Bölüm 1: README

\# Lesson 54: Principal Component Analysis (PCA) - Structural Insights

\### The Mathematical Essence

PCA is an orthogonal linear transformation that identifies the directions (Principal Components) along which the variance of the data is maximized. It allows us to represent high-dimensional data in a 2D or 3D space while preserving the maximum amount of "information."

\---

\### Real-World Applications \& Examples

PCA is not just a theoretical tool; it is the backbone of modern data science. Here is how we use it in the real world:

\#### 1. Facial Recognition (Eigenfaces)

In computer vision, a digital image of a face has thousands of pixels (dimensions). PCA reduces these thousands of dimensions to a few dozen "Eigenfaces" that capture the essential features (eyes, nose, jawline).

* **Use Case:** Matching a face in a security database quickly by comparing only the top 50 components instead of 10,000 pixels.

\#### 2. Genetic Research (Population Genetics)

Human DNA has millions of genetic markers. Researchers use PCA to project this massive data into a 2D plot.

* **Example:** When PCA is applied to the genetic data of Europeans, the resulting 2D plot surprisingly mirrors the actual geographic map of Europe, showing how genetics correlate with location.

\#### 3. Financial Markets \& Stock Portfolios

A stock market has thousands of moving tickers. PCA helps analysts find "Market Factors" (e.g., the overall health of the Tech sector) that explain the movement of hundreds of individual stocks simultaneously.

Use Case: Portfolio risk management by identifying hidden correlations.

Visualizing Information Compression

In this module, we simulate a 3D data cloud and perform the following analysis:

I. Principal Components in 3D Space

We visualize the original 3D data cloud along with the **Eigenvectors**.

- **PC1 (Red):** Points in the direction of the highest spread.

- **PC2 (Green):** Orthogonal to PC1, capturing the second most significant variance.

![PCA 3D Eigenvectors](../../assets/54_pca_3d.png)

II. 2D Projection (The "Shadow" of Data)

By projecting the 3D cloud onto the PC1-PC2 plane, we create a 2D summary. Even though we dropped a dimension, the high **Variance Explained Ratio** confirms that we haven't lost the core story of the data.

![PCA 2D Projection](../../assets/54_pca_2d.png)

Ph.D. Insights: The Logic of Projection

- **Dimensionality Reduction:** It's like looking at the shadow of an object. A well-placed light (PCA) ensures the shadow tells you almost everything about the object's shape.

- **Noise Filtering:** Components with very small eigenvalues often represent random noise. Dropping them actually "cleans" the data.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import os

def run_pca_master():
    # --- PROFESSIONAL PATH SETUP (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Create a Synthetic 3D Dataset with a clear 'main direction'
    np.random.seed(42)
    n_points = 300
    x = np.random.normal(0, 1, n_points)
    y = 0.8 * x + np.random.normal(0, 0.2, n_points)
    z = 0.5 * x + 0.5 * y + np.random.normal(0, 0.1, n_points)
    data = np.vstack([x, y, z]).T

    # 2. Standardize and Run PCA
    scaler = StandardScaler()
    data_scaled = scaler.fit_transform(data)

    pca = PCA(n_components=3)
    pca.fit(data_scaled)
    data_pca = pca.transform(data_scaled)

    # --- ASSET 1: 3D DATA CLOUD & EIGENVECTORS (54_pca_3d.png) ---
    fig1 = plt.figure(figsize=(10, 8))
    ax3d = fig1.add_subplot(111, projection='3d')

    ax3d.scatter(data_scaled[:,0], data_scaled[:,1], data_scaled[:,2], alpha=0.4, color='gray')

    # Plot Eigenvectors (Principal Components)
    colors = ['r', 'g', 'b']
    for i, (length, vector) in enumerate(zip(pca.explained_variance_, pca.components_)):
        v = vector * 3 * np.sqrt(length) # Scale for visibility
        ax3d.quiver(0, 0, 0, v[0], v[1], v[2], color=colors[i], lw=3, label=f'PC{i+1}')

    ax3d.set_title("PCA 3D: Identifying Principal Components (Eigenvectors)", fontsize=14)
    ax3d.set_xlabel("X (Standardized)"); ax3d.set_ylabel("Y"); ax3d.set_zlabel("Z")
    ax3d.legend()

    plt.savefig(os.path.join(asset_path, '54_pca_3d.png'), dpi=300)
    print(f" 3D Asset saved: {asset_path}/54_pca_3d.png")
    plt.show()

    # --- ASSET 2: 2D PROJECTION (54_pca_2d.png) ---
    fig2 = plt.figure(figsize=(10, 6))
    plt.scatter(data_pca[:, 0], data_pca[:, 1], c=data_pca[:, 0], cmap='viridis', alpha=0.7)

    var_exp = pca.explained_variance_ratio_
    plt.title(f"PCA 2D Projection (Explains {sum(var_exp[:2])*100:.1f}% of Variance)", fontsize=14)
    plt.xlabel(f"Principal Component 1 ({var_exp[0]*100:.1f}%)")
    plt.ylabel(f"Principal Component 2 ({var_exp[1]*100:.1f}%)")
    plt.grid(alpha=0.3)
    plt.colorbar(label='PC1 Magnitude')

    plt.savefig(os.path.join(asset_path, '54_pca_2d.png'), dpi=300)
    print(f" 2D Asset saved: {asset_path}/54_pca_2d.png")
    plt.show()
```

```
if __name__ == "__main__":  
    run_pca_master()
```

Ders: 55 Mle Estimation

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\55_mle_e

Bölüm 1: README

\# Lesson 55: Maximum Likelihood Estimation (MLE)

\### The Mathematical Essence

Maximum Likelihood Estimation (MLE) is a method of estimating the parameters of a probability distribution by maximizing a *likelihood function*, so that, under the assumed statistical model, the observed data is most probable.

The Likelihood Function:

Given data x , the likelihood of parameter θ is:

$$\mathcal{L}(\theta | x) = P(x | \theta)$$

In practice, we maximize the *Log-Likelihood* because it simplifies calculations for exponential families:

$$\ell(\theta) = \sum \log P(x_i | \theta)$$

\---

\### Real-World Applications & Examples

\#### 1. Machine Learning (Logistic Regression)

When you train a classifier to distinguish between spam and ham, the algorithm uses MLE to find the weights (parameters) that make the observed labels most likely given the input features.

\#### 2. Biomedical Research (Dosage Response)

In clinical trials, researchers use MLE to estimate the "ED50" (effective dose for 50% of the population) by fitting a distribution to the observed patient recovery data.

3. Econometrics (Consumer Behavior)

Economists use MLE to estimate parameters in discrete choice models—for example, predicting the likelihood of a consumer choosing a specific brand based on price and features.

Visualizing the Likelihood Surface

In this module, we simulate a set of data points from a Normal Distribution and "search" for the best μ (mean) and σ (standard deviation).

I. The Log-Likelihood Curve (2D)

A plot showing how the likelihood changes as we vary one parameter (e.g., μ). The "peak" of this curve is our MLE estimate—the most logical explanation for our data.

![MLE 2D Likelihood Curve](../../assets/55\mle_2d.png)

II. The Likelihood Landscape (3D)

A 3D surface plot where the X and Y axes represent μ and σ , and the Z axis represents the Total Likelihood. We visualize the "Optimization Peak" that the algorithm must climb.

![MLE 3D Surface Optimization](../../assets/55\mle_3d.png)

Ph.D. Insights: The "Frequentist" Philosophy

- **Objective Probability:** Unlike Bayesian methods, MLE assumes there is a fixed "true" parameter in nature, and we are just finding the value that makes our specific sample most plausible.

- **Asymptotic Normality:** One of the beauties of MLE is that as the sample size grows, the distribution of the MLE estimate itself becomes Normal (thanks to CLT!), allowing for easy confidence interval calculations.

Bölüm 2: Python Kodu

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
import os

def run_mle_master():
    # --- PROFESSIONAL PATH SETUP (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Generate synthetic data (The "Observed" Reality)
    true_mu = 5.0
    true_sigma = 2.0
    data = np.random.normal(true_mu, true_sigma, 100)

    # 2. Log-Likelihood Function for Normal Distribution
    def log_likelihood(mu, sigma, data):
        if sigma <= 0: return -np.inf
        return np.sum(norm.logpdf(data, mu, sigma))

    # --- ASSET 1: 2D LOG-LIKELIHOOD CURVE (55_mle_2d.png) ---
    mu_range = np.linspace(true_mu - 2, true_mu + 2, 100)
    ll_values = [log_likelihood(m, true_sigma, data) for m in mu_range]

    fig1 = plt.figure(figsize=(10, 6))
    plt.plot(mu_range, ll_values, color='#e74c3c', lw=2)
    plt.axvline(true_mu, color='black', linestyle='--', label=f'True Mu = {true_mu}')
    plt.axvline(mu_range[np.argmax(ll_values)], color='green', linestyle=':', label='MLE Estimate')

    plt.title("2D Log-Likelihood Search for Mean (mu)", fontsize=14)
    plt.xlabel("Candidate mu Value"); plt.ylabel("Log-Likelihood")
    plt.legend(); plt.grid(alpha=0.3)

    plt.savefig(os.path.join(asset_path, '55_mle_2d.png'), dpi=300)
    plt.show()

    # --- ASSET 2: 3D LIKELIHOOD SURFACE (55_mle_3d.png) ---
    fig2 = plt.figure(figsize=(12, 9))
    ax3d = fig2.add_subplot(111, projection='3d')

    mu_space = np.linspace(true_mu - 1.5, true_mu + 1.5, 50)
    sigma_space = np.linspace(true_sigma - 1, true_sigma + 1, 50)
    MU, SIGMA = np.meshgrid(mu_space, sigma_space)

    # Calculate Z (Log-Likelihood) for each pair
    Z_ll = np.array([[log_likelihood(m, s, data) for m in mu_space] for s in sigma_space])

    surf = ax3d.plot_surface(MU, SIGMA, Z_ll, cmap='viridis', alpha=0.8, edgecolor='none')

    ax3d.set_title("3D MLE Landscape: Finding the Optimal (mu, sigma)", fontsize=14)
    ax3d.set_xlabel("Mean (mu)"); ax3d.set_ylabel("Std Dev (sigma)")
    ax3d.set_zlabel("Log-Likelihood")
    fig2.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)

    ax3d.view_init(elev=30, azim=210)

    plt.savefig(os.path.join(asset_path, '55_mle_3d.png'), dpi=300)
    print(f"3D Asset saved: {asset_path}/55_mle_3d.png")
    plt.show()

if __name__ == "__main__":
    run_mle_master()

```

Ders: 56 Gradient Descent

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\56_gradient_descent

Bölüm 1: README

\# Lesson 56: Gradient Descent Dynamics

The Mathematical Essence

Gradient Descent is an iterative first-order optimization algorithm used to find the local minimum of a differentiable function. It works by taking steps proportional to the negative of the gradient of the function at the current point.

The Update Rule:

$$\theta_{next} = \theta_{curr} - \eta \cdot \nabla J(\theta)$$

\- **\nabla J(\theta)** The Gradient (vector of partial derivatives).

\- **\eta* (Eta):** The Learning Rate (how big of a step we take).

\---

Real-World Applications & Examples

1. Neural Network Training (Backpropagation)

When a neural network learns to recognize a cat, it uses Gradient Descent to adjust millions of weights to minimize the "loss" (the difference between its guess and the truth).

2. Logistics & Supply Chain

Companies use optimization algorithms to minimize the fuel cost of delivery trucks. The "cost function" represents fuel consumption, and Gradient Descent helps find the optimal route and speed.

3. Economic Modeling

Central banks use similar optimization techniques to find interest rate levels that minimize the "misery index" (a combination of inflation and unemployment).

\---

\### Visualizing the Optimization Path

\#### I. Convergence Plot (2D)

We see the "loss" (error) decreasing over each iteration. It shows how the algorithm quickly drops at first and then slowly approaches the global minimum.

![[Gradient Descent 2D Convergence](../../assets/56_gd_2d.png)]

\#### II. The 3D Loss Surface & Trajectory

This is the "Visual Feast." We visualize a 3D "bowl" (Cost Function) and watch our "ball" (the parameters) roll down the slopes until it reaches the bottom of the valley.

![[Gradient Descent 3D Optimization Path](../../assets/56_gd_3d.png)]

\---

\### Ph.D. Insights: The Challenge of the Surface

\- **Learning Rate Sensitivity:** If η is too high, the ball "overshoots" and bounces out of the valley. If it's too low, it takes forever to reach the bottom.

\- **Local Minima vs. Global Minima:** In complex non-convex functions (like Deep Learning), Gradient Descent might get stuck in a "pothole" (local minimum) instead of the deepest valley.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_gd_master():
    # --- PROFESSIONAL PATH SETUP (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Cost Function Definition:  $J(w) = w^2$  (Simple Convex Bowl)
    def cost_function(w):
        return w**2

    def gradient(w):
        return 2 * w

    # 2. Gradient Descent Simulation Parameters
    w = 4.0      # Initial starting point
    eta = 0.1    # Learning rate
    iterations = 30

    path_w = [w]
    path_cost = [cost_function(w)]

    # Optimization Loop
    for _ in range(iterations):
        w = w - eta * gradient(w)
        path_w.append(w)
        path_cost.append(cost_function(w))

    # --- ASSET 1: 2D CONVERGENCE PROGRESS (56_gd_2d.png) ---
    plt.figure(figsize=(10, 6))
    plt.plot(range(len(path_cost)), path_cost, 'o-', color='#2980b9', lw=2, label='Cost J(w)')
    plt.title("2D Convergence: Cost Reduction via Gradient Descent", fontsize=14)
    plt.xlabel("Iteration Step")
    plt.ylabel("Cost Value")
    plt.grid(alpha=0.3)
    plt.legend()

    # Critical Fix: Save command for 2D
    save_path_2d = os.path.join(asset_path, '56_gd_2d.png')
    plt.savefig(save_path_2d, dpi=300)
    print(f"2D Asset saved: {save_path_2d}")
    plt.show()

    # --- ASSET 2: 3D OPTIMIZATION TRAJECTORY (56_gd_3d.png) ---
    fig = plt.figure(figsize=(12, 9))
    ax3d = fig.add_subplot(111, projection='3d')

    # Surface Data:  $J(w_1, w_2) = w_1^2 + w_2^2$ 
    w1_vals = np.linspace(-5, 5, 100)
    w2_vals = np.linspace(-5, 5, 100)
    W1, W2 = np.meshgrid(w1_vals, w2_vals)
    Z = W1**2 + W2**2

    # Draw the objective surface
    surf = ax3d.plot_surface(W1, W2, Z, cmap='terrain', alpha=0.6, edgecolor='none')

    # Generate a synchronized 3D path for visualization
    # Starting from (4, -4) down to (0, 0)
    p_w1 = np.linspace(4, 0, 25)
    p_w2 = np.linspace(-4, 0, 25)
```

```
p_z = p_w1**2 + p_w2**2

ax3d.plot(p_w1, p_w2, p_z, 'r-o', markersize=5, lw=3, label='Descent Path')

ax3d.set_title("3D Gradient Descent: Navigating the Cost Landscape", fontsize=14)
ax3d.set_xlabel("Parameter W1")
ax3d.set_ylabel("Parameter W2")
ax3d.set_zlabel("Cost J(w)")
ax3d.legend()

# Optimal view angle for the 'bowl'
ax3d.view_init(elev=35, azim=120)

save_path_3d = os.path.join(asset_path, '56_gd_3d.png')
plt.savefig(save_path_3d, dpi=300)
print(f" 3D Asset saved: {save_path_3d}")
plt.show()

if __name__ == "__main__":
    run_gd_master()
```

Ders: 57 Eigen Analysis

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\57_eigen

Bölüm 1: README

\# Lesson 57: Eigenvalues & Eigenvectors

\### The Mathematical Essence

An *eigenvector* of a square matrix A is a non-zero vector v that, when multiplied by A , yields a scaled version of itself. The scaling factor is the *eigenvalue* λ .

The Characteristic Equation:

$$Av = \lambda v \implies (A - \lambda I)v = 0$$

To find λ , we solve the determinant equation:

$$\det(A - \lambda I) = 0$$

\---

\### Real-World Applications & Examples

\#### 1. Google's PageRank Algorithm

The importance of a webpage is determined by the "Dominant Eigenvector" of the world wide web's hyperlink matrix. Google essentially solves an eigenvalue problem to rank your search results.

\#### 2. Structural Engineering (Resonance)

Architects use eigenvalues to find the "Natural Frequencies" of a building. If an earthquake's frequency matches an eigenvalue, the building resonates and collapses. Engineers design to keep eigenvalues away from danger zones.

\#### 3. Quantum Mechanics

The energy states of an atom are represented by the eigenvalues of a mathematical operator called the Hamiltonian. Chemistry is, in a sense, the study of these eigenvalues.

\---

\### Visualizing the Transformation

\#### I. Linear Transformation Mapping (2D)

We visualize how a matrix "stretches" space. We plot original unit vectors and their transformed versions. The vectors that don't change their span (direction) are our Eigenvectors.

![Eigen 2D Transformation](../../assets/57_eigen_2d.png)

\#### II. The Eigen-Value Surface (3D)

A 3D visualization of the quadratic form $v^T A v$. This surface reveals the "principal axes" of the matrix, showing where the transformation is strongest and weakest.

![Eigen 3D Surface](../../assets/57_eigen_3d.png)

\---

\### Ph.D. Insights: The Essence of Stability

\- **Decomposition:** Eigenvectors provide a new "coordinate system" where the matrix acts diagonally. This makes complex systems easy to solve.

\- **Spectral Radius:** The largest eigenvalue determines the growth or decay of a system over time. If $|\lambda| > 1$, the system explodes.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_eigen_master():
    # --- PROFESSIONAL PATH SETUP (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Define a 2x2 Transformation Matrix
    A = np.array([[2, 1],
                  [1, 2]])

    # Calculate Eigenvalues and Eigenvectors
    eigenvalues, eigenvectors = np.linalg.eig(A)

    # --- ASSET 1: 2D TRANSFORMATION (57_eigen_2d.png) ---
    plt.figure(figsize=(8, 8))

    # Plot Unit Circle to see the stretch
    theta = np.linspace(0, 2*np.pi, 100)
    circle = np.array([np.cos(theta), np.sin(theta)])
    transformed_circle = A @ circle

    plt.plot(circle[0], circle[1], 'k--', alpha=0.3, label='Original Unit Circle')
    plt.plot(transformed_circle[0], transformed_circle[1], 'b-', lw=2, label='Transformed Space')

    # Plot Eigenvectors
    colors = ['r', 'g']
    for i in range(len(eigenvalues)):
        v = eigenvectors[:, i] * eigenvalues[i]
        plt.quiver(0, 0, v[0], v[1], color=colors[i], angles='xy', scale_units='xy', scale=1,
                  width=0.015, label=f'Eigenvector {i+1} ( $\lambda={eigenvalues[i]:.1f}$ )')

    plt.title("2D Linear Transformation & Eigenvectors", fontsize=14)
    plt.xlabel("X-Axis"); plt.ylabel("Y-Axis")
    plt.grid(alpha=0.3); plt.axhline(0, color='black', lw=1); plt.axvline(0, color='black', lw=1)
    plt.legend(); plt.axis('equal')

    save_path_2d = os.path.join(asset_path, '57_eigen_2d.png')
    plt.savefig(save_path_2d, dpi=300)
    plt.show()

    # --- ASSET 2: 3D QUADRATIC FORM SURFACE (57_eigen_3d.png) ---
    fig = plt.figure(figsize=(12, 9))
    ax3d = fig.add_subplot(111, projection='3d')

    x = np.linspace(-3, 3, 50)
    y = np.linspace(-3, 3, 50)
    X, Y = np.meshgrid(x, y)

    # Compute Quadratic Form:  $Z = v^T \cdot A \cdot v$ 
    # For  $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ ,  $Z = 2x^2 + 2xy + 2y^2$ 
    Z = A[0,0]*X**2 + (A[0,1] + A[1,0])*X*Y + A[1,1]*Y**2

    surf = ax3d.plot_surface(X, Y, Z, cmap='plasma', alpha=0.8, edgecolor='none')

    ax3d.set_title("3D Quadratic Form Surface: Principal Axes of A", fontsize=14)
    ax3d.set_xlabel("X"); ax3d.set_ylabel("Y"); ax3d.set_zlabel(" $v^T A v$ ")
    fig.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)

    ax3d.view_init(elev=25, azim=45)
```

```
save_path_3d = os.path.join(asset_path, '57_eigen_3d.png')
plt.savefig(save_path_3d, dpi=300)
print(f" 3D Asset saved: {save_path_3d}")
plt.show()

if __name__ == "__main__":
    run_eigen_master()
```

Ders: 58 Markov Chains

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\58_markov_chains

Bölüm 1: README

\# Lesson 58: Markov Chains - Stochastic Transitions

\### The Mathematical Essence

A **Markov Chain** is a stochastic model describing a sequence of possible events in which the probability of each event depends only on the state attained in the previous event.

The Transition Matrix (P):

If a system has n states, P is an $n \times n$ matrix where P_{ij} is the probability of moving from state i to state j .

- Each row must sum to 1.

- **Steady State (π):** The long-term probability distribution where $\pi P = \pi$.

\---

\### Real-World Applications & Examples

\#### 1. Weather Forecasting

A simple 2-state model (Sunny, Rainy). If it's sunny today, there's a 70% chance it's sunny tomorrow. Markov Chains help calculate the probability of it being rainy 10 days from now.

\#### 2. Natural Language Processing (Next Word Prediction)

Your smartphone's keyboard uses Markov-like models to predict the next word. It looks at the current word you typed and suggests the most likely "next state" (word) based on frequency data.

\#### 3. Finance & Credit Scoring

Banks use Markov models to predict the probability of a customer moving from "On-time Payer" to "Default" over several months, helping them manage financial risk.

\---

\### Visualizing the Flow

\#### I. State Transition Dynamics (2D)

We track how the probability of being in each state evolves over "time steps." We witness the system converging from an initial guess to a stable **Steady State**.

![[Markov 2D Transition Flow](../../assets/58_markov_2d.png)]

\#### II. The Probability Simplex (3D)

For a 3-state system, all possible probability distributions lie on a triangle (simplex). We visualize the trajectory of the system as it "walks" towards its equilibrium point in 3D space.

![[Markov 3D Simplex Path](../../assets/58_markov_3d.png)]

\---

\### Ph.D. Insights: The Power of Equilibrium

\- **Ergodicity** If a chain is ergodic, it will reach the same steady state regardless of where it starts. It's the "destiny" of the system.

\- **Eigen-Link** The steady state of a Markov Chain is actually the eigenvector of the transition matrix corresponding to the eigenvalue $\lambda = 1$. (A beautiful bridge to Lesson 57!)

Bölüm 2: Python Kodu

```

import numpy as np
import matplotlib.pyplot as plt
import os

def run_markov_master():
    # --- PROFESSIONAL PATH SETUP (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Define Transition Matrix (3x3 - e.g., Sunny, Cloudy, Rainy)
    # Rows sum to 1
    P = np.array([[0.7, 0.2, 0.1],
                  [0.3, 0.4, 0.3],
                  [0.2, 0.3, 0.5]])

    # 2. Simulation over 20 steps
    n_steps = 20
    state_dist = np.array([1.0, 0.0, 0.0]) # Start with 100% Sunny
    history = [state_dist]

    for _ in range(n_steps):
        state_dist = state_dist @ P
        history.append(state_dist)

    history = np.array(history)

    # --- ASSET 1: 2D EVOLUTION TO STEADY STATE (58_markov_2d.png) ---
    plt.figure(figsize=(10, 6))
    plt.plot(history[:, 0], 'o-', label='State 1 (Sunny)', color='#f1c40f')
    plt.plot(history[:, 1], 'o-', label='State 2 (Cloudy)', color='#95a5a6')
    plt.plot(history[:, 2], 'o-', label='State 3 (Rainy)', color='#3498db')

    plt.title("2D Markov Chain: Convergence to Steady State", fontsize=14)
    plt.xlabel("Time Step (n)"); plt.ylabel("Probability")
    plt.grid(alpha=0.3); plt.legend()

    save_path_2d = os.path.join(asset_path, '58_markov_2d.png')
    plt.savefig(save_path_2d, dpi=300)
    print(f" 2D Asset saved: {save_path_2d}")
    plt.show()

    # --- ASSET 2: 3D SIMPLEX TRAJECTORY (58_markov_3d.png) ---
    fig = plt.figure(figsize=(12, 9))
    ax3d = fig.add_subplot(111, projection='3d')

    # Plot the trajectory in 3D probability space
    ax3d.plot(history[:, 0], history[:, 1], history[:, 2], 'r-o', markersize=6, lw=2, label='System Path')

    # Add a visual 'Simplex' plane (x+y+z = 1)
    x_s = [1, 0, 0, 1]
    y_s = [0, 1, 0, 0]
    z_s = [0, 0, 1, 0]
    ax3d.plot(x_s, y_s, z_s, 'k--', alpha=0.4, label='Probability Simplex boundary')

    ax3d.set_title("3D Markov Trajectory: Walking toward Equilibrium", fontsize=14)
    ax3d.set_xlabel("P(State 1)"); ax3d.set_ylabel("P(State 2)"); ax3d.set_zlabel("P(State 3)")
    ax3d.legend()

    ax3d.view_init(elev=20, azim=30)

    save_path_3d = os.path.join(asset_path, '58_markov_3d.png')
    plt.savefig(save_path_3d, dpi=300)

```

```
print(f" 3D Asset saved: {save_path_3d}")
plt.show()

if __name__ == "__main__":
    run_markov_master()
```

Ders: 59 Bayesian Inference

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\59_bayes

Bölüm 1: README

\# Lesson 59: Bayesian Inference - Dynamic Knowledge Updating

\### The Mathematical Essence

Bayesian Inference is a method of statistical inference in which Bayes' theorem is used to update the probability for a hypothesis as more evidence or information becomes available.

Bayes' Theorem:

$$P(\theta | D) = \frac{P(D | \theta) P(\theta)}{P(D)}$$

\- * $P(\theta | D)$ (Posterior):* What we believe after seeing the data.

\- * $P(D | \theta)$ (Likelihood):* How well the data fits our hypothesis.

\- * $P(\theta)$ (Prior):* What we believed before seeing the data.

\- * $P(D)$ (Evidence):* The total probability of the data.

\---

\### Real-World Applications & Examples

\#### 1. Medical Diagnosis

If a patient tests positive for a rare disease, a doctor doesn't just look at the test accuracy. They use *Bayesian Inference* to combine the test result with the disease's "Prior" prevalence in the population to find the *true* probability of infection.

\#### 2. Spam Filtering (Naive Bayes)

Your email client has a "prior" idea of which words are spammy. Every time you mark an email as spam, it updates its "posterior" beliefs, becoming smarter at filtering future junk mail.

3. Search and Rescue

When a ship goes missing at sea, rescuers use Bayesian models to create a "probability map." As they search areas and find nothing (new data), they update the map to focus on more likely locations.

Visualizing the Learning Process

I. Prior to Posterior Transition (2D)

We watch a distribution "shift" and "narrow." We start with a vague idea (flat prior) and, as more data comes in, the distribution sharpens around the true value.

![[Bayesian 2D Posterior Update](../../assets/59_bayesian_2d.png)]

II. The Bayesian Learning Surface (3D)

A 3D visualization showing the evolution of the Posterior PDF as the number of observations (n) increases. It looks like a wave that starts wide and becomes a sharp, tall peak as certainty grows.

![[Bayesian 3D Learning Landscape](../../assets/59_bayesian_3d.png)]

Ph.D. Insights: The Subjective vs. Objective

- **Prior Power:** In small datasets, your "prior" belief has a huge impact. In large datasets, the "data" (Likelihood) overwhelms the prior, leading everyone to the same conclusion regardless of their initial belief.

- **Sequential Learning:** One of the greatest strengths of this method is that today's Posterior becomes tomorrow's Prior . Learning is a continuous loop.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import beta
import os

def run_bayesian_master():
    # --- PROFESSIONAL PATH SETUP (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Simulation: Estimating the "Fairness" of a Coin (Theta)
    # True Theta = 0.7 (It's a biased coin)
    true_theta = 0.7
    n_trials = 100
    data = np.random.binomial(1, true_theta, n_trials)

    # 2. Sequential Updating (Using Beta-Binomial Conjugate)
    # Prior: Alpha=2, Beta=2 (A weak belief that it might be fair)
    alpha_prior, beta_prior = 2, 2

    # --- ASSET 1: POSTERIOR UPDATING STEPS (59_bayesian_2d.png) ---
    plt.figure(figsize=(10, 6))
    x = np.linspace(0, 1, 100)

    # Plot Prior
    plt.plot(x, beta.pdf(x, alpha_prior, beta_prior), 'r--', label='Prior (Initial Belief)')

    # Update with different amounts of data
    for n in [5, 20, 100]:
        heads = np.sum(data[:n])
        tails = n - heads
        a_post = alpha_prior + heads
        b_post = beta_prior + tails
        plt.plot(x, beta.pdf(x, a_post, b_post), label=f'Posterior after n={n}')

    plt.axvline(true_theta, color='black', linestyle=':', label=f'True Theta={true_theta}')
    plt.title("2D Bayesian Updating: From Belief to Evidence", fontsize=14)
    plt.xlabel("Probability of Heads (Theta)"); plt.ylabel("Density")
    plt.legend(); plt.grid(alpha=0.3)

    save_path_2d = os.path.join(asset_path, '59_bayesian_2d.png')
    plt.savefig(save_path_2d, dpi=300)
    plt.show()

    # --- ASSET 2: 3D LEARNING LANDSCAPE (59_bayesian_3d.png) ---
    fig = plt.figure(figsize=(12, 9))
    ax3d = fig.add_subplot(111, projection='3d')

    N_steps = np.arange(1, n_trials + 1, 2)
    X_val = np.linspace(0, 1, 60)
    X_grid, N_grid = np.meshgrid(X_val, N_steps)
    Z_density = np.zeros_like(X_grid)

    for i, n in enumerate(N_steps):
        heads = np.sum(data[:n])
        tails = n - heads
        Z_density[i, :] = beta.pdf(X_val, alpha_prior + heads, beta_prior + tails)

    surf = ax3d.plot_surface(X_grid, N_grid, Z_density, cmap='viridis', alpha=0.8, edgecolor='none')

    ax3d.set_title("3D Bayesian Learning: The Evolution of Certainty", fontsize=14)
    ax3d.set_xlabel("Theta (Success Rate)"); ax3d.set_ylabel("Number of Trials (n)"); ax3d.set_zlabel("Posterior Density")
```

```
fig.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)

ax3d.view_init(elev=30, azim=-45)

save_path_3d = os.path.join(asset_path, '59_bayesian_3d.png')
plt.savefig(save_path_3d, dpi=300)
print(f" 3D Asset saved: {save_path_3d}")
plt.show()

if __name__ == "__main__":
    run_bayesian_master()
```

Ders: 60 Svd Analysis

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\06_linear_algebra\60_svd_a

Bölüm 1: README

\# Lesson 60: Singular Value Decomposition (SVD)

\### The Mathematical Essence

\[*SVD\]* factorizes any $m \times n$ matrix A into three distinct matrices:

$$A = U \Sigma V^T$$

\- \[*U\] (Left Singular Vectors): \[*\] Represents the "geometry" of the rows (e.g., how documents relate to topics).

\- \[*\Sigma\] (Singular Values): \[*\] A diagonal matrix containing the "strength" of each component, ordered from most important to least.

\- \[*V^T\] (Right Singular Vectors): \[*\] Represents the "geometry" of the columns (e.g., how words relate to topics).

\---

\### Real-World Applications & Examples

\#### 1. Image Compression

An image is just a large matrix of pixels. SVD allows us to keep only the top 10% of the "Singular Values" and still recognize the image perfectly. This is how we save storage space while keeping visual essence.

\#### 2. Recommender Systems (Netflix/Amazon)

Netflix uses a form of SVD (Matrix Factorization) to predict your ratings. It decomposes the "User-Movie" matrix into hidden "features" (like Genre, Actor, Vibe) to find matches between your tastes and available content.

\#### 3. Latent Semantic Analysis (LSA)

In NLP, SVD helps computers understand that "Doctor" and "Physician" are related, even if they are different words, by finding the underlying "latent" concepts in a text corpus.

\---

\### Visualizing Information Hierarchy

\#### I. Singular Value Decay (2D)

We plot the values of Σ . This "Scree Plot" shows how a few components hold almost all the power (information), while most are just noise. This is the mathematical proof that we can "compress" data.

![SVD 2D Decay](../../assets/60_svd_2d.png)

\#### II. Low-Rank Reconstruction Surface (3D)

We visualize a complex 3D mathematical surface and then "reconstruct" it using only a fraction of its SVD components. You can see the surface's core shape emerge from the approximation.

![SVD 3D Reconstruction](../../assets/60_svd_3d.png)

\---

\### Ph.D. Insights: The Perfection of Approximation

\- **Eckart-Young-Mirsky Theorem:** SVD provides the *best possible* low-rank approximation of a matrix. No other method can represent the data better using the same amount of memory.

\- **Link to PCA:** PCA is actually a specific application of SVD where the data is centered. SVD is the more general and computationally stable older brother.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_svd_master():
    # --- PROFESSIONAL PATH SETUP (Relative Path) ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Create a Complex 2D Surface (represented as a matrix)
    x = np.linspace(-3, 3, 50)
    y = np.linspace(-3, 3, 50)
    X, Y = np.meshgrid(x, y)
    Z = np.sin(X) * np.cos(Y) + np.exp(-0.1 * (X**2 + Y**2)) # Complex math surface

    # 2. Perform Singular Value Decomposition
    U, s, Vt = np.linalg.svd(Z)
    S = np.zeros(Z.shape)
    np.fill_diagonal(S, s)

    # --- ASSET 1: SINGULAR VALUE DECAY (60_svd_2d.png) ---
    plt.figure(figsize=(10, 6))
    plt.plot(s, 'o-', color='#8e44ad', lw=2)
    plt.title("2D SVD Analysis: Singular Value Spectrum (The Scree Plot)", fontsize=14)
    plt.xlabel("Component Index"); plt.ylabel("Singular Value (Magnitude)")
    plt.yscale('log') # Log scale to see the decay better
    plt.grid(alpha=0.3, which='both')

    save_path_2d = os.path.join(asset_path, '60_svd_2d.png')
    plt.savefig(save_path_2d, dpi=300)
    plt.show()

    # --- ASSET 2: LOW-RANK RECONSTRUCTION (60_svd_3d.png) ---
    fig = plt.figure(figsize=(14, 8))

    # Original Surface
    ax1 = fig.add_subplot(121, projection='3d')
    ax1.plot_surface(X, Y, Z, cmap='viridis', alpha=0.8)
    ax1.set_title("Original Complex Surface (Rank 50)")

    # Reconstruct using only top K components
    k = 3
    Z_approx = U[:, :k] @ S[:, :k] @ Vt[:, :k]

    ax2 = fig.add_subplot(122, projection='3d')
    surf = ax2.plot_surface(X, Y, Z_approx, cmap='plasma', alpha=0.8)
    ax2.set_title(f"Reconstructed Surface (Rank {k})")

    plt.suptitle(f"SVD: Capturing Essence through Low-Rank Approximation (k={k})", fontsize=15)

    save_path_3d = os.path.join(asset_path, '60_svd_3d.png')
    plt.savefig(save_path_3d, dpi=300)
    print(f"3D Asset saved: {save_path_3d}")
    plt.show()

if __name__ == "__main__":
    run_svd_master()
```

Ders: 61 Parabolic Projectiles

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\07_high_school_math\61_Parabolic Projectiles

Bölüm 1: README

\# Lesson 61: Parabolic Projectiles - The Geometry of Flight

\### The Mathematical Essence (High School Level)

A projectile's path is defined by a *Quadratic Function*. Because gravity only pulls downward, the vertical position (y) changes quadratically over time, while the horizontal position (x) changes linearly.

The Vertex Form:

$$y = a(x - h)^2 + k$$

\- *(h, k): The highest point (Vertex), where the basketball reaches its peak.

\- *Roots (x-intercepts): Where the ball starts and where it hits the ground.

\---

\### Real-World Applications & Examples

\#### 1. Sports Science (The Perfect 3-Pointer)

Basketball players and engineers use parabolas to calculate the "Entry Angle" into the hoop. A higher arc (steeper parabola) increases the effective area of the basket, making a score more likely.

\#### 2. Civil Engineering (Bridge Arches)

Look at the Golden Gate Bridge. The main cables follow a parabolic shape (technically a catenary, but near-parabolic) to distribute weight evenly to the towers, ensuring structural stability.

\#### 3. Fountain Design

Engineers at luxury hotels (like the Bellagio in Vegas) use quadratic equations to program water jets so that hundreds of streams cross each other at precise "Vertex" points without colliding.

\---

\### Visualizing the Arc

\#### I. Multiple Shot Angles (2D)

We visualize how changing the initial launch angle (30°, 45°, 60°) changes the parabola's shape. This proves that 45° is the magic number for maximum distance.

![Projectile 2D Comparison](../../assets/61\projectile_2d.png)

\#### II. The 3D Trajectory in a Crosswind

In the real world, a ball doesn't stay in a flat 2D plane. We visualize a 3D parabolic path that is "pushed" sideways by a crosswind, showing how wind affects the root of the function.

![Projectile 3D Wind Analysis](../../assets/61\projectile_3d.png)

\---

\### Teacher's Corner: Why Learn This?

\- **Predictive Power:** If you know the equation of your parabola, you can predict exactly where an object will be at any second.

\- **Optimization:** We use the vertex formula ($x = -b/2a$) to find the maximum height without even needing a calculator.

Bölüm 2: Python Kodu

```

import numpy as np
import matplotlib.pyplot as plt
import os

def run_projectile_master():
    # --- PROFESSIONAL PATH SETUP ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Constants
    g = 9.81 # Gravity
    v0 = 20 # Initial velocity (m/s)
    t_flight = np.linspace(0, 3, 100)

    # --- ASSET 1: 2D MULTI-ANGLE COMPARISON (61_projectile_2d.png) ---
    plt.figure(figsize=(10, 6))
    angles = [30, 45, 60, 75]
    colors = ['#e74c3c', '#2ecc71', '#3498db', '#9b59b6']

    for angle, col in zip(angles, colors):
        rad = np.radians(angle)
        x = v0 * np.cos(rad) * t_flight
        y = v0 * np.sin(rad) * t_flight - 0.5 * g * t_flight**2

        # Only plot points above ground
        x, y = x[y >= 0], y[y >= 0]
        plt.plot(x, y, label=f'Angle {angle}°', lw=2, color=col)

    plt.title("2D Projectile Motion: Finding the Optimal Launch Angle", fontsize=14)
    plt.xlabel("Distance (m)"); plt.ylabel("Height (m)")
    plt.axhline(0, color='black', lw=1); plt.grid(alpha=0.3)
    plt.legend()

    plt.savefig(os.path.join(asset_path, '61_projectile_2d.png'), dpi=300)
    plt.show()

    # --- ASSET 2: 3D TRAJECTORY WITH CROSSWIND (61_projectile_3d.png) ---
    fig = plt.figure(figsize=(12, 9))
    ax3d = fig.add_subplot(111, projection='3d')

    # Launch at 45 degrees
    rad = np.radians(45)
    wind_speed = 2.5 # m/s pushing in Z direction

    # X = Horizontal forward, Y = Vertical, Z = Horizontal side (wind)
    X_fwd = v0 * np.cos(rad) * t_flight
    Y_alt = v0 * np.sin(rad) * t_flight - 0.5 * g * t_flight**2
    Z_side = 0.5 * wind_speed * t_flight**2 # Wind acceleration effect

    # Filter ground
    mask = Y_alt >= 0
    ax3d.plot(X_fwd[mask], Z_side[mask], Y_alt[mask], 'r-', lw=4, label='Path with Wind')

    # Draw shadows on the ground for depth
    ax3d.plot(X_fwd[mask], Z_side[mask], 0, 'k--', alpha=0.3, label='Ground Shadow')

    ax3d.set_title("3D Projectile Lab: Influence of Crosswind on Quadratic Flight", fontsize=14)
    ax3d.set_xlabel("Forward Distance (X)"); ax3d.set_ylabel("Wind Drift (Z)"); ax3d.set_zlabel("Altitude (Y)")
    ax3d.legend()
    ax3d.view_init(elev=20, azim=-60)

    plt.savefig(os.path.join(asset_path, '61_projectile_3d.png'), dpi=300)

```

```
print(f" 3D Asset saved: {asset_path}/61_projectile_3d.png")
plt.show()

if __name__ == "__main__":
    run_projectile_master()
```

Ders: 62 Trigonometric Waves

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\07_high_school_math\62_trigonometric_waves

Bölüm 1: README

\# Lesson 62: Trigonometric Waves - The Pulse of Technology

\### The Mathematical Essence (High School Level)

Trigonometry is not just about triangles; it's about Periodic Motion . The Sine and Cosine functions describe anything that repeats over time.

$\text{The Wave Equation:}$

$$y = A \cdot \sin(B(x - C)) + D$$

- A (Amplitude): The loudness of a sound or the brightness of a light.

- Period ($2\pi/B$): The pitch of a musical note or the frequency of a Wi-Fi signal.

- C (Phase Shift): The starting position of the wave.

\---

\### Real-World Applications & Examples

\#### 1. Music & Sound Engineering

Every sound you hear is a combination of sine waves. A "Pure Tone" is a single frequency. When you adjust the "Bass" or "Treble" on your speakers, you are mathematically altering the Amplitude of specific trigonometric waves.

\#### 2. Telecommunications (Wi-Fi & 5G)

Information is sent through the air by modulating waves. Your phone translates binary (0s and 1s) into shifts in a wave's phase or frequency. Without trigonometry, there is no internet.

3. Medical Imaging (EKG/ECG)

The human heart beats in a periodic rhythm. Doctors look at the trigonometric patterns of your heart's electrical activity to ensure the "period" and "amplitude" are within a healthy range.

Visualizing the Pulse

I. Sound Interference (2D)

We visualize what happens when two different musical notes (Sine waves) play at the same time. This explains the "Beat" phenomenon and how noise-canceling headphones work (Phase cancellation).

![[Trig 2D Wave Interference](../../assets/62_trig_2d.png)]

II. The 3D Radio Wave Propagation

We visualize a wave traveling through space and time. This shows how a signal loses strength (Amplitude) as it moves away from a Wi-Fi router.

![[Trig 3D Signal Decay](../../assets/62_trig_3d.png)]

Teacher's Corner: Why Learn This?

- **Periodic Thinking:** Many things in life (seasons, tides, heartbeats) are not linear; they are cyclical. Trigonometry gives us the language to describe "cycles."

- **From Circles to Waves:** It bridges the gap between geometry (circles) and physics (waves).

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_trig_master():
    # --- PROFESSIONAL PATH SETUP ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Generate Wave Data
    x = np.linspace(0, 4 * np.pi, 500)
    wave1 = np.sin(x)      # Frequency 1
    wave2 = np.sin(2.5 * x) # Frequency 2.5
    interference = wave1 + wave2 # Combined wave

    # --- ASSET 1: SOUND INTERFERENCE WITH FORMULAS (62_trig_2d.png) ---
    plt.figure(figsize=(12, 7))
    plt.plot(x, wave1, label=r'$y_1 = \sin(x)$ (Base Tone)', alpha=0.4, linestyle='--')
    plt.plot(x, wave2, label=r'$y_2 = \sin(2.5x)$ (High Tone)', alpha=0.4, linestyle='--')
    plt.plot(x, interference, label=r'$Y = y_1 + y_2$ (Resultant)', color='red', lw=2.5)

    # Grafik üzerine açıklayıcı metin ekleme
    plt.text(0.5, 1.8, r"Interference: $y = \sin(ax) + \sin(bx)$", fontsize=12,
             bbox=dict(facecolor='white', alpha=0.8))
    plt.text(7, -1.8, "Constructive & Destructive Interference", color='darkred', weight='bold')

    plt.title("2D Trigonometry: Wave Superposition in Sound Engineering", fontsize=14)
    plt.xlabel("Phase (Radians)"); plt.ylabel("Amplitude (A)")
    plt.grid(alpha=0.3); plt.legend(loc='upper right')

    save_path_2d = os.path.join(asset_path, '62_trig_2d.png')
    plt.savefig(save_path_2d, dpi=300)
    print(f" 2D Asset saved: {save_path_2d}")
    plt.show()

    # --- ASSET 2: 3D SIGNAL DECAY WITH FORMULAS (62_trig_3d.png) ---
    fig = plt.figure(figsize=(12, 9))
    ax3d = fig.add_subplot(111, projection='3d')

    t_vals = np.linspace(0, 10, 100)
    x_vals = np.linspace(0, 10, 100)
    T, X = np.meshgrid(t_vals, x_vals)

    # Wave Equation with Exponential Decay
    # Z = A * sin(X - T) * e^(-0.2X)
    Z = np.sin(X - T) * np.exp(-0.2 * X)

    surf = ax3d.plot_surface(X, T, Z, cmap='plasma', alpha=0.75, edgecolor='none')

    # 3D Grafik üzerine formül ekleme
    ax3d.text(0, 0, 1.5, r"$f(x,t) = \sin(kx - \omega t) \cdot e^{-\alpha x}$",
             fontsize=14, color='darkblue', weight='bold')

    ax3d.set_title("3D Wi-Fi Signal Propagation: Amplitude Decay Over Distance", fontsize=14)
    ax3d.set_xlabel("Distance (x)"); ax3d.set_ylabel("Time (t)"); ax3d.set_zlabel("Signal (A)")
    fig.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)

    ax3d.view_init(elev=25, azim=-45)

    save_path_3d = os.path.join(asset_path, '62_trig_3d.png')
    plt.savefig(save_path_3d, dpi=300)
    print(f" 3D Asset saved: {save_path_3d}")
```

```
plt.show()
```

```
if __name__ == "__main__":  
    run_trig_master()
```

Ders: 63 Exponential Dynamics

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\07_high_school_math\63_exponential_dynamics

Bölüm 1: README

\# Lesson 63: Exponential Growth & Decay

\### The Mathematical Essence (High School Level)

Exponential functions describe processes where the rate of change is proportional to the current value. As the value gets bigger, it grows even faster.

The General Formula:

$$f(t) = a \cdot e^{rt}$$

\- a : Initial value (Starting point).

\- r : Growth rate (if positive) or Decay rate (if negative).

\- e : Euler's constant (≈ 2.718), the natural base of growth.

\---

\### Real-World Applications & Examples

\#### 1. Social Media Virality

If each person shares a video with 2 friends every hour, the number of views doesn't just increase; it explodes. This is why "going viral" happens so suddenly—the curve turns almost vertical.

\#### 2. Finance (The 8th Wonder: Compound Interest)

Saving money with compound interest is exponential growth. Your interest earns interest. Over 30 years, this "r" rate can turn small savings into a fortune.

3. Carbon Dating & Medicine (Half-Life)

Radioactive elements and medicines in your bloodstream follow e^{-rt} . Doctors use the "half-life" formula (e^{-rt}) to calculate exactly when you need your next dose of medicine.

Visualizing the Explosion and the Fade

I. Viral Growth vs. Linear Growth (2D)

A comparison showing how a linear process (adding 10 every day) is quickly overtaken by an exponential process (growing by 10% every day).

![Exponential 2D Growth](../../assets/63_exp_2d.png)

II. The Surface of Interest Rates (3D)

We visualize how your final balance (Z) changes based on both the t (Time) (X) and the r (Interest Rate) (Y). This 3D slope shows why even a 1% difference in rates matters immensely over time.

![Exponential 3D Interest Surface](../../assets/63_exp_3d.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_exponential_master():
    asset_path = r'../../assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # 1. 2D Data: Linear vs Exponential
    t = np.linspace(0, 10, 100)
    linear = 20 * t + 50
    exponential = 50 * np.exp(0.4 * t)

    # --- ASSET 1: GROWTH COMPARISON (63_exp_2d.png) ---
    plt.figure(figsize=(10, 6))
    plt.plot(t, linear, 'b--', label=r'Linear: $50 + 20t$', lw=2)
    plt.plot(t, exponential, 'r-', label=r'Exponential: $50 \cdot e^{0.4t}$', lw=3)

    plt.text(1, 1500, "Exponential Overtakes Linear!", color='darkred', weight='bold')
    plt.title("2D Analysis: Linear vs Exponential Growth Dynamics", fontsize=14)
    plt.xlabel("Time (Steps)"); plt.ylabel("Value / Population")
    plt.grid(alpha=0.3); plt.legend()

    plt.savefig(os.path.join(asset_path, '63_exp_2d.png'), dpi=300)
    plt.show()

    # --- ASSET 2: 3D COMPOUND INTEREST (63_exp_3d.png) ---
    fig = plt.figure(figsize=(12, 9))
    ax3d = fig.add_subplot(111, projection='3d')

    time = np.linspace(0, 30, 50) # 0 to 30 years
    rates = np.linspace(0.01, 0.15, 50) # 1% to 15% interest
    T, R = np.meshgrid(time, rates)

    # Final Balance = Principal * e^(rate * time)
    Principal = 1000
    Balance = Principal * np.exp(R * T)

    surf = ax3d.plot_surface(T, R, Balance, cmap='terrain', alpha=0.8)

    ax3d.text2D(0.05, 0.85, r"$A = P \cdot e^{rt}$", transform=ax3d.transAxes,
               fontsize=16, bbox=dict(facecolor='white', alpha=0.7))

    ax3d.set_title("3D Wealth Accumulation: Time vs Interest Rate", fontsize=14)
    ax3d.set_xlabel("Years (t)"); ax3d.set_ylabel("Annual Rate (r)"); ax3d.set_zlabel("Balance ($)")
    fig.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)

    ax3d.view_init(elev=20, azim=-120)
    plt.savefig(os.path.join(asset_path, '63_exp_3d.png'), dpi=300)
    plt.show()

if __name__ == "__main__":
    run_exponential_master()
```

Ders: 64 Conic Sections

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\07_high_school_math\64_conic_sections

Bölüm 1: README

\# Lesson 64: Ellipses & Orbits - Conic Sections in the Sky

\### The Mathematical Essence (High School Level)

An \(*\text{Ellipse}\(*\) is the set of all points where the sum of distances from two fixed points (foci) is constant. In space, the Sun sits at one of these "foci."

\(*\text{The Standard Equation:}\(*\)

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

\- \(*\text{\\$a\\$}\(*\) Semi-major axis (the "long" radius).

\- \(*\text{\\$b\\$}\(*\) Semi-minor axis (the "short" radius).

\- \(*\text{\\$c\\$}\(*\) Distance to the focus ($\sqrt{a^2 - b^2}$).

\- \(*\text{Eccentricity (\\$e = c/a\\$)}\(*\) How "stretched" the ellipse is. If $e=0$, it's a perfect circle!

\---

\### Real-World Applications & Examples

\#### 1. Planetary Motion (Kepler's First Law)

Contrary to what many think, planets don't move in circles. They follow \(*\text{Elliptical Orbits}\(*\). The Earth is actually closer to the Sun in January (Perihelion) than in July!

\#### 2. Medical Tech (Lithotripsy)

Doctors use the reflective property of ellipses to break kidney stones. An ultrasound source is placed at one focus, and the kidney stone at the other. Every wave emitted from the first focus hits the stone at the second focus with perfect precision.

3. Architecture (Whispering Galleries)

In certain elliptical rooms (like St. Paul's Cathedral), if you whisper at one focus, someone standing at the other focus can hear you perfectly, while people in the middle hear nothing!

Visualizing the Celestial Path

I. Orbit Eccentricity (2D)

We visualize how different "Eccentricity" values change a planet's path. We compare a nearly circular orbit (Earth) with a highly stretched one (Halley's Comet).

![Ellipse 2D Comparison](../../assets/64\ellipse_2d.png)

II. The Gravity Well (3D)

We visualize space-time as a 3D fabric. A massive object (Sun) creates a "well," and the elliptical path of the planet is shown as it rolls along the curved walls of this potential surface.

![Ellipse 3D Gravity Well](../../assets/64\ellipse_3d.png)

Teacher's Corner: Why Learn This?

- Conic Sections: Understanding that a simple slice through a cone creates circles, ellipses, parabolas, and hyperbolas is the ultimate "Aha!" moment in geometry.

- Precision: Ellipses allow us to calculate satellite paths so your GPS knows exactly where you are.

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_ellipse_master():
    # --- PROFESSIONAL PATH SETUP ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # 1. Generate Ellipse Data
    theta = np.linspace(0, 2*np.pi, 200)

    def get_ellipse(a, b):
        x = a * np.cos(theta)
        y = b * np.sin(theta)
        return x, y

    # --- ASSET 1: ECCENTRICITY COMPARISON (64_ellipse_2d.png) ---
    plt.figure(figsize=(10, 8))

    # Earth-like (Low eccentricity)
    x1, y1 = get_ellipse(5, 4.8)
    plt.plot(x1, y1, label=r"Earth-like (e $\approx$ 0.28)", color='blue', lw=2)

    # Comet-like (High eccentricity)
    x2, y2 = get_ellipse(8, 3)
    plt.plot(x2, y2, label=r"Comet-like (e $\approx$ 0.92)", color='orange', lw=2)

    # Plot Focus (The Sun)
    plt.scatter([2], [0], color='red', s=100, label="Sun (Focus)")

    plt.text(-7, 4, r"Equation: $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$", fontsize=12, bbox=dict(facecolor='white', alpha=0.7))
    plt.title("2D Elliptical Orbits: From Circles to Comets", fontsize=14)
    plt.axis('equal'); plt.grid(alpha=0.3); plt.legend()

    plt.savefig(os.path.join(asset_path, '64_ellipse_2d.png'), dpi=300)
    plt.show()

    # --- ASSET 2: GRAVITY WELL SURFACE (64_ellipse_3d.png) ---
    fig = plt.figure(figsize=(12, 9))
    ax3d = fig.add_subplot(111, projection='3d')

    # Create a Gravity Well: Potential V = -1 / sqrt(x^2 + y^2)
    x_range = np.linspace(-5, 5, 60)
    y_range = np.linspace(-5, 5, 60)
    X, Y = np.meshgrid(x_range, y_range)
    R = np.sqrt(X**2 + Y**2) + 0.5 # Add 0.5 to avoid singularity at center
    Z = -5 / R

    surf = ax3d.plot_surface(X, Y, Z, cmap='magma', alpha=0.6, edgecolor='none')

    # Plot an elliptical path on this surface
    xe, ye = get_ellipse(3.5, 2.5)
    ze = -5 / (np.sqrt(xe**2 + ye**2) + 0.5)
    ax3d.plot(xe, ye, ze, color='cyan', lw=4, label='Planetary Path')

    ax3d.text2D(0.05, 0.9, "3D Gravity Well Projection", transform=ax3d.transAxes, fontsize=14, weight='bold')
    ax3d.set_title("Space-Time Curvature & Elliptical Motion", fontsize=14)
    ax3d.set_zlim(-10, 0)
    ax3d.axis('off') # Hide axes for a more 'space' feel

    ax3d.view_init(elev=40, azim=30)
    plt.savefig(os.path.join(asset_path, '64_ellipse_3d.png'), dpi=300)
```

```
plt.show()
```

```
if __name__ == "__main__":  
    run_ellipse_master()
```

Ders: 65 Geometric Series

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\07_high_school_math\65_g

Bölüm 1: README

\# Lesson 65: Geometric Series - The Wealth Accumulator

\\### The Mathematical Essence (High School Level)

A **Geometric Series** is the sum of the terms of a geometric sequence, where each term is found by multiplying the previous one by a fixed, non-zero number called the common ratio (\$r\$).

The Sum Formula:

$$S_n = \frac{a_1(1 - r^n)}{1 - r}$$

\\- **a_1** Starting amount (Initial deposit).

\\- **r** Common ratio (Interest rate + 1).

\\- **n** Number of steps (Years or Months).

\\---

\\### Real-World Applications & Examples

\\#### 1. Compound Interest (Wealth Growth)

When you save money in a high-yield account, you aren't just adding money; you are multiplying it. Every year, your interest is added to your principal, and next year you earn interest on that interest. This is a **Geometric Series** in action.

\\#### 2. Digital Marketing (Viral Marketing Loops)

If 1 customer brings in 1.1 customers (a ratio of \$r=1.1\$), your user base grows as a series. Companies like Uber and Dropbox used this math to grow from zero to millions using "Referral Codes."

3. Infinite Series in Physics (Zeno's Paradoxes)

Can you ever reach a wall if you always walk half the remaining distance?

$$\sum_{n=1}^{\infty} \frac{1}{2^n} = 1$$

Mathematics proves that even with infinite steps, the total sum can be a finite, predictable number!

Visualizing the Accumulation

I. Linear vs. Geometric Accumulation (2D)

A stark comparison showing the "Total Sum" (S_n). While linear saving looks like a straight staircase, geometric growth looks like a rocket taking off after a slow start.

![Geometric 2D Sum Comparison](../../assets/65_series_2d.png)

II. The "Snowball Effect" Surface (3D)

We visualize the "Total Wealth" (Z) as a function of the "Interest Rate" (X) and "Time" (Y). The steepness of the surface shows why starting early is the most important factor in finance.

![Geometric 3D Wealth Surface](../../assets/65_series_3d.png)

Teacher's Corner: Why Learn This?

- **The 8th Wonder:** Einstein allegedly called compound interest the 8th wonder of the world. Understanding r^n is the difference between being a borrower or a lender.

- **Decision Making:** It helps students understand why small, consistent improvements (1% better every day) lead to massive changes over time ($1.01^{365} \approx 37.8$).

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os
import warnings

# RuntimeWarning (0'a bölme) mesajlarını terminalde görmemek için (kod içinde zaten hallediyoruz)
warnings.filterwarnings("ignore", category=RuntimeWarning)

def run_series_master():
    # --- PROFESSIONAL PATH SETUP ---
    asset_path = r'../../assets'
    if not os.path.exists(asset_path):
        os.makedirs(asset_path)

    # 1. Generate Series Data
    n_terms = 20
    a1 = 100 # Başlangıç miktarı
    r_const = 1.2 # %20 artış oranı

    steps = np.arange(1, n_terms + 1)

    # Geometrik Terimler ve Toplam
    terms_geo = a1 * (r_const**(steps - 1))
    sums_geo = a1 * (1 - r_const**steps) / (1 - r_const)

    # Lineer Karşılaştırma (Her adımda sadece 100 ekleme)
    sums_lin = a1 * steps

    # --- ASSET 1: 2D ACCUMULATION COMPARISON (65_series_2d.png) ---
    plt.figure(figsize=(10, 6))

    # RAW STRING (r"") kullanarak SyntaxWarning'i engelliyoruz
    plt.bar(steps, sums_lin, alpha=0.5, label=r'Linear Saving: $a \cdot n$', color='gray')
    plt.plot(steps, sums_geo, 'o-', color='#27ae60', lw=3, label=r'Geometric Sum: $S_n = \frac{a_1(1-r^n)}{1-r}$')

    plt.text(2, sums_geo[-1]*0.7, "The Power of Compounding!", fontsize=12, weight='bold', color='#1e8449')
    plt.title("2D Analysis: Linear vs. Geometric Wealth Accumulation", fontsize=14)
    plt.xlabel("Time Periods (n)"); plt.ylabel("Total Accumulated Value ($)")
    plt.grid(alpha=0.3)
    plt.legend()

    save_path_2d = os.path.join(asset_path, '65_series_2d.png')
    plt.savefig(save_path_2d, dpi=300)
    print(f" 2D Asset saved: {save_path_2d}")
    plt.show()

    # --- ASSET 2: 3D WEALTH SURFACE (65_series_3d.png) ---
    fig = plt.figure(figsize=(12, 9))
    ax3d = fig.add_subplot(111, projection='3d')

    # R (Oran) ve N (Zaman) düzlemi
    r_vals = np.linspace(1.01, 1.3, 50) # 1.0 yerine 1.01'den başlatarak bölme hatasını kökten çözüyoruz
    n_vals = np.linspace(1, 20, 50)
    R, N = np.meshgrid(r_vals, n_vals)

    # Geometrik Toplam Fonksiyonu
    Z_wealth = a1 * (1 - R**N) / (1 - R)

    surf = ax3d.plot_surface(R, N, Z_wealth, cmap='viridis', alpha=0.8, edgecolor='none')

    # Formül Etiketleri (Raw String)
    ax3d.text2D(0.05, 0.9, r"Wealth Function: $W(r, n) = \frac{a_1(1-r^n)}{1-r}$",
        transform=ax3d.transAxes, fontsize=12, fontweight='bold',
```

```
bbox=dict(facecolor='white', alpha=0.7))

ax3d.set_title("3D Wealth Surface: Rate and Time Interaction", fontsize=14)
ax3d.set_xlabel("Growth Ratio (r)"); ax3d.set_ylabel("Time Steps (n)"); ax3d.set_zlabel("Total Wealth ($)")
fig.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)

ax3d.view_init(elev=30, azim=-140)

save_path_3d = os.path.join(asset_path, '65_series_3d.png')
plt.savefig(save_path_3d, dpi=300)
print(f" 3D Asset saved: {save_path_3d}")
plt.show()

if __name__ == "__main__":
    run_series_master()
```

Ders: 66 Optimization Derivatives

Konum: C:\Users\ozhan\Desktop\The-Mathematical-Lab-Bench-main\The-Mathematical-Lab-Bench-main\modules\07_high_school_math\66_optimization

Bölüm 1: README

\# Lesson 66: Optimization with Derivatives

\### The Mathematical Essence (High School Level)

Optimization is the process of finding the maximum or minimum value of a function. The derivative ($f'(x)$) tells us the slope of the function. When the slope is 0 , we have reached a peak (maximum) or a valley (minimum).

The Process:

- 1\ Define your Cost or Profit function $f(x)$.
- 2\ Take the derivative: $f'(x)$.
- 3\ Set $f'(x) = 0$ and solve for x (The Critical Points).

\---

\### Real-World Applications & Examples

\#### 1. Minimum Material Packaging

Imagine you are a soda company. You want to make a can that holds exactly 330ml but uses the minimum amount of aluminum to save costs. Derivatives help you find the exact radius and height to minimize surface area.

\#### 2. Maximum Profit in Business

If you lower the price of a game, you sell more copies. But if the price is too low, you lose money. Derivatives help companies find the "Sweet Spot" price that maximizes total profit.

\#### 3. Fuel Efficiency in Aviation

Pilots and flight computers calculate the optimal cruising altitude where fuel consumption is minimized. This "valley" in the fuel-consumption curve is found using calculus.

\---

\### Visualizing the Sweet Spot

\#### I. The Slope Hunter (2D)

We visualize a profit function and its derivative. You will see that exactly where the derivative crosses the X-axis (zero), the profit function reaches its highest point.

!\[Optimization 2D Peak](../../assets/66_opt_2d.png)

\#### II. The Efficiency Surface (3D)

We visualize how profit changes based on two variables: *Production Cost* (\$X\$) and *Selling Price* (\$Y\$). The "Summit" of this 3D mountain is where the business should operate.

!\[Optimization 3D Surface](../../assets/66_opt_3d.png)

Bölüm 2: Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
import os

def run_optimization_master():
    asset_path = r'../../assets'
    if not os.path.exists(asset_path): os.makedirs(asset_path)

    # 1. Define Profit Function:  $P(x) = -2x^2 + 80x - 200$ 
    # Derivative:  $P'(x) = -4x + 80$ 
    # Peak at  $x = 20$ 
    x = np.linspace(0, 40, 100)
    profit = -2*x**2 + 80*x - 200
    derivative = -4*x + 80

    # --- ASSET 1: PROFIT VS DERIVATIVE (66_opt_2d.png) ---
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10), sharex=True)

    ax1.plot(x, profit, color='#2c3e50', lw=3, label=r'Profit  $P(x)$ ')
    ax1.scatter([20], [600], color='red', s=100, zorder=5)
    ax1.annotate('Maximum Profit!', xy=(20, 600), xytext=(25, 550),
                arrowprops=dict(facecolor='black', shrink=0.05))
    ax1.set_ylabel("Profit ($)"); ax1.legend(); ax1.grid(alpha=0.3)

    ax2.plot(x, derivative, color='#e67e22', lw=2, label=r'Derivative  $P'(x)$  (Slope)')
    ax2.axhline(0, color='black', lw=1)
    ax2.scatter([20], [0], color='red', s=100)
    ax2.set_ylabel("Slope Rate"); ax2.set_xlabel("Units Sold / Price"); ax2.legend(); ax2.grid(alpha=0.3)

    plt.suptitle("2D Optimization: Finding the Peak where Slope = 0", fontsize=14)
    plt.savefig(os.path.join(asset_path, '66_opt_2d.png'), dpi=300)
    plt.show()

    # --- ASSET 2: PROFIT LANDSCAPE (66_opt_3d.png) ---
    fig3d = plt.figure(figsize=(12, 9))
    ax3d = fig3d.add_subplot(111, projection='3d')

    x_prod = np.linspace(5, 35, 50) # Production Volume
    y_price = np.linspace(10, 100, 50) # Price Point
    X, Y = np.meshgrid(x_prod, y_price)

    # A hypothetical 3D Profit Surface
    Z_profit = -(X-20)**2 - 0.5*(Y-50)**2 + 5000

    surf = ax3d.plot_surface(X, Y, Z_profit, cmap='viridis', alpha=0.8)

    ax3d.set_title("3D Profit Landscape: Optimizing Volume and Price", fontsize=14)
    ax3d.set_xlabel("Production Vol (x)"); ax3d.set_ylabel("Price (y)"); ax3d.set_zlabel("Profit ($)")
    fig3d.colorbar(surf, ax=ax3d, shrink=0.5, aspect=10)

    ax3d.view_init(elev=25, azim=135)
    plt.savefig(os.path.join(asset_path, '66_opt_3d.png'), dpi=300)
    plt.show()

if __name__ == "__main__":
    run_optimization_master()
```